中山大学专业硕士学位论文

基于并行计算的金融大规模数据异常检测系统研究与实现

专　　业：　工程硕士 人工智能（高性能计算方向）

学位申请人：　黄裕文

指 导 教 师：　黄聃

论文答辩委员会主席：

论文答辩委员会成员：

2026 年 4 月

论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究作出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

学位论文作者签名：

日期：　　年　　月　　日

学位论文使用授权声明

本人完全了解中山大学有关保留、使用学位论文的规定，即：学校有权保留学位论文并向国家主管部门或其指定机构送交论文的电子版和纸质版；有权将学位论文用于非赢利目的的少量复制并允许论文进入学校图书馆、院系资料室被查阅；有权将学位论文的内容编入有关数据库进行检索；可以采用复印、缩印或其他方法保存学位论文；可以为建立了馆际合作关系的兄弟高校用户提供文献传递服务和交换服务。

保密论文保密期满后，适用本声明。

学位论文作者签名：　　导师签名：

日期：　　年　　月　　日　　日期：　　年　　月　　日

摘　　要

面向金融风控、反欺诈、反洗钱与交易监测等场景，异常检测系统不仅需要具备对少量高风险样本的识别能力，还需要在大规模数据条件下满足严格的吞吐与时延要求。随着金融交易规模持续增长，异常检测系统面临的主要矛盾已经从单一模型精度不足，逐渐演化为异常评分复杂度高、候选召回开销大、CPU 与 GPU 之间数据传输频繁、多 GPU 扩展效率受限等系统性问题[1-2]。传统基于 CPU 的异常检测系统在高维和大规模数据条件下难以同时兼顾检测效果与近实时处理能力；而仅对单一检测算法或单一向量检索模块进行 GPU 加速，也难以从系统层面打通候选召回、异常评分与并行扩展之间的协同关系[2-3]。

针对上述问题，本文围绕金融大规模数据异常检测任务，提出并实现了一种基于并行计算的 **PyTOD–FlashANNS 融合异常检测系统**。该系统以 PyTOD 作为异常评分主干，以 FlashANNS 作为高吞吐候选召回模块，在统一的数据流、算子流与调度流框架下构建候选召回与异常评分协同执行链路[2-3]。

在单 GPU 场景下，本文提出了 **GPU 主导的端到端执行链路**。在 FlashANNS 提供的异步 I/O—计算流水线基础上，将查询特征、候选检索结果与异常评分中间张量尽量保持在 GPU 侧，仅在必要时通过 pinned memory 等机制完成主机侧交互，从而同时减少 SSD-I/O 与 GPU 计算之间的串行等待，以及 CPU 与 GPU 之间的不必要数据搬运[3-5]。针对原始系统中存在的中间结果序列化与主机侧频繁参与候选组织的问题，本文进一步引入 RAPIDS 对特征处理与表格型中间结果组织路径进行 GPU 化重构，并结合 FAISS-GPU 对候选检索后端进行工程化增强[6-7]。

在多 GPU 场景下，本文设计并实现了基于**数据并行与索引复制**的扩展方案。系统优先采用 replicated 架构，使每张 GPU 独立完成本地候选召回与异常评分，CPU 主要承担控制面调度与最终小规模结果汇总职责[8-9]。当索引容量或业务约束导致必须跨卡归并时，本文将归并逻辑尽量下沉到 GPU 侧，并通过 NCCL 实现设备间通信，同时结合拓扑感知的 I/O lane 绑定与准入控制，降低共享存储路径对多卡扩展效率的影响[10-11]。

在实验设计方面，本文采用 1M 规模真实或半真实数据集对方法有效性进行验证，并构建 10M 与 50M 等规模的高保真合成扩展数据集对系统吞吐、时延与扩展能力进行测试[12-14]。实验结果表明，本文所提出的 PyTOD–FlashANNS 融合系统能够在保证检测效果可接受的前提下显著提升系统整体吞吐率，并有效降低 CPU–GPU 数据往返与阶段同步开销；在多 GPU 场景中，数据并行与索引

复制策略能够较好地支撑系统扩展[3,8]。

本文的研究重点不在于提出新的异常检测理论模型，而在于围绕金融大规模数据异常检测任务，构建一条覆盖候选召回、异常评分、数据路径优化与多GPU扩展的完整系统实现路线。研究结果表明，PyTOD与FlashANNS的融合能够有效打通异常检测中的计算瓶颈、存储瓶颈与并行扩展瓶颈，为金融场景下高吞吐、近实时异常检测系统的工程实现提供可行方案[2-3]。

关键词：异常检测，PyTOD，FlashANNS，RAPIDS，FAISS-GPU，GPU常驻执行，多GPU扩展，近似最近邻检索，金融风控

ABSTRACT

In financial risk control scenarios such as fraud detection and anti-money laundering, outlier detection systems must not only identify a small number of high-risk samples, but also satisfy stringent throughput and latency requirements under large-scale data. As transaction volumes grow, the major challenges have shifted from purely model accuracy to system-level constraints, including expensive scoring, costly candidate retrieval, frequent CPU–GPU transfers, and limited scaling efficiency on multiple GPUs.

To address these challenges, this thesis proposes and implements a parallel-computing-based *PyTOD–FlashANNS* integrated outlier detection system. PyTOD serves as the GPU scoring backbone, while FlashANNS provides high-throughput candidate retrieval. The two components are orchestrated in a unified pipeline so that the system first retrieves a bounded candidate set from massive data, and then performs tensorized scoring on the candidates to improve end-to-end efficiency while maintaining acceptable detection effectiveness.

For the single-GPU setting, we design a GPU-dominant end-to-end execution path. Query features, retrieved candidates, and intermediate scoring tensors are kept on the device whenever possible, and host interaction is restricted to necessary boundaries via pinned memory, thereby reducing both SSD I/O stalls and avoidable CPU–GPU data movement. We further migrate selected feature processing and tabular intermediate transformations to RAPIDS, and enhance the retrieval backend with FAISS-GPU to improve device-side data processing efficiency and retrieval throughput.

For the multi-GPU setting, we implement a data-parallel replicated architecture in which each GPU independently completes local candidate retrieval and scoring, while the CPU primarily acts as a control-plane scheduler for load balancing and small-result collection. When cross-device merging is unavoidable due to index capacity constraints, the merge logic is pushed down to GPUs with NCCL communication, and topology-aware I/O lane binding and admission control are employed to mitigate contention on shared storage paths.

Extensive experiments on real/semi-real datasets at the 1M scale and high-fidelity synthetic datasets at the 10M and 100M scales show that the proposed system improves end-to-end throughput while reducing CPU–GPU transfers and synchronization over-

head, and demonstrates stable throughput scaling under multi-GPU deployment.

Keywords: Outlier Detection, PyTOD, FlashANNS, RAPIDS, FAISS-GPU, GPU-Resident Execution, Multi-GPU Scaling, Approximate Nearest Neighbor Search, Financial Risk Control

目　　录

第 1 章　绪论 1
1.1　研究背景与意义 1
1.2　研究问题与研究对象 3
1.3　核心问题分析 5
1.4　研究目标与技术路线 6
1.5　本文主要工作与创新点 7
1.6　论文组织结构 7
第 2 章　相关研究工作与关键技术基础 9
2.1　异常检测方法研究现状 9
2.2　TOD/PyTOD 的方法学基础 11
2.3　FlashANNS 与候选召回系统 14
2.4　RAPIDS、FAISS-GPU 与工程可部署性 16
2.5　多 GPU 调度与在线推理启示 19
2.6　本章小结 21
第 3 章　PyTOD-FlashANNS 融合检测模型与数据准备 23
3.1　金融异常检测任务建模 23
3.2　实验数据集选取与构建 24
3.3　数据预处理与特征构造 26
3.4　PyTOD 异常评分主干设计 28
3.5　FlashANNS 候选召回模块设计 29
3.6　PyTOD–FlashANNS 融合执行流程 31
3.7　基础实验与初步验证 32
3.8　本章小结 34
第 4 章　单卡场景下的系统设计与数据路径优化 35
4.1　单卡系统需求与总体架构 35
4.2　单卡场景下的瓶颈分析 37
4.3　GPU 主导的端到端执行链路设计 39
4.4　基于 FlashANNS 的异步 I/O—计算流水线融合 43
4.5　数据处理与计算后端优化 45

4.6 候选检索后端优化 47
4.7 单卡执行链路的工程增强 48
4.8 单卡实验与性能分析 50
4.9 本章小结 53

第 5 章 多 GPU 扩展与聚合优化 55
5.1 多 GPU 扩展需求分析 55
5.2 多 GPU 扩展策略设计原则 57
5.3 数据并行与索引复制的多 GPU 执行架构 59
5.4 CPU 控制面调度与负载均衡 61
5.5 跨卡归并场景下的 GPU 侧归并与 NCCL 通信 63
5.6 拓扑感知 I/O lane 绑定与准入控制 64
5.7 多 GPU 工程增强 66
5.8 多 GPU 扩展实验与结果分析 67
5.9 本章小结 70

第 6 章 综合实验设计、结果分析与讨论 71
6.1 实验环境与软硬件配置 71
6.2 数据集与任务设置 73
6.3 评价指标与比较原则 75
6.4 核心结果展示 77
6.5 结果讨论 79
6.6 局限性分析 80
6.7 本章小结 81

第 7 章 全文总结与展望 82
7.1 全文总结 82
7.2 未来工作展望 84

参考文献 86

附录 A 实验脚本与日志模板 88
A.1 待补充内容 88

附录 B 图表补充说明 89
B.1 待补充内容 89

附录 C 缩写、变量与配置对照 90

致谢 91

符号和缩略语说明

缩写	含义
ANN	Approximate Nearest Neighbor，近似最近邻
AUC	Area Under Curve，曲线下面积
BTO	Basic Tensor Operator，基础张量算子
FAISS	Facebook AI Similarity Search
FO	Functional Operator，功能算子
GPU	Graphics Processing Unit，图形处理器
KNN	K-Nearest Neighbors，k 近邻
PR-AUC	Precision-Recall Area Under Curve
QPS	Queries Per Second，每秒查询数
RAPIDS	面向 GPU 数据科学与分析计算的开源加速生态
ROC-AUC	Receiver Operating Characteristic Area Under Curve
TOD	Tensor Operations for Outlier Detection
Top-k overlap	近似排序结果与基线排序结果的前 k 重合度
`adaptive_ratio`	多 GPU 动态分流比例策略
bounded batching	在时延约束下受控执行的微批处理策略
rerank	对粗排候选执行更高精度精排计算的步骤

第 1 章　绪论

1.1　研究背景与意义

异常检测是金融风控、反欺诈、反洗钱、交易监测与平台治理等场景中的基础能力，其目标是在大量正常样本中识别出少量偏离总体分布的高风险对象。与传统监督学习任务相比，金融异常检测通常面临标签稀缺、异常样本极度不平衡、异常模式持续变化以及业务时效性要求高等现实约束，因此研究重点不仅在于检测模型能否识别出异常，更在于系统能否在真实业务环境中以可接受的吞吐率与时延稳定运行[1-2]。随着金融数据规模从百万级扩展至千万级乃至更高量级，异常检测任务已不再是单纯的模型计算问题，而成为一个包含数据准备、候选检索、异常评分、设备间通信、日志追踪与在线扩展在内的系统工程问题[2-3]。

如图 1-1 所示，在金融业务场景中，异常检测通常服务于实时交易风控、支付反欺诈、账户行为审计、洗钱交易排查与交易路径审查等任务。这些场景具有两类显著特征：其一，数据规模大、维度高、更新快，交易行为、账户状态、地理位置、设备指纹、操作轨迹与时间统计特征往往需要共同参与异常判定；其二，系统必须在较短时间内给出可执行的异常评分或告警结果，延迟过高会直接影响业务决策有效性，吞吐不足则会导致高峰期检测链路阻塞[1]。因此，一个面向金融场景的异常检测系统必须同时具备较高计算效率、较低数据路径开销以及良好的工程可扩展性。

传统基于 CPU 的异常检测系统在小规模数据环境下能够完成较为稳定的训练与推理，但当样本规模不断增大时，距离计算、排序、候选组织与统计聚合等操作所带来的时间与空间开销会迅速放大[1-2]，如图 1-2 所示。近年来研究者开始将 GPU 用于异常检测加速，希望借助并行能力提升距离计算、排序和统计聚合效率。然而，仅对单一算子进行加速并不必然能够构建高性能异常检测系统：随着单算子执行速度提升，系统瓶颈会进一步转移到 CPU↔GPU 数据传输、检索与评分阶段间的中间结果组织，以及多 GPU 场景下的任务调度与结果聚合上[4,7]。

与此同时，大规模近似最近邻（ANN）检索系统的发展可以成为异常检测系统的新思路。对于金融大规模异常检测任务而言，并不一定需要对全量样本直接执行高代价评分，而可以先借助高吞吐候选召回模块在大规模数据中快速筛出候选集合，再对候选集合执行更精细的异常评分。这种“候选召回—精排评分”的两阶段思路，一方面能够显著降低后续异常评分的计算负担，另一方面也

使系统具备更好的吞吐可扩展性[3]。然而，在工程实现层面，候选召回与异常评分并不是简单串联即可，还需要解决中间结果表示、跨阶段数据搬运、GPU 内存组织、I/O 与计算重叠以及多 GPU 扩展等一系列问题[3,8]。

基于上述背景，本文选择从系统角度研究金融大规模异常检测问题，聚焦于 PyTOD 与 FlashANNS 的融合实现。PyTOD 通过张量算子抽象支撑异常评分主干，FlashANNS 通过异步 I/O—计算流水线提供高吞吐候选召回能力[2-3]。将二者结合，有助于降低全量异常评分的计算负担，并通过统一的数据流、算子流与调度流优化异常检测系统的整体性能。更进一步，在单卡场景中若能构建 GPU 主导端到端执行链路，使查询特征、候选结果与评分中间张量尽量驻留 GPU，则可减少 CPU 与 GPU 间的数据搬运；在多卡场景中若采用数据并行与索引复制，使每张 GPU 独立完成本地闭环，并让 CPU 尽量退出数据平面，仅保留控制面职责，则有望在保持复杂度可控的同时提升整体吞吐[8-9]。

因此，围绕金融大规模异常检测系统的研究具有明确的理论意义和工程意义：它推动异常检测研究从“单一检测模型优化”向“模型—系统协同优化”发展，并为金融风控场景下构建高吞吐、低时延、可扩展的异常检测系统提供可行路线。

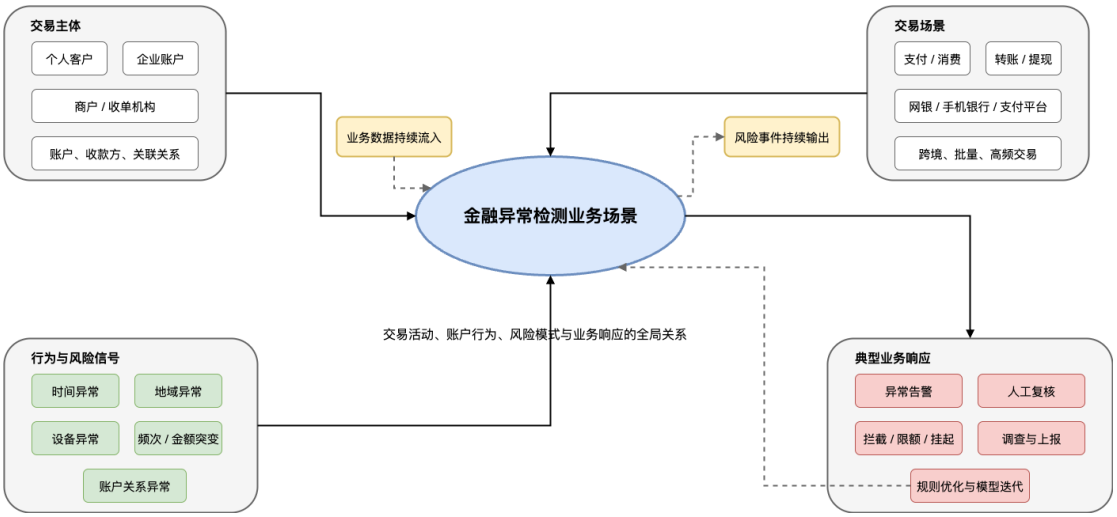

图 1-1　金融异常检测业务场景总览图

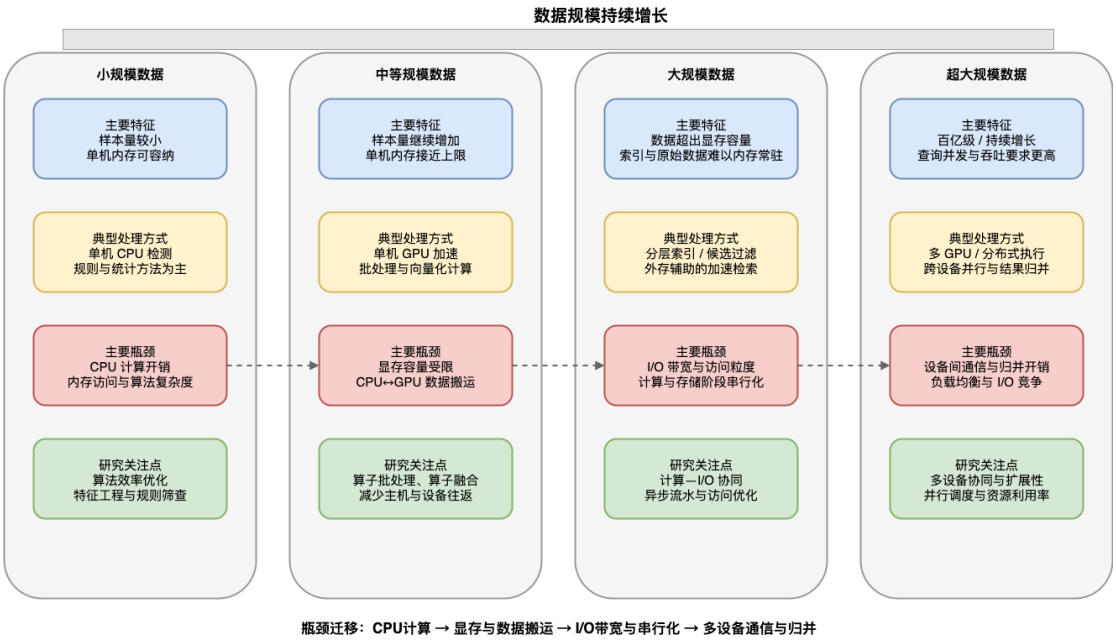

图 1-2　数据规模增长与系统瓶颈迁移示意图

1.2　研究问题与研究对象

金融大规模异常检测任务的核心，在于对大量交易、账户与行为记录进行快速筛查，并在保证检测效果的前提下及时输出异常分数或告警结果，如图 1-3 所示。与传统离线分析任务不同，金融场景中的异常检测往往具有连续到达、时延敏感和峰值流量显著等特点，这使得系统必须在有限时间内完成特征构造、候选检索、异常评分与结果输出等多个阶段[1]。由于不同阶段的计算模式、数据访问模式和资源需求各不相同，因此本文将研究对象定义为一个完整的异常检测系统，而非某一个独立的异常检测算法。

从任务流程上看，本文关注的系统可抽象为如下处理链路：对输入的金融交易记录、账户行为统计和上下文特征进行规范化与向量化表示；基于向量化表示执行高吞吐候选召回，以缩小后续异常评分所需处理的样本范围；在候选集合上执行张量化异常评分，输出异常分数或排序结果；结合日志、统计指标与告警规则完成结果输出与追踪[2-3]。

围绕这一系统处理流程，本文选择 **PyTOD–FlashANNS 融合异常检测系统**作为研究对象。PyTOD 的优势在于将异常检测过程抽象为一组可复用的张量算子，使距离计算、排序、聚合与统计过程能够在 GPU 上以统一语义执行；FlashANNS 的优势则在于为大规模候选召回提供异步 I/O—计算流水线和高吞吐检索能力，如图 1-4 所示[2-3]。

然而，PyTOD 与 FlashANNS 的简单串联并不能自动得到高性能系统。若查询特征在 CPU 上构造，再传输至 GPU 进行候选检索，而候选结果又返回 CPU

重新组织后再送回 GPU 评分，则系统瓶颈将从算子执行转移到 CPU↔GPU 数据搬运与阶段同步上[4,7]。进一步地，在多 GPU 场景中，若采用索引分片并将多个 GPU 上的局部结果集中到 CPU 归并，则 CPU 可能重新成为系统瓶颈[15]。

基于此，本文的研究问题可归纳为以下四点：

1. 如何将 PyTOD 的张量化异常评分能力与 FlashANNS 的高吞吐候选召回能力有机结合，构建统一的金融异常检测执行链路；
2. 如何在单卡场景中构建 GPU 主导的端到端执行链路，使查询特征、候选结果与评分中间张量尽量保持在 GPU 侧，从而减少 CPU 与 GPU 之间的非必要往返与 I/O 串行等待；
3. 如何在多 GPU 场景下优先采用数据并行与索引复制策略，使每张 GPU 独立完成候选召回与异常评分，降低 CPU 聚合和跨卡通信对系统吞吐的影响；
4. 当索引容量或业务约束导致必须跨卡归并时，如何将归并逻辑尽量下沉至 GPU 并通过 NCCL 等设备间通信机制完成结果合并，同时结合拓扑感知的 I/O lane 绑定与准入控制避免共享存储路径成为新瓶颈[5,10]。

因此，本文的研究对象不是单一“异常检测模型”，也不是单一“向量检索组件”，而是一个面向金融大规模异常检测任务的融合系统。本文重点回答的问题是：**如何在保证检测效果可接受的前提下，将候选召回、异常评分、数据路径优化和多 GPU 扩展统一到一个可部署、可扩展、可验证的系统框架中。**

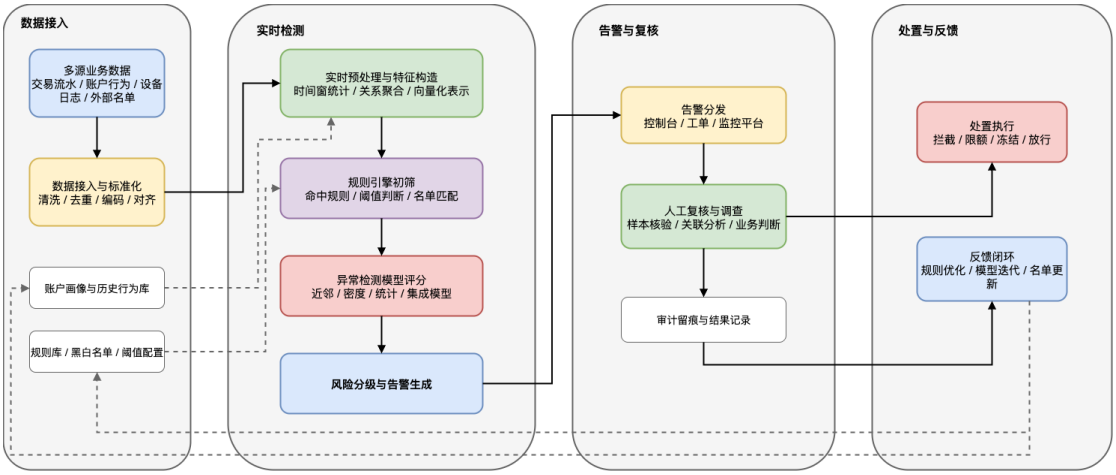

图 1-3　金融大规模异常检测系统标准处理流程图

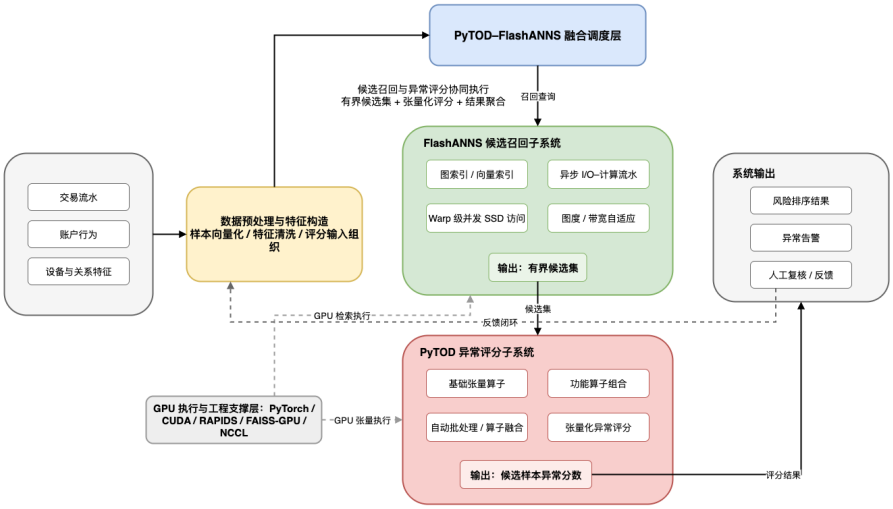

图 1-4　PyTOD–FlashANNS 融合系统概念图

1.3　核心问题分析

对于面向金融场景的大规模异常检测系统，性能瓶颈并不来自单一环节，而是由异常评分复杂度、候选召回开销、数据路径组织成本以及多 GPU 扩展代价共同决定。为明确本文研究重点，本节对核心问题进行分析。

1.3.1　异常评分阶段的高复杂度计算问题

许多异常检测方法依赖样本之间的距离、近邻关系或统计偏离程度进行评分，尤其是基于近邻、密度或局部异常因子的检测方法，需要大量距离计算、排序与聚合操作[1,16]。当数据规模上升到百万级以上时，全量距离矩阵的显式构造会带来极高计算开销与内存压力。PyTOD 通过张量算子抽象提升异常评分计算效率[2]，但其优势主要体现在“如何高效计算评分”，而非“如何缩小评分对象范围”，因此评分复杂度必须与候选召回联合考虑。

1.3.2　候选召回阶段的存储访问与吞吐瓶颈问题

候选召回为后续评分提供规模可控且质量可接受的候选集合[3]。在大规模数据环境中，当索引规模超过显存容量并进入 out-of-core 路径时，I/O 与计算协同效率成为吞吐上限的关键因素。FlashANNS 的异步 I/O—计算流水线证明，通过重叠 SSD 访问与 GPU 计算可显著提升持续吞吐[3]。

1.3.3 CPU–GPU 数据传输与阶段同步问题

在工程系统中，瓶颈往往来自阶段边界的数据组织、格式转换与同步等待。若查询特征与候选结果频繁在 CPU 与 GPU 之间往返，即便候选召回和异常评分算子已被加速，端到端吞吐仍会被 H2D/D2H 与阶段同步拖慢[4,7]。

1.3.4 多 GPU 扩展中的结果聚合与共享 I/O 瓶颈问题

多 GPU 扩展并不必然线性：一方面存在索引复制、查询分发、设备间通信与归并开销；另一方面多卡并发访问共享 PCIe/NUMA/SSD 可能放大 I/O 竞争[5,8]。因此，多 GPU 扩展的核心问题是如何在保持复杂度可控前提下提升吞吐，并避免 CPU 聚合与共享 I/O 再次成为瓶颈。

1.4 研究目标与技术路线

本文的研究目标是在保证异常检测效果可接受的前提下，构建一套面向大规模金融数据运行、具备较高吞吐率、较低时延并支持多 GPU 扩展的融合异常检测系统。围绕该目标，本文从系统架构、单卡数据路径优化和多卡扩展机制三个层面展开研究。系统的整体技术路线图如图 1-5 所示。

系统功能层面，以 PyTOD 为异常评分主干、以 FlashANNS 为候选召回模块，建立二阶段执行结构：通过候选召回缩小评分范围，再在候选集合上执行张量化异常评分[2-3]。

单卡性能优化层面，提出 GPU 主导端到端执行链路：在 FlashANNS 异步 I/O—计算流水线基础上，使查询特征、候选结果与评分中间张量尽量保持在 GPU 侧，仅在必要边界通过 pinned memory 与主机交互；并引入 RAPIDS 与 FAISS-GPU 进行后端增强[4,6-7]。

多 GPU 扩展层面，优先采用数据并行与索引复制策略，使每张 GPU 独立完成本地闭环，CPU 仅保留控制面职责；当索引无法复制时，采用 GPU 侧归并与 NCCL 通信作为兜底，并结合拓扑感知 I/O lane 控制降低共享路径冲突[8,10-11]。

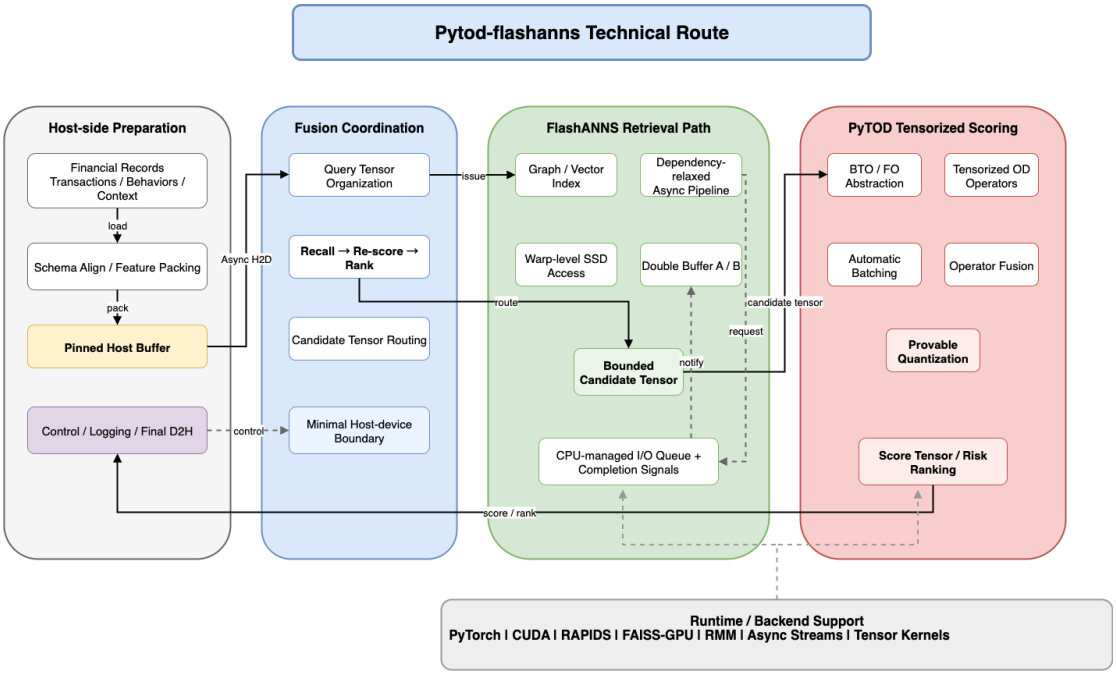

图 1-5　全文技术路线图

1.5　本文主要工作与创新点

本文主要工作包括：构建 PyTOD–FlashANNS 融合框架；提出单卡 GPU 主导端到端执行链路并进行数据路径压缩；引入 RAPIDS 与 FAISS-GPU 重组数据处理与检索后端；提出多 GPU 数据并行与索引复制主方案，并给出 GPU 侧归并与拓扑感知 I/O 控制兜底机制[2-3,8]。

本文创新点概括为三方面：

1. **系统融合创新**：提出面向金融异常检测任务的 PyTOD–FlashANNS 融合架构，构建候选召回—异常评分协同执行链路；
2. **单卡数据路径优化创新**：提出 GPU 主导端到端执行链路，在异步 I/O—计算流水线基础上压缩 CPU↔GPU 数据路径，并结合 RAPIDS/FAISS-GPU/pinned memory/异步流降低整体开销；
3. **多 GPU 扩展创新**：提出数据并行与索引复制的多 GPU 扩展策略，使 CPU 退出数据平面；在必要时引入 GPU 侧跨卡归并、NCCL 通信与拓扑感知 I/O 优化支撑更复杂场景。

1.6　论文组织结构

全文围绕“问题提出—相关研究—融合模型构建—单卡系统优化—多 GPU 扩展—综合实验验证—总结展望”的逻辑展开，各章内容如下：第一章为绪论；第二章为相关研究工作与关键技术基础；第三章为融合检测模型与数据准备；第

四章为单卡系统设计与数据路径优化；第五章为多 GPU 扩展与聚合优化；第六章为综合实验与讨论；第七章为全文总结与展望。

第 2 章　相关研究工作与关键技术基础

2.1　异常检测方法研究现状

异常检测的目标是在大量正常样本中识别出少量偏离总体分布的对象，其研究长期围绕距离型、密度型、统计型、集成型以及深度学习型方法展开[1-2]。对于金融场景而言，异常对象既可能表现为单笔交易的数值偏离，也可能体现为账户行为、设备特征、时序模式或多实体交互关系中的结构性异常。因此，金融异常检测不仅要求方法具有一定的识别能力和可解释性，还要求其能够在高吞吐、低时延和大规模数据条件下保持可执行性。本文后续所关注的，并非“哪一类方法在理论上最强”，而是“哪一类方法在系统层面更容易被组织为可部署、可扩展的检测链路”。如图 2-1所示，异常检测方法可以从建模对象和评分机制两个维度进行归类；表 2-1进一步比较了各类代表方法在复杂度、可解释性以及大规模金融场景适配性上的差异。

距离型方法是金融异常检测中最常见的一类路线，其基本思想是根据样本与近邻之间的距离、局部连通结构或几何关系来刻画异常程度[1,17]。例如，基于 k 近邻的异常检测通常将样本与其第 k 个近邻的距离、平均近邻距离或邻域集合结构作为风险评分依据；ABOD、COF、LOF 等方法则进一步引入角度、可达距离或局部密度等信息，以提高对复杂异常模式的识别能力。该类方法的优势在于业务解释直观，与金融风控中“异常样本偏离正常交易模式”的经验认知较为一致。然而，距离型方法通常需要显式或隐式地处理大规模近邻关系，其时间与空间复杂度在朴素实现下往往较高。TOD 对表格型异常检测算法的整理表明，多种 proximity-based 方法在朴素实现下至少具有二次时间或空间复杂度，这直接限制了其在近实时金融场景中的可部署性[2]。

与距离型方法相比，密度型和统计型方法更强调样本在局部分布或全局分布中的稀有程度。密度型方法通常通过比较样本局部密度与邻域密度之间的差异来识别异常，典型代表包括 LOF、LOCI 等；统计型方法则通过特征分布、边缘概率或直方图结构来评估异常程度，例如 HBOS、COPOD 与 ECOD 等[2,16]。这类方法在部分低维或中等维度表格数据上具有较好的效率优势，也更容易与批处理统计过程结合。但从系统实现角度看，密度型方法往往仍然依赖近邻结构，统计型方法则对数据预处理、分布表达和批量组织方式较为敏感，因此其大规模效率并不只由算法公式决定，还强烈依赖底层数据路径与计算资源组织方式。

集成型方法与深度学习型方法则分别代表了两种不同的提升方向。前者通

过组合多个检测器或多个子空间模型来提高稳健性，后者则利用深层表征能力处理更复杂的数据分布和非线性模式[1-2]。但对于本文关注的金融大规模异常检测系统而言，这两类方法并不天然占优：一方面，集成方法通常会进一步提高推理成本和系统复杂度；另一方面，深度模型虽然表达能力更强，却往往伴随更高的在线部署成本、更复杂的训练依赖以及较弱的业务可解释性。因此，本文不以提出新的异常检测模型为主要目标，而是更关注如何在保持合理检测效果的前提下，选择**可解释、可张量化、可分阶段执行**的方法作为评分主干，并通过候选召回与系统优化降低整体检测成本。基于这一判断，本文后续采用“候选召回 + 张量化异常评分”的两阶段系统路线，而不是直接追求更复杂的端到端检测模型。

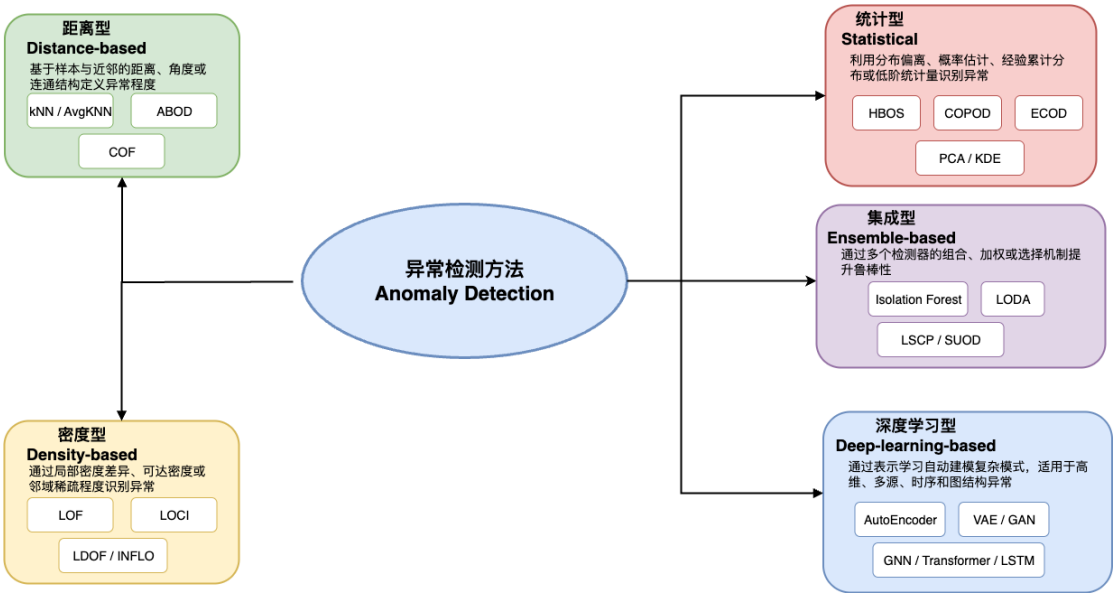

图 2-1　异常检测方法分类图

表 2-1　各类异常检测方法复杂度与系统适配性比较

方法类别	代表方法	复杂度与系统特征	可解释性	大规模金融场景适配性
距离型	*k*NN、ABOD、COF	依赖近邻搜索与距离计算，朴素实现下时间和空间开销较高	强	适合作为评分主干，但通常需要候选召回和 GPU 加速共同支撑
密度型	LOF、LOCI	需要局部密度估计与邻域聚合，复杂度随近邻结构迅速上升	较强	适合描述相对异常，但系统效率仍受近邻组织和数据路径制约
统计型	HBOS、COPOD、ECOD	依赖边缘分布、分位统计或直方图结构，单次评分较轻量	中等	适合作为基线或辅助评分模块，但对复杂金融异常模式覆盖有限
集成型	Feature bagging、Detector ensemble	通过组合多个检测器提升稳健性，但推理成本和系统复杂度进一步增大	中等	适合离线分析或稳健性增强，不利于高吞吐近实时链路
深度学习型	Autoencoder、GNN、时序模型	表达能力强，但训练与部署依赖重，在线解释和维护成本高	较弱	适合复杂模式建模，但需要更高工程投入，不是本文主路线

2.2　TOD/PyTOD 的方法学基础

为解决传统异常检测算法难以直接迁移到 GPU 并高效扩展的问题，TOD 提出了一种面向异常检测系统的张量化抽象方法，其核心思想不是为每一种检测器分别编写定制化 GPU 实现，而是将异常检测过程分解为一组基础张量算子与功能算子，并在现代深度学习基础设施上统一执行[2]。如图 2-2所示，TOD 的系统总览可以概括为三层：底层是可由 GPU 高效执行的基础张量算子，中间层是由基础算子组合而成的功能算子，上层则是具体异常检测算法及其组合形式。图 2-3进一步展示了典型异常检测方法到张量算子的映射关系，例如 *k*NN、LOF、COPOD 等算法可被重写为 `cdist`、`topk`、聚合与统计运算的组合[2]。这一抽象的直接意义在于：异常评分不再被视为“只能在 CPU 上完成的算法逻辑”，而可以被组织为标准化、可批处理、可并行的 GPU 计算过程。

TOD 的第二个重要贡献在于其面向大规模场景的系统优化机制，包括 provable quantization、automatic batching 以及 multi-GPU support[2]。其中，量化机制通过在满足精度边界的条件下降低部分算子的内存与计算开销，自动批处理则将原本难以放入单卡显存的大规模异常检测计算拆分为可流水执行的小批次，多 GPU 支持则进一步把这些批次分配到多个设备并行执行。TOD 的相关实验表明，在异常检测任务中，GPU 时间往往占总运行时间的绝大部分，而通过算子融合与批处理优化，可以显著减少大规模中间矩阵的物化和搬运开销，从而提升整体效率[2]。这说明，对于本文所关注的大规模金融数据场景，异常评分是否能够被“张量化 + 批处理 + 多 GPU 化”，比单一检测器的理论公式更决定其系统可落地性。

PyTOD 可以看作 TOD 的开源实现与工程化延展，其价值在于把张量化异常检测从论文级概念推进为可复用的软件能力。对本文而言，PyTOD 的意义主要体现在三个方面：第一，它提供了 *k*NN、LOF、HBOS、ABOD 等多类检测器的统一接口，使本文能够在同一评分主干上比较不同检测器的适配性；第二，它继承了 TOD 所强调的张量算子抽象与 GPU 批处理思路，为后续将异常评分嵌入候选召回链路提供了方法学基础；第三，它天然适合作为“评分主干”而非“完整检测系统”，因为其重点在于异常评分的 GPU 化，而不直接覆盖大规模外存候选召回、GPU–SSD 数据路径以及多设备在线调度等更高层系统问题。表 2-2据此总结了 PyTOD 在本文中的能力边界：可直接复用的是张量化评分能力，需要补强的是候选驱动接口、GPU 常驻输出路径以及与外部召回模块的融合方式。

需要强调的是，早期 GPU 异常检测研究多采用“面向特定算法定制加速”的思路，例如针对距离型异常检测或局部离群检测单独设计 CUDA 版本。与之相

比，TOD/PyTOD 所代表的路线更适合本文的研究目标：本文并不打算为某一个检测器单独做高度特化优化，而是希望建立一个能够同时兼容候选召回、批处理评分和多 GPU 扩展的统一系统框架。因此，本文后续以 PyTOD 作为异常评分主干，并不是因为它能够独立解决全部系统问题，而是因为它为“评分阶段的 GPU 化与系统化组织”提供了最合适的基础。

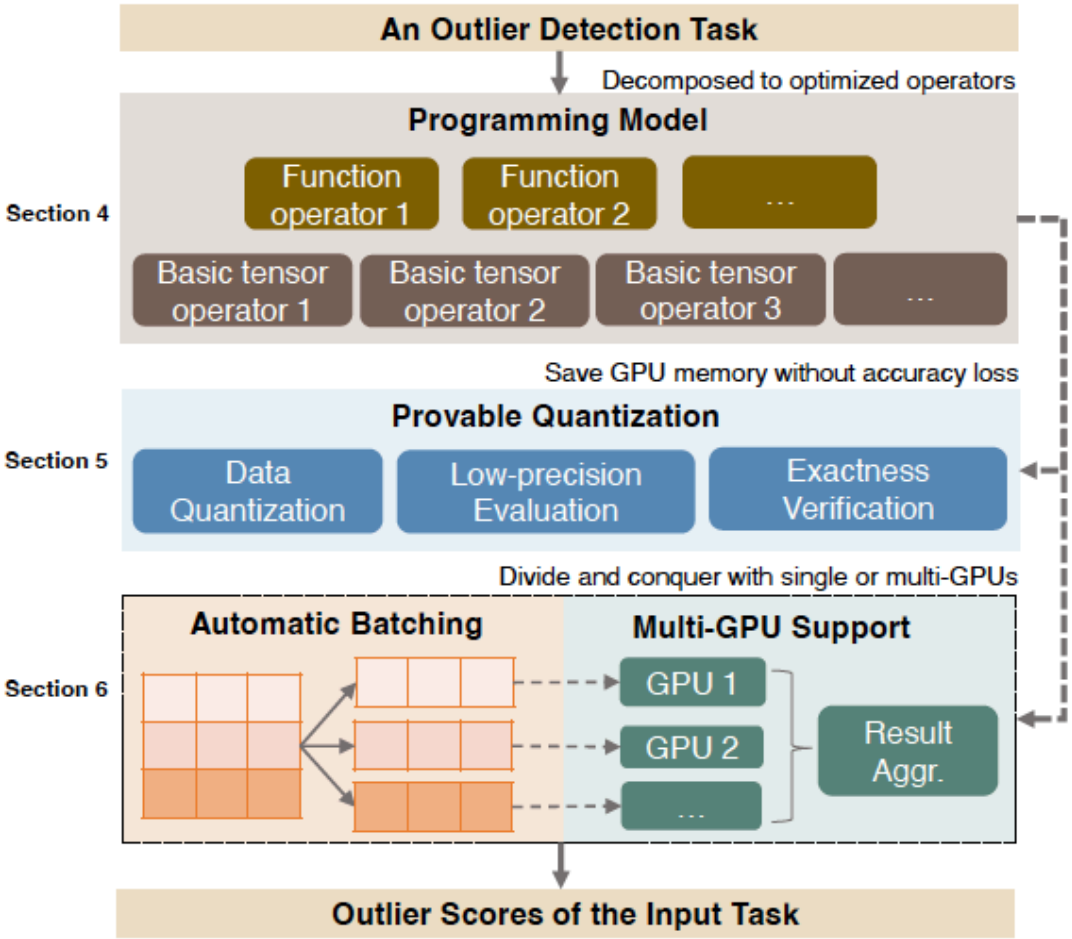

图 2-2　TOD/PyTOD 系统总览图

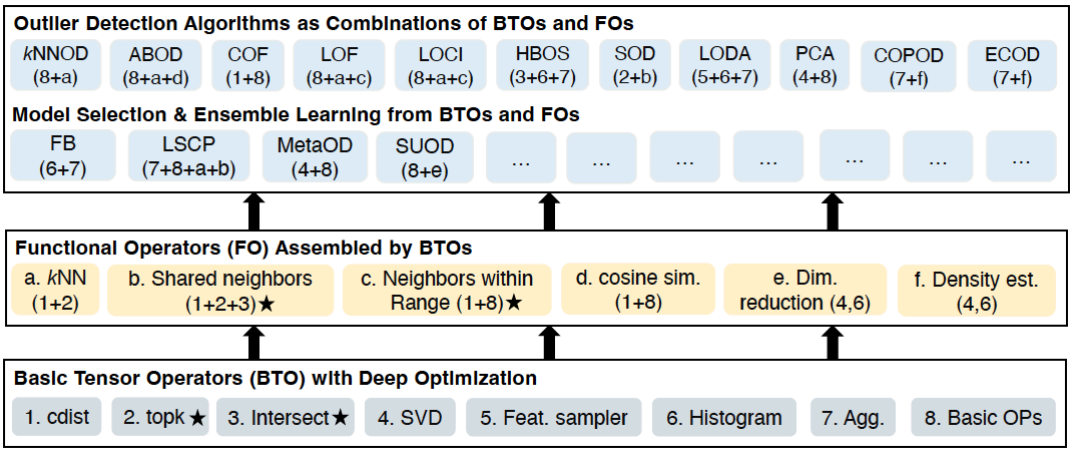

图 2-3　典型异常检测算法到张量算子的映射

表 2-2　PyTOD 在本文中的能力边界

能力类别	PyTOD 已支持 / 可直接复用内容	本文未直接覆盖的能力边界	本文中的补强方式
方法学抽象	将异常检测算法抽象为 **BTOs + FOs**，支持将多种经典 OD 方法映射到 GPU 张量执行语义	不负责候选召回与两阶段检索–评分协同	由 **FlashANNS** 提供候选召回，PyTOD 只承担异常评分主干
异常评分执行	支持 *k*NN、LOF、HBOS、COPOD、ECOD、PCA 等多类 OD 算法的 GPU 化评分	不直接处理金融业务场景下的多阶段风险检测流水	在第三章中将 PyTOD 嵌入到融合检测模型，作为评分后端
算子层优化	支持 **provable quantization**、operator abstraction、部分 operator fusion 思路	不覆盖候选集组织、重排序与 ANN 检索路径管理	由本文融合层补充候选集输入组织与评分接口适配
大规模执行	支持 **automatic batching**，缓解显存不足问题；支持 **multi-GPU support**	不解决 SSD-resident 索引、异步 I/O、检索–计算重叠	由 FlashANNS 提供 out-of-core 检索与 I/O–compute overlap
系统角色	适合作为 **评分主干**，提供统一、可复用、可 GPU 化的异常评分框架	不是完整金融异常检测系统，不包含数据接入、召回、告警与复核闭环	本文在第三、四、五章中补足系统融合、数据路径优化和多 GPU 扩展
工程部署	基于 PyTorch，具备较好可扩展性，便于复用现有 GPU 张量基础设施	不直接提供面向金融风控在线系统的端到端部署结构	本文结合 RAPIDS、FAISS-GPU、FlashANNS 与调度层进行工程化整合

2.3 FlashANNS 与候选召回系统

在大规模异常检测系统中，若直接对全量样本执行精细异常评分，系统往往会因近邻搜索、候选组织和内存占用过大而难以满足吞吐要求。因此，引入高吞吐候选召回模块，以先筛选潜在相关样本、再执行精细评分，是构建近实时检测系统的常见路线。FlashANNS 正是面向这一问题的代表性工作，其核心关注点不是单纯“如何找近邻”，而是“在外存条件下如何使近邻检索中的 I/O 与计算协同推进，从而打破存储—计算瓶颈”[3]。如图 2-4所示，FlashANNS 的基本思想可以概括为异步 I/O—计算流水线：在保持 GPU 计算持续进行的同时，将外存访问与候选扩展过程重叠执行，以减少传统 ANN 图搜索中“先读后算”的严格串行开销。

具体而言，FlashANNS 针对 out-of-core ANN 场景强调三个方面的设计。首先，它通过依赖放松的异步流水线，使部分计算不必等待全部 I/O 完成后再启动，从而提高 GPU 的忙碌度；其次，它利用 warp 级并发和细粒度任务组织提升设备侧对候选扩展与结果筛选的执行效率；再次，它把系统瓶颈从单纯的“计算速度”扩展为“存储路径与计算资源的协同平衡”，强调 ANN 检索在大规模场景下是一个 **I/O—计算联合优化问题**而非单纯算子优化问题[3]。这意味着，FlashANNS 与本文的关系不是“替代异常检测”，而是承担候选召回前端，为后续更昂贵的异常评分缩小搜索空间。图 2-5展示了 FlashANNS 在本文系统中的位置：它位于候选召回阶段，并与后续 PyTOD 评分阶段构成两阶段融合链路。

从候选召回系统的研究脉络看，传统 ANN 组件、GPU 内存内 ANN 检索和 out-of-core ANN 系统各自关注的瓶颈并不相同。前者更关注索引结构与搜索精度，后者强调设备内向量搜索效率，而 out-of-core 系统则必须额外解决 SSD 访问、对齐 I/O、异步完成信号和大规模索引布局等问题。与一般的内存内 ANN 路线相比，FlashANNS 更适合被放在本文的大规模金融数据背景下讨论，因为本文所面临的问题并不是“单卡内存足够时如何做到最快”，而是在索引规模增大、候选池变大、设备数增加时，如何保持系统整体吞吐不被存储路径拖垮。表 2-3将传统 ANN 组件、out-of-core ANN 系统与 FlashANNS 的主要特征进行了对比，重点突出它们在 I/O—计算重叠、大规模外存适配性以及与本文系统关系上的差异。

需要指出的是，FlashANNS 解决的是“大规模候选召回”的问题，而不是“异常评分”的问题。它可以显著减轻评分阶段面对的全量搜索压力，但并不直接给出异常分数，也不自动解决后续评分过程中的 GPU 常驻执行、张量组织、候选到评分的接口契约等问题。因此，本文并不将 FlashANNS 视为完整异常检测系统，而是将其与 PyTOD 组合：前者负责高吞吐候选召回，后者负责张量化异常评分。正是这种角色分工，使本文的两阶段系统既继承了 GPU 异常评分的计算

优势，也继承了大规模 ANN 检索在外存条件下的吞吐优势。

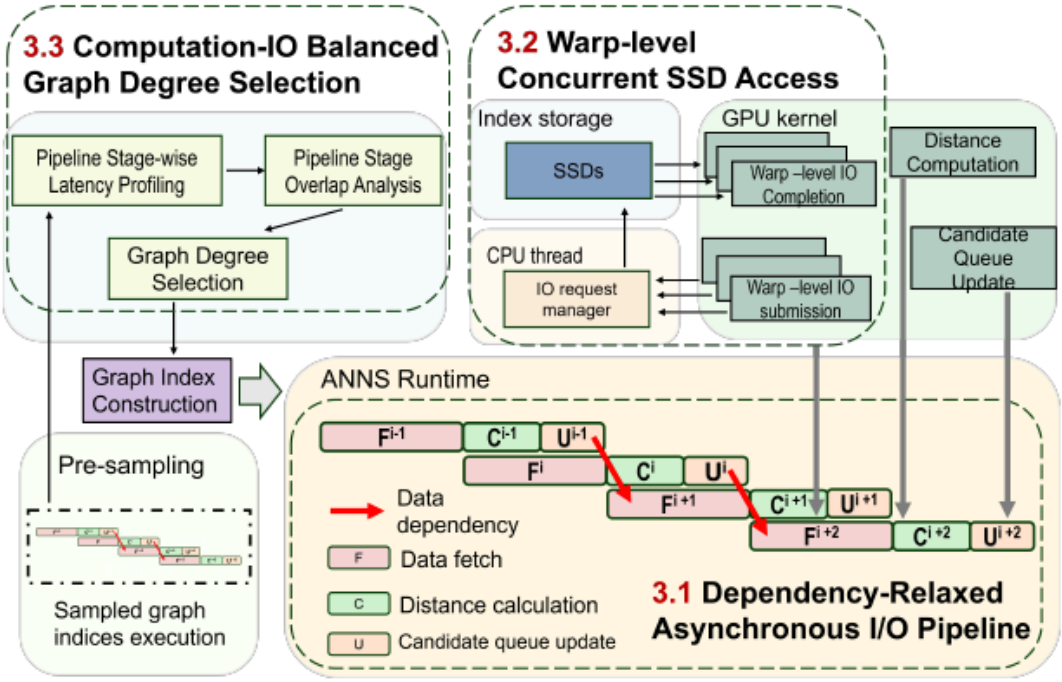

图 2-4　FlashANNS 异步 I/O—计算流水线原理

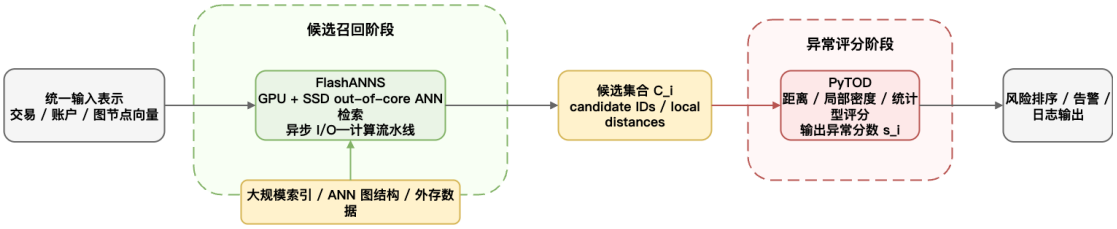

图 2-5　FlashANNS 在本文系统中的位置

表 2-3　候选召回系统比较

类别	代表方案	主要存储假设	是否考虑 I/O–计算重叠	是否适合大规模外存	主要特点
传统 ANN 组件	FAISS、HNSW、IVF/IVF-PQ 等	以内存内索引为主，通常假设索引或主要检索结构可驻留内存/显存	通常不重点考虑	一般较弱	检索延迟低、工程成熟、适合中等规模向量召回；但面对超大规模数据时易受内存/显存容量约束
out-of-core ANN 系统	SPANN、DiskANN、FusionANNS 等	面向 SSD/外存驻留索引，通过磁盘存储支撑更大规模数据	部分考虑，但多未充分实现端到端重叠	较强	能突破内存容量限制，支持更大规模检索；但仍可能存在粗粒度候选过多、CPU 供数瓶颈和 I/O 串行化问题
FlashANNS	FlashANNS	明确面向 **GPU + SSD** 的 out-of-core 图索引检索	**是**	**强**	通过 dependency-relaxed asynchronous pipeline、warp-level concurrent SSD access 与 computation–I/O balanced graph degree selection 实现高吞吐候选召回

2.4　RAPIDS、FAISS-GPU 与工程可部署性

如果仅从算法流程出发，PyTOD–FlashANNS 融合系统似乎已经能够完成“候选召回—异常评分”两阶段任务；但从工程实现角度看，系统中仍包含大量与算法无关但对性能影响极大的环节，例如金融表格数据预处理、特征整理、批量统计、中间结果表达、向量检索接口组织以及 GPU 内存资源管理等。为此，本文引入 RAPIDS 与 FAISS-GPU 作为系统后端增强组件，以降低这些环节的实现复杂度，并尽可能让关键数据路径保持在 GPU 侧[4,6-7,18]。如图 2-6所示，RAPIDS 在本文系统中的作用主要体现在“数据处理路径增强”上；如图 2-7所示，FAISS-GPU 则更多承担“设备侧向量检索与互操作能力增强”的职责。表 2-4总结了二者在本文中的功能分工。

RAPIDS 的核心价值在于为 GPU 上的表格处理和数据科学工作流提供统一支持。cuDF 作为其重要组成部分，可以视为面向 GPU 的 DataFrame 引擎，适合处理金融场景中大量以表格形式存在的交易记录、账户属性、设备特征与时间

统计结果。对本文而言，RAPIDS 并不承担异常检测本身，而主要承担候选召回前后的数据组织任务，包括字段清洗、特征拼接、归一化、窗口统计、中间结果重排和批量结果合并等。如果这些操作长期停留在 CPU 侧，即便候选召回和异常评分本身已经 GPU 化，系统整体也会因频繁的 CPU↔GPU 往返而失去吞吐优势。因此，RAPIDS 的真正意义在于压缩“前处理—候选召回—评分—后处理”之间的数据路径长度，为后续单卡 GPU 主导执行链路奠定基础。

与此同时，RAPIDS 并不意味着“只要用了 GPU DataFrame，整条链路就天然保持在 GPU 上”。在兼容模式或未覆盖操作出现时，部分数据处理流程可能退回 CPU 执行；一旦这种回退发生，系统就会重新引入额外的数据搬运与同步等待。由此可见，在本文中 RAPIDS 的价值不应被简单理解为“自动加速”，而应被理解为“为构建统一 GPU 数据路径提供可能性，但仍需结合具体算子覆盖范围与运行路径进行约束”。这也意味着，本文后续在第四章讨论数据路径优化时，必须把“哪些处理确实保留在 GPU、哪些处理可能回退到 CPU”作为工程边界显式说明，而不能把名义上的 GPU 化直接等同于真实执行链路中的 GPU 常驻。

FAISS-GPU 的价值则更多体现在设备侧向量检索能力和 GPU 资源组织能力两个方面。作为成熟的向量检索后端，FAISS-GPU 支持在 GPU 上完成向量索引构建、近邻检索和部分结果筛选，其 `StandardGpuResources` 通过预分配 scratch memory 与 CUDA stream 组织方式，能够减少频繁 `cudaMalloc/cudaFree` 带来的同步与抖动[7,18]。此外，FAISS-GPU 还明确区分 host pointer 输入和 device pointer 输入两类路径：若输入已经位于同一 GPU 上，则可以避免额外拷贝；若输入位于主机端或位于不同设备上，则会引入相应的数据搬运开销。总的来说，FAISS-GPU 在本文中的作用不仅是“提供一个能跑的向量检索后端”，更重要的是为后续分析 CPU/GPU 互操作路径、解释何处发生额外拷贝以及设计 GPU 常驻数据路径提供工程依据。

从角色分工上看，RAPIDS 与 FAISS-GPU 分别服务于“数据处理路径”和“向量检索路径”。前者让金融表格型数据及中间结果组织尽量留在 GPU；后者为候选检索提供高性能设备侧实现与成熟资源管理。二者的共同目标是压缩 CPU 与 GPU 之间的非必要往返，使 PyTOD–FlashANNS 融合系统不止在候选召回和异常评分两个单点上更快，而是在整条执行链路上更低开销、更可部署。基于以上考虑，本文在后续章节中不会把 RAPIDS 与 FAISS-GPU 当作“额外附属工具”，而是把它们视为构建 GPU 主导端到端执行链路的工程支撑组件。

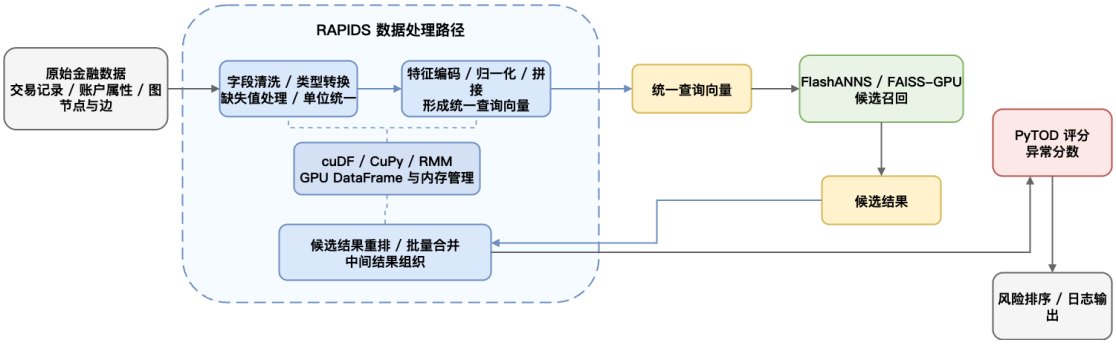

图 2-6　RAPIDS 在本文系统中的数据处理位置

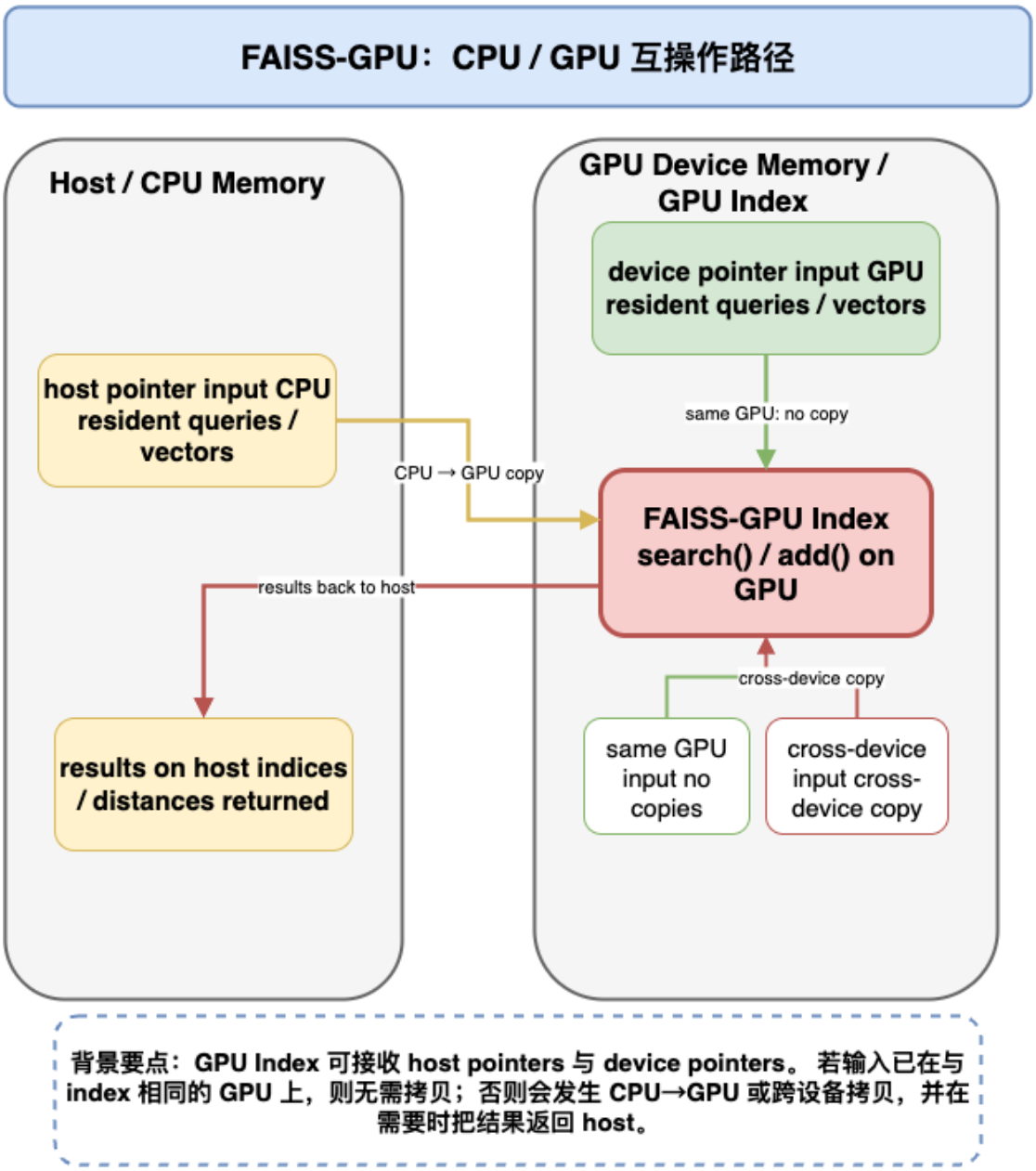

图 2-7　FAISS-GPU 的 CPU/GPU 互操作路径

表 2-4　RAPIDS 与 FAISS-GPU 的功能分工

组件	主要功能	在本文系统中的位置	主要收益	能力边界 / 潜在限制
RAPIDS cuDF	GPU DataFrame、字段清洗、特征拼接、窗口统计、中间结果重排	位于候选召回前后的数据处理路径	缩短 CPU↔GPU 往返，提升表格型数据处理吞吐	若走兼容模式或未覆盖算子，可能发生 CPU fallback
RAPIDS RMM	内存池、页锁定主机内存、设备与主机资源管理	位于关键数据交换与缓冲区管理层	降低频繁分配释放带来的抖动，提升必要边界传输稳定性	不能替代链路设计本身，仍需控制数据交换边界
FAISS-GPU	GPU 侧索引构建、近邻检索、资源对象与 stream 管理	位于候选召回与设备侧向量检索路径	提供成熟高性能检索后端，减少额外拷贝与资源抖动	host/device/cross-device 路径代价不同，若接口组织不当仍会引入数据搬运

2.5　多 GPU 调度与在线推理启示

当单卡系统的吞吐率接近上限，或单卡显存无法容纳更大索引与更大批次时，多 GPU 扩展便成为提升系统能力的自然方向。然而，多 GPU 并不意味着性能一定线性增加，因为更多 GPU 同时也意味着更复杂的查询分发、索引组织、设备间通信、结果归并和共享 I/O 竞争。TOD 的多 GPU 支持已经说明，批处理任务可以被分配到多个设备并行执行，从而在算子级别提升总体吞吐[2]；但对于本文所关注的“候选召回 + 异常评分”两阶段系统而言，多 GPU 扩展不仅是“多做一点同样的计算”，而是需要在候选召回、评分、归并和 I/O 之间重新组织数据路径和控制路径。图 2-8对比了 replicated 与 sharded 两类多 GPU 扩展模式，图 2-9展示了 CPU 聚合与 GPU 侧归并两种路径的差异，表 2-5进一步总结了这些策略的适用条件与系统代价。

replicated 模式倾向于在每张 GPU 上保留完整索引或完整功能链路，使每个设备可以独立完成本地候选召回和异常评分；其优点在于查询路径简单、设备间协同较少，更适合吞吐优先的在线推理场景。sharded 模式则通过在多设备间分片索引或数据来扩展容量，能够支持更大规模索引，但也会引入额外的跨设备归并与同步开销。对本文而言，replicated 更符合金融近实时检测场景的目标：单次请求的关键是稳定吞吐和低尾延迟，而不是极限压缩单卡显存占用。因此，后续多 GPU 主线优先考虑 replicated 架构；当容量约束使得跨卡归并不可避免时，再进一步讨论 sharded 及其归并代价[8,15]。

在归并路径上，CPU 聚合与 GPU 侧归并的差异尤为关键。若多个设备在本地生成候选或分数后，统一回到 CPU 做聚合，则虽然实现简单，但会把 CPU 重新拉回数据平面，进而造成额外的 D2H/H2D 搬运和同步等待。相反，如果将跨卡归并逻辑尽量下沉到 GPU，并结合 NCCL 等设备间通信机制，则可以把更多中间结果留在设备侧，减少主机聚合带来的瓶颈。NCCL 官方文档指出，其集体通信

操作会异步入队到指定 CUDA stream 上执行，但在同一 `ncclGroupStart/End()` 组内混用多个 stream 会形成所有 stream 之间的全局同步点[10]。这说明，GPU 侧归并虽优于 CPU 聚合，但如果通信流和计算流组织不当，也可能引入新的同步开销。

除了设备间通信，存储路径与硬件拓扑同样会限制多 GPU 扩展上限。无论采用外存候选召回，还是采用多设备共享存储路径，GPU—SSD—PCIe—NUMA 之间的拓扑关系都可能使“理论上的并行收益”无法在实际系统中完全实现。共享 PCIe 交换机、共享 SSD 通道或非对称 NUMA 路径，都可能使部分设备在高并发下遭遇 I/O 争用，从而把瓶颈重新转移到存储子系统。GPUDirect Storage 文档指出，如果 GPU、NIC 与存储设备位于更理想的 PCIe 拓扑关系下，可以减少 CPU bounce buffer 并提升带宽利用效率[5,11]。这意味着，多 GPU 扩展设计的时候，不能只看 GPU 的硬件指标来进行多 GPU 之间的负载均衡，还必须考虑拓扑感知 I/O lane 绑定、准入控制以及共享路径下的资源组织策略。

综上所述，现有多 GPU 调度与在线推理研究为本文提供了三点启示：第一，在吞吐优先的金融异常检测场景下，应优先采用 replicated 架构，让每张 GPU 尽量独立完成候选召回与异常评分；第二，当必须跨卡归并时，应尽量将归并逻辑下沉到 GPU 并利用 NCCL 通信，而不是回到 CPU 聚合；四三，多 GPU 扩展的上限不仅受设备数目影响，也受制于存储路径、PCIe/NUMA 拓扑与 I/O 资源组织方式。因此，本文后续的多 GPU 设计不是单纯“加更多 GPU”，而是围绕“CPU 控制面 + GPU 数据平面 + 拓扑感知 I/O 控制”构建完整扩展路径。

正式版补齐图 2-8

图 2-8　replicated 与 sharded 两类多 GPU 扩展模式对比

正式版补齐图 2-9

图 2-9　CPU 聚合路径与 GPU 侧归并路径对比

表 2-5　多 GPU 扩展策略比较

策略	核心思路	优势	代价 / 风险	本文中的定位
Replicated	每张 GPU 持有完整索引与完整执行链路，查询在设备间分发	查询路径简单，跨卡协同少，吞吐扩展更直接	需要更多显存，索引复制成本较高	多 GPU 主方案，适合吞吐优先金融场景
Sharded	多张 GPU 分片保存索引或数据，单次查询需跨卡协同	可支持更大容量，降低单卡显存压力	需要跨卡归并与更复杂同步，尾延迟更敏感	容量受限时的兜底路径
CPU 聚合	各 GPU 输出局部结果后统一回 CPU 归并	实现相对简单，工程门槛较低	CPU 重回数据平面，D2H/H2D 往返和同步开销高	不作为主方案，仅用于对比和边界分析
GPU 侧归并	在设备侧完成局部结果合并，并结合 NCCL 通信	减少主机聚合瓶颈，中间结果更易留在 GPU 上	通信流组织不当时可能形成新的同步点	sharded 或特殊约束场景下优先采用
拓扑感知 I/O 控制	结合 PCIe/NUMA/SSD 拓扑进行 lane 绑定与准入控制	降低共享存储路径争用，更接近真实扩展收益	需要硬件拓扑感知和额外调度策略	多 GPU 上限控制的重要配套机制

2.6　本章小结

本章围绕本文研究对象所涉及的关键技术与相关研究进行了系统梳理，并尝试解释“本文为何采用这一技术路线”，同时阐述如何重构相关工作。首先，从异常检测方法角度看，距离型、密度型和统计型方法为金融异常检测提供了理论基础，但在大规模场景下普遍面临复杂度与可扩展性约束，因此单纯比较模型类别并不足以支撑系统设计[1-2]。其次，从 GPU 异常检测系统角度看，TOD/PyTOD 通过张量算子抽象、自动批处理与多 GPU 支持证明了异常评分可以被统一映射到 GPU 执行框架中，从而为本文选取 PyTOD 作为评分主干提供了方法学依据[2]。另外，从候选召回系统角度看，FlashANNS 强调异步 I/O—计算流水线和存储—计算平衡，说明大规模 ANN 检索的关键问题并不只是近邻精度，而是候选召回如何与底层 I/O 路径协同推进[3]。同时，RAPIDS 与 FAISS-GPU 分别为 GPU 表格数据处理与 GPU 向量检索提供了成熟后端能力，使构建统一 GPU 数

据路径成为可能[6-7]。最后，在多 GPU 扩展方面，多 GPU 调度与在线推理研究表明，replicated 与 sharded 的选择、GPU 侧归并、NCCL 通信以及拓扑感知 I/O 控制共同决定了多设备场景下的系统扩展能力[5,8,10-11]，这对之后的多 GPU 设计策略提供了理论基础和方向指引。

通过上述分析可以看出，现有研究已经分别在异常评分、候选召回和多 GPU 扩展等方面提供了较为充分的基础，但尚缺少一个面向金融大规模异常检测任务、能够同时统筹候选召回、异常评分、数据路径优化与多 GPU 扩展的统一系统框架。本文正是在这一空白处切入：以 PyTOD 为异常评分主干，以 FlashANNS 为候选召回模块，并结合 RAPIDS、FAISS-GPU、GPU 常驻执行链路以及多 GPU 数据并行与 GPU 侧归并机制，构建一套可部署、可扩展、可验证的金融异常检测系统。为便于后续章节衔接，表 2-6对本章涉及的相关技术、能力边界及其与本文系统设计的关系进行了归纳。第三章将在此基础上进一步给出 PyTOD–FlashANNS 融合模型、实验数据准备与基础执行流程设计。

表 2-6　本章相关技术、能力边界及其与本文系统设计的关系

技术 / 研究方向	已提供能力	能力边界 / 未解决问题	与本文系统设计的关系
传统异常检测方法	提供距离、密度、统计等异常性定义与业务解释基础	大规模场景下复杂度高，难以单独支撑近实时系统	为本文选择“候选召回 + 张量化评分”路线提供方法学背景
TOD / PyTOD	提供张量算子抽象、GPU 评分、自动批处理与多 GPU 评分基础	不直接覆盖大规模候选召回、外存索引与端到端系统调度	作为本文异常评分主干与 GPU 评分骨架
FlashANNS	提供 out-of-core ANN 检索、异步 I/O—计算流水线和高吞吐候选召回能力	不直接输出异常分数，也不自动解决后续评分接口与数据路径问题	作为本文候选召回前端，与 PyTOD 形成两阶段链路
RAPIDS	提供 GPU DataFrame、数据清洗、统计处理和内存资源管理能力	若使用不当仍可能回退 CPU，不能替代链路级优化	作为数据处理路径增强组件，支撑 GPU 主导执行链路
FAISS-GPU	提供成熟的设备侧向量检索、资源对象与 host/device 互操作机制	不同输入路径代价不同，若接口组织不当仍会引入拷贝	作为候选检索后端增强组件，支撑设备侧连续执行
多 GPU 调度与在线推理研究	提供 replicated / sharded、设备通信与拓扑优化等经验	需要结合具体数据路径和存储拓扑做系统化落地	为第五章的数据并行、GPU 侧归并和 I/O 控制提供设计依据

第 3 章　PyTOD-FlashANNS 融合检测模型与数据准备

3.1　金融异常检测任务建模

金融大规模异常检测任务的本质，是在高维、多源、强时序的交易与行为数据中，快速识别少量偏离正常模式的高风险样本。与一般的二分类任务不同，金融异常检测既受到标签稀缺、样本极度不平衡等问题的影响，也受到系统吞吐、响应时延和资源约束的直接限制[1-2]。在工程实现层面，系统必须能够同时处理原始交易记录、账户行为统计、设备与地理位置特征、时间窗口统计量以及可能存在的图结构关系。因此，本文首先对金融异常检测任务进行统一建模，以明确系统输入、候选召回目标、异常评分目标和输出形式，为后续 PyTOD–FlashANNS 融合框架设计奠定基础。

从输入形式上看，本文将待检测对象抽象为一个由样本集合 $X = \{x_1, x_2, \cdots, x_n\}$ 组成的金融行为数据集，其中每个样本 x_i 对应一条交易记录、一个账户时间窗行为摘要或一个节点行为表示。每个样本由连续型数值特征、离散型类别特征和时间上下文特征组成，在预处理阶段被统一映射为固定维度向量表示 $v_i \in \mathbb{R}^d$。对于图结构金融数据，还需将节点特征与边关系经时间窗统计、邻域聚合或嵌入映射后转换为向量化表示，从而与表格型交易数据形成统一输入接口[14,19]。这一抽象使本文系统可以统一处理交易级异常、账户级异常以及图节点级异常，而不必为不同数据形式分别构建独立执行链路。

从系统目标上看，本文的异常检测任务并非直接对全量数据执行精确评分，而是采用“候选召回—异常评分”两阶段处理框架。第一阶段利用高吞吐近似最近邻检索在大规模样本中快速找到与查询样本最相关的候选集合 C_i，从而避免在全体样本上进行高代价比较；第二阶段利用 PyTOD 在候选集合上执行张量化异常评分，生成异常分数 s_i。形式上，可将系统目标写为

$$C_i = \mathcal{R}(v_i, \mathcal{I}), \qquad s_i = \mathcal{F}(v_i, C_i), \tag{3-1}$$

其中 $\mathcal{R}$ 表示候选召回过程，$\mathcal{I}$ 表示候选索引，$\mathcal{F}$ 表示异常评分函数。这样的两阶段处理结构使候选召回质量与异常评分质量共同决定最终检测效果，也使系统能够在大规模场景下获得更高吞吐率与更低资源开销[2-3]。

从输出形式上看，系统可产生三类结果：其一是每个样本的异常分数，用于后续阈值判断或人工复核排序；其二是 Top-k 风险样本集合，用于构造重点审查名单；其三是与异常结果相关的候选邻域、关键特征统计及日志信息，用于系统

追踪和后续解释分析。在金融风控场景中，这三类结果分别对应在线筛查、风险排序和审计追踪三种实际需求。因此，本文设计的 PyTOD–FlashANNS 融合系统，不仅追求更高的候选召回吞吐和异常评分效率，也强调输出结果能够支持实验可重复性和工程可观察性。

图 3-1 对这一统一任务建模过程进行了整体展示，表 3-1 则进一步归纳了研究对象、输入表示与输出形式之间的对应关系。

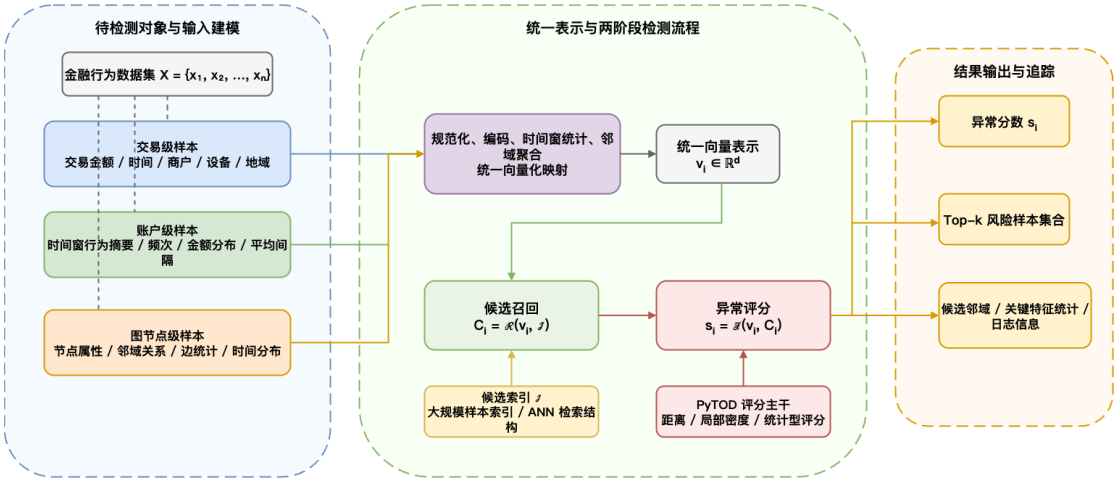

图 3-1　金融异常检测任务建模图

表 3-1　研究对象、输入形式与输出形式定义

研究对象	原始输入形式	统一向量表示形式	主要输出形式	典型业务场景
交易级异常	单笔交易记录，包含金额、时间、商户、设备、地域、支付渠道等字段	对连续型、类别型与时间上下文特征进行规范化、编码与拼接，形成交易级向量 ($v_i \in \mathbb{R}^d$)	异常分数 (s_i)、Top-k 风险交易集合、候选邻域及日志信息	实时交易风控、支付反欺诈
账户级异常	账户在时间窗口内的行为摘要，包括交易频次、金额分布、平均间隔、夜间交易比例、跨地域切换次数等	将账户静态属性与时间窗行为统计特征拼接，形成账户级行为向量 ($v_i \in \mathbb{R}^d$)	异常分数、重点审查账户清单、行为统计解释信息	账户审计、反洗钱筛查、账户风险监测
图节点级异常	图节点属性及其邻域边关系信息，包括节点属性、边方向、金额统计、时间分布等	对节点属性规范化编码，并对邻域关系做聚合统计，形成节点级向量 ($v_i \in \mathbb{R}^d$)	异常分数、Top-k 风险节点、邻域候选及追踪日志	团伙欺诈识别、关系网络异常排查

3.2　实验数据集选取与构建

本文的实验数据安排遵循“真实/半真实数据用于有效性验证，合成扩展数据用于系统吞吐与扩展实验”的基本原则。这样安排的原因在于：一方面，公开可获取的真实金融数据规模通常有限，且多经过脱敏、压缩或竞赛裁剪，适合验证模型与系统在现实场景中的可用性，但不足以单独支撑超大规模系统实验；另一方面，大规模性能验证需要控制数据规模、异常比例和字段分布的一致性，因此需要借助高保真合成数据集构建 10M 与 50M 规模实验环境[13,20]。基于这

一考虑，本文将实验数据集分为两类：第一类为 1M 规模的真实或半真实公开数据集，用于验证候选召回与异常评分在现实数据上的有效性；第二类为 10M 与 50M 规模的高保真合成数据集，用于验证系统在高吞吐与多 GPU 扩展场景下的性能表现。

在 1M 规模的有效性验证中，本文主要使用三类真实或半真实公开数据。第一类为 **IEEE-CIS Fraud Detection** 数据集，该数据集包含交易表与身份表两部分，特征维度高、缺失值复杂，较接近现实支付风控场景，可用于检验系统在高维结构化金融数据上的向量化表示与候选召回效果[12]。第二类为 **Credit Card Fraud Detection** 数据集，该数据集虽规模相对较小，但具有经典性和广泛使用基础，可作为基准异常检测数据集用于对照验证[21-22]。第三类为 **DGraphFin** 金融图数据集，其包含大规模节点与边关系以及金融行为特征，适用于验证系统在图金融数据经过向量化映射后的可用性[14,19,23]。在具体实验组织时，本文将控制单次实验输入规模在约 1M 量级，以保证实验设计的一致性和可比较性。

在 10M 与 50M 规模的性能验证中，本文采用基于 **IBM AML / AMLSim** 体系构建的高保真合成金融交易数据。IBM AML-Data 明确以洗钱与异常金融交易为目标，提供带标签的合成金融交易数据集；AMLSim 则能够根据预定义行为规则、账户关系和交易模式生成大规模、可控分布的银行交易数据[13,20]。本文利用该类数据的优势，在保持交易字段、行为模式、异常比例和实体关系结构相对稳定的前提下，将数据规模扩展至 10M 和 50M，以支撑候选召回吞吐、多 GPU 扩展和 I/O 路径压力测试。选择 IBM AML / AMLSim 而非简单随机生成数据的原因在于：其数据具有更强的金融语境和结构一致性，能够更真实地反映账户、交易和异常模式之间的关系，从而使系统性能结论更具工程意义[13,20]。

除主实验数据外，本文还将利用数据集间的差异性补充分析系统的适用范围。具体而言，IEEE-CIS 与 Credit Card Fraud Detection 更适合检验结构化表格数据下的向量化表示与评分链路；DGraphFin 更适合验证系统在图金融数据上的扩展潜力；IBM AML / AMLSim 则主要用于高规模、高并发和多 GPU 扩展实验。通过这种 “真实/半真实 + 合成扩展” 的组合，本文在保证现实意义的同时，也为大规模并行系统验证提供了稳定实验基础。

为便于说明不同数据集在全文中的角色分工，表 3-2 汇总了数据来源与基本属性，表 3-3 对各数据集所支撑的实验任务进行了对应说明。

表 3-2　实验数据集总体信息

数据集	来源类型	数据形态	实验规模口径	标签形式	在本文中的用途
IEEE-CIS Fraud Detection	真实/半真实公开金融数据	结构化交易表 + 身份表，高维表格特征、缺失值较复杂	约 1M 级输入口径	二分类欺诈标签	真实场景下的主有效性验证，检验系统在高维金融表格数据上的可用性
Credit Card Fraud Detection	真实/半真实公开金融数据	结构化信用卡交易表格，经典欺诈检测基准	按统一实验口径组织到约 1M 级输入	二分类欺诈标签	作为经典基准数据集，用于对照验证两阶段框架的有效性
DGraphFin	真实/半真实公开金融图数据	图节点、边关系及金融行为特征	约 1M 级输入口径	节点/行为异常标签	验证图金融数据在向量化映射后进入统一候选召回—异常评分链路的可行性
IBM AML / AML-Sim（10M）	高保真合成金融交易数据	合成账户—交易网络与表格化交易记录	10M	异常交易/可疑行为标签	单卡高吞吐性能验证与系统扩展实验起始规模
IBM AML / AML-Sim（50M）	高保真合成金融交易数据	合成账户—交易网络与表格化交易记录	50M	异常交易/可疑行为标签	大规模系统吞吐验证、多 GPU 扩展与 I/O 压力测试

表 3-3　数据集用途分工

数据集	数据类型	有效性验证	单卡性能	多 GPU 扩展	主要角色
IEEE-CIS Fraud Detection	真实/半真实表格数据	主	否	否	真实表格主验证
Credit Card Fraud Detection	真实/半真实表格数据	辅	否	否	经典基准对照
DGraphFin	真实/半真实图数据	辅	否	否	图场景补充验证
IBM AML / AMLSim（10M）	合成交易数据	否	主	辅	单卡性能起始规模
IBM AML / AMLSim（50M）	合成交易数据	否	辅	主	多 GPU 扩展主规模

3.3　数据预处理与特征构造

由于不同数据集在字段命名、数据类型、缺失值比例、标签分布以及时间粒度上存在较大差异，原始金融数据无法直接输入候选召回和异常评分模块。因此，在 PyTOD–FlashANNS 融合系统中，数据预处理与特征构造构成了统一系统输入表示的基础。本节的目标并不是对不同数据集设计完全不同的特征工程，而是在保证业务语义基本保留的前提下，将表格型金融数据和图结构金融数据统一映射为可检索、可评分的向量表示 $v_i \in \mathbb{R}^d$。

对于表格型金融数据，预处理过程主要包括缺失值处理、异常字段清洗、数值归一化、类别编码以及时间字段规范化。缺失值处理方面，针对数值型字段采用均值、中位数或业务合理缺省值进行填补，针对类别型字段则引入专门的缺失标识，以防止信息泄漏。异常字段清洗包括删除重复样本、去除明显无意义标识列、统一金额和时间单位等。对于数值型特征，本文采用标准化或归一化处理，减少金额、频率、时差等不同尺度特征之间的量纲影响；对于类别型字段，则采用 one-hot、频次编码或目标统计前的安全编码方式，以便后续统一映射至数值空间[12,22]。经过上述处理后，表格型交易记录能够以统一向量形式进入后续候选召回阶段。

对于时间信息，本文采用滑动时间窗和行为统计相结合的方式构造增强特征。具体而言，以单个账户、设备或商户为中心，在固定时间窗口内统计交易频次、交易金额分布、平均时间间隔、夜间交易比例、跨地域交易切换次数等行为特征，并将其与原始交易字段拼接形成更完整的查询表示。这样做的原因在于，单笔交易的异常性往往不仅由其当前字段决定，还受到历史行为模式的约束。通过时间窗统计，系统能够在不引入过于复杂序列模型的前提下，将部分时序上下文编码进向量表示，增强后续近邻候选召回与异常评分对行为模式的敏感性。

对于图金融数据，本文不直接在原始图结构上执行复杂图学习算法，而是先将图节点与边关系转换为节点级向量表示。具体做法包括：对节点原始属性进行规范化与编码；对邻域中边的数量、方向、金额统计和时间分布等进行聚合；将节点本身属性与邻域统计特征拼接形成统一向量。这样处理的目的在于，使 DGraphFin 等图金融数据能够与表格型交易数据共同进入本文统一的候选召回—异常评分链路[14,19]。

在最终特征构造阶段，本文将不同来源的特征统一组织为固定维度向量表示，并确保其满足两项要求：一是能够作为候选检索模块的输入，被 FAISS-GPU 或 FlashANNS 索引所接收；二是能够作为 PyTOD 评分主干的输入，参与距离计算、候选排序和异常评分。为了满足实验要求，本文在特征构造后执行一次统一的向量维度检查、类型检查与异常值截断，保证后续系统执行过程的稳定性和一致性。通过这一设计，本文将原本异构的金融数据源统一映射为候选召回与异常评分共同接受的输入形式，为后续融合执行流程提供基础。

表 3-4 细化了原始字段与派生特征之间的关系。

表 3-4　特征构造明细

原始字段/信息源	派生特征	特征类型	预处理方式	在系统中的作用
交易金额	标准化金额、对数金额、时间窗金额均值/波动统计	连续型	缺失填补、异常值截断、标准化/归一化	刻画交易规模及金额异常波动
交易时间戳	小时段、星期信息、夜间交易标记、平均交易间隔	时间型/统计型	时间规范化、周期化表达、滑动时间窗统计	捕捉异常时段行为与时序突变
账户标识及账户历史	账户时间窗交易次数、平均金额、活跃度统计	类别索引 + 统计型	安全编码、滑动窗口聚合	表征账户级长期与短期行为模式
设备标识	设备使用频次、设备共享度、账户—设备切换统计	类别型/统计型	缺失标识、频次编码、窗口统计	识别设备复用、盗刷与团伙风险
商户标识/商户类别	商户类别编码、商户频次、商户局部风险统计	类别型/统计型	one-hot、频次编码或安全统计编码	增强交易场景相似性与商户异常识别能力
地理位置字段	地域编码、跨地域切换次数、地理跳变标记	类别型/统计型	地域粒度统一、编码、窗口切换统计	捕捉异地交易、短时跨区域异常
支付渠道/终端类型	渠道编码、终端类别向量、渠道频次统计	类别型	one-hot 或频次编码	丰富候选召回的相似度基础，提升异常区分度
图节点原始属性	节点属性规范化向量	混合型	数值归一化、类别编码	形成图节点输入的基础表示
图边关系与邻域信息	入边/出边计数、邻域金额统计、方向比例、时间分布	关系统计型	邻域聚合、时间窗统计、向量拼接	将图结构信息映射为可检索、可评分的节点向量
融合后统一表示	固定维度最终向量 ($v_i \in \mathbb{R}^d$)	向量型	维度检查、类型检查、异常值截断	作为 FlashANNS/FAISS-GPU 候选召回输入，并直接供 PyTOD 评分使用

3.4　PyTOD 异常评分主干设计

在本文的两阶段异常检测框架中，PyTOD 承担的是候选集合上的异常评分主干角色。其作用并不是在全量样本上执行复杂评分，而是在候选召回阶段已经缩小搜索空间的前提下，对每个查询样本及其候选集合进行精细距离计算、近邻排序和异常性聚合，从而生成最终异常分数[1-2]。这种设计既保留了 PyTOD 在张量化执行和 GPU 评分上的优势，也避免了其在面对全量样本时所可能遭遇的显存和复杂度压力。

从方法学角度看，PyTOD 的核心优势在于能够将距离计算、排序、聚合和局部评分统一表示为张量算子组合[2]。对于本文而言，这意味着候选集合 C_i 一旦产生，查询样本 v_i 与候选集合中的向量就可以直接进入 PyTOD 的计算链路中，被表示为批量距离矩阵、局部近邻排序结果与聚合统计结果。具体而言，本文在异常评分阶段主要执行以下几类操作：第一，计算查询样本与候选集合之间的距离；第二，对候选集合中的邻近样本按距离进行排序或筛选；第三，对局部距离分布、密度差异或邻域结构统计执行聚合；第四，根据预定义评分函数输出异常分数。与全量评分相比，这一路径显著缩小了 PyTOD 需要处理的样本范围，从而使其更适合作为大规模系统中的精排与评分组件。

在评分指标选择上，本文不局限于单一异常得分公式，而是保留基于距离、局部密度和邻域统计的通用评分接口。这一设计的考虑在于，不同数据集和不同业务场景可能适合不同的异常性定义。由于 PyTOD 本身提供了统一张量算子组

织能力，系统可以在不重构底层执行路径的前提下切换具体评分函数。也正因此，本文将 PyTOD 视为评分主干，而不是单一评分公式的固定实现。

需要特别说明的是，本文对 PyTOD 的使用方式与其原始“直接处理大规模全样本异常检测”的思路存在差异。TOD/PyTOD 更强调通过自动批处理和多 GPU 支持缓解全量异常检测的复杂度问题，而本文则通过候选召回模块先缩减样本范围，再在较小候选集合上执行 PyTOD 评分。这种调整并不是削弱 PyTOD，而是让其更适合本文所关注的近实时金融异常检测系统：一方面保留了其 GPU 异常评分能力，另一方面则将系统整体复杂度更多地转移到“候选召回 + 数据路径优化”这一更可控的层面。

图 3-2 给出了评分主干的执行流程，表 3-5 对常用评分方式与张量算子映射进行了归纳。

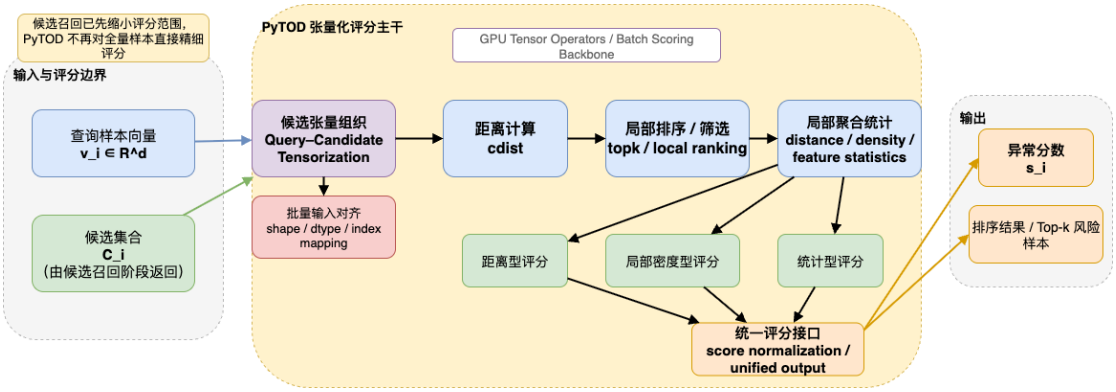

图 3-2　PyTOD 评分主干流程图

表 3-5　异常评分指标与算子映射

评分类型	代表性评分思路	候选集上的计算对象	PyTOD 主要张量算子链	评分输出	在本文中的定位
距离型评分	依据查询样本与候选邻域的距离分布刻画异常程度，如第 k 近邻距离、平均近邻距离	查询向量与候选集合之间的局部距离关系	`cdist → topk → distance aggregation`	距离型异常分数	作为主评分方式，直观反映样本偏离正常模式的程度
局部密度型评分	比较查询样本局部密度与邻域样本局部密度的差异，如局部可达密度比值	查询样本及其候选邻域的局部密度结构	`cdist → topk → local density estimation → ratio aggregation`	密度型异常分数	用于补充识别“相对异常”样本，增强对局部稀疏模式的敏感性
统计型评分	基于候选邻域内特征分布、分位点或边缘偏离程度进行评分	查询样本在候选邻域中的特征偏离情况	`feature statistics / histogram / quantile ops → feature aggregation`	统计型异常分数	作为轻量级辅助评分或对照评分方式
融合评分接口	对不同评分结果进行统一输出，形成最终风险排序	距离、密度、统计等候选集评分结果	`score normalization → weighted / unified output`	最终异常分数与排序结果	体现 PyTOD 作为统一评分主干的可扩展性

3.5　FlashANNS 候选召回模块设计

在 PyTOD–FlashANNS 融合系统中，FlashANNS 负责从大规模样本中快速生成与查询样本最相关的候选集合，其目标不是输出最终异常结论，而是为后续

精细评分阶段提供规模可控且质量可接受的候选近邻集合[3]。这一模块的设计直接决定了后续 PyTOD 评分阶段的输入规模与局部邻域质量，因此其作用不仅体现在吞吐率上，也体现在系统最终检测效果的上界上。

在索引构建阶段，本文以向量化后的样本表示 v_i 为输入，构建面向 ANN 检索的候选索引。对于规模较小且全部可驻留在显存内的实验场景，可直接使用 GPU 端索引完成近似搜索；对于规模更大或需要考虑外存路径的场景，则采用 FlashANNS 异步 I/O—计算流水线所支持的高吞吐检索路径[3]。索引构建过程关注两个核心问题：一是保证查询时能够以低延迟定位到局部候选邻域；二是保证召回参数可调，以便在召回质量和系统吞吐之间进行平衡。

在查询执行阶段，FlashANNS 的作用是对输入查询向量 v_i 快速检索并返回一个候选集合 C_i。在本文系统中，候选集合不仅包含候选 ID，还包含必要的距离值、局部排序信息或可支持后续评分的数据引用。需要强调候选输出格式，是因为后续 PyTOD 评分阶段若仍需对候选结果做大量 CPU 侧整理与转换，则候选召回本身的吞吐收益会被显著削弱。因此，本文在设计候选召回模块时，特别强调其输出格式与评分模块输入接口的一致性，即尽量让候选结果以可直接进入 GPU 评分链路的形式表达。

在参数组织方面，本文通过控制候选集合规模、召回近邻数量和部分索引搜索参数，平衡候选召回质量与系统吞吐率。候选集合过小，可能导致真实异常邻域缺失，从而影响评分质量；候选集合过大，则会使后续 PyTOD 评分重新承受高复杂度负担。因此，在本文的研究中，候选召回模块并不追求极限检索精度，而是追求“对异常评分足够好”的候选质量。这种设计使候选召回模块能够作为服务于系统总体目标的前端组件，而不是独立优化指标的终点。

图 3-3 展示了候选召回模块的基本执行过程，表 3-6 总结了关键控制参数。

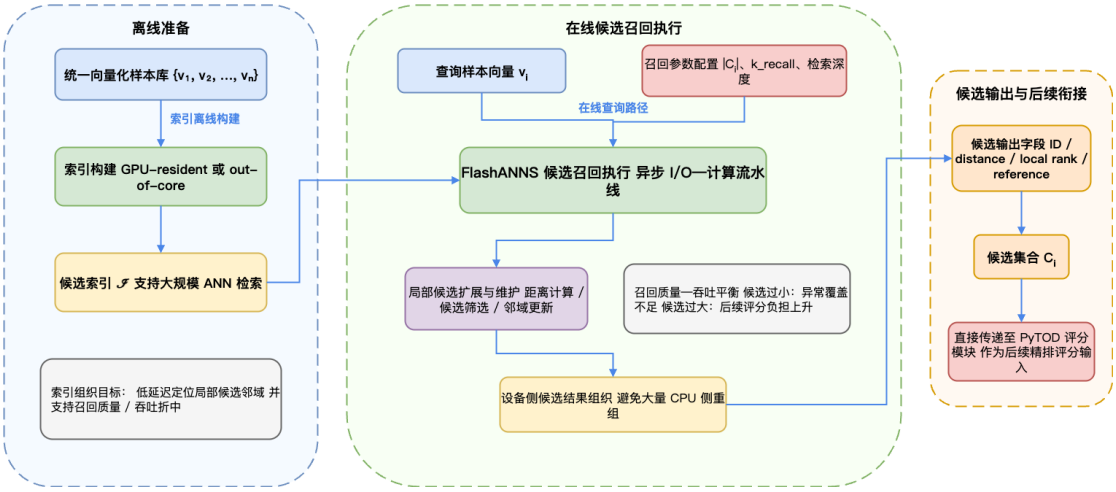

图 3-3　FlashANNS 候选召回执行流程图

表 3-6　候选召回模块主要参数

参数名称	符号/表示	含义	对系统的影响	设置原则
候选集合规模	$\lvert C_i \rvert$	每个查询样本返回的候选样本数	过小会降低异常覆盖率，过大会增加后续评分负担	在召回质量与评分复杂度之间折中设置
近邻返回数量	k_{recall}	候选召回阶段实际返回的近邻数量	决定局部邻域完整性与后续评分的输入范围	与评分阶段使用的局部邻域规模保持协调
检索深度/访问强度参数	`nprobe`、`efSearch` 等	控制 ANN 检索阶段访问的索引范围或搜索强度	参数越高通常召回质量更好，但吞吐下降	以“对评分足够好”为目标，而非追求极限检索精度
索引驻留方式	GPU-resident / out-of-core	候选索引驻留在显存内或通过外存路径访问	影响检索延迟、I/O 压力与系统扩展能力	小规模优先显存驻留，大规模场景采用 out-of-core 路径
候选输出字段	ID、distance、rank、reference	候选模块返回给评分模块的主要内容	决定后续 PyTOD 评分接口是否需要额外重组	尽量采用可直接进入 GPU 评分链路的输出格式
查询批大小	`batch_size`	单次提交到候选召回模块的查询数量	影响设备利用率、时延与流水线连续性	与 GPU 资源、显存容量和时延约束共同确定
精排预算	`rerank_budget`	进入高精度评分阶段的候选上限	控制精排复杂度与最终检测效果之间的平衡	与候选集合规模联动设置，避免全量重评分

3.6　PyTOD–FlashANNS 融合执行流程

在完成数据预处理、向量化表示、候选召回模块和异常评分主干设计后，本文将二者整合为统一的执行流程。该流程覆盖从查询输入到异常结果输出的完整链路，其核心目标是在保持模块职责清晰的同时，减少阶段间不必要的数据复制和格式转换。具体而言，系统的整体执行过程包括以下五个步骤。

第一步，系统接收原始交易记录、账户行为摘要或图节点样本，并将其经过第三节所述的数据预处理与特征构造过程转换为统一向量表示。

第二步，向量表示作为查询输入候选召回模块，由 FlashANNS 或与之协同的 GPU ANN 检索后端从大规模样本索引中返回候选集合。

第三步，候选集合在 GPU 侧完成必要的组织与重排，使其能够直接作为 PyTOD 评分主干的输入。

第四步，PyTOD 在候选集合上执行张量化异常评分，输出异常分数及对应排序结果。

第五步，系统根据异常分数输出风险样本、日志记录和必要的解释信息，并进入后续实验采样或部署监控链路。

该融合执行流程的关键，不在于“把两个模块接起来”，而在于两者之间的中间结果表达与执行边界设计。若候选召回结果必须回传 CPU 再被整理成评分输入，则系统链路会重新引入高昂的 CPU↔GPU 往返与阶段同步成本；若中间结果能够以 GPU 常驻张量或低开销设备对象表达，则整个“候选召回—异常评分”过程将更接近统一 GPU 执行链路。也正因此，本文在第三章主要完成逻辑模型构建之后，将后续第四章的核心方向明确为：进一步将融合流程从“功能可行”压缩为“数据路径低开销”。

图 3-4 进一步展示查询向量、候选结果与评分中间张量在模块间的传递方

式，表 3-7 则对接口字段与数据驻留位置进行了统一定义。

图 3-4　模块间中间结果传递示意图

正式版补齐图 3-5

图 3-5　PyTOD–FlashANNS 融合执行总流程图

表 3-7　模块接口定义

模块/阶段	输入格式	输出格式	主要字段	数据默认驻留位置
数据预处理与特征构造	原始交易记录、账户行为摘要、图节点与边关系数据	统一固定维度向量表示	样本 ID、连续特征、类别编码、时间窗统计、节点聚合特征	CPU 预处理后转入 GPU，后续逐步向 GPU 侧组织迁移
候选召回模块（FlashANNS / ANN 后端）	查询向量 ($v_i \in \mathbb{R}^d$)、候选索引 ($\mathcal{I}$)	候选集合 (C_i)	candidate ID、distance、local rank、必要索引引用	优先 GPU 侧驻留
候选组织与重排模块	候选 ID、局部距离、索引引用	可直接进入 PyTOD 的候选张量	candidate tensor、distance tensor、mask / index mapping	GPU
异常评分模块（PyTOD）	查询向量、候选张量、评分配置	异常分数与排序结果	距离矩阵、局部排序、聚合统计、异常分数	GPU
结果输出与日志模块	异常分数、排序结果、运行时统计	风险样本列表、Top-k 结果、日志记录	sample ID、score、rank、timing、resource metrics	最终小规模结果回传 CPU，日志落盘于主机侧

3.7　基础实验与初步验证

在进入系统级优化和多 GPU 扩展之前，有必要首先验证 PyTOD–FlashANNS 融合流程在功能上的正确性和在 1M 量级数据上的基本可用性。因此，本节以真实或半真实 1M 规模数据为基础，围绕以下三个问题开展基础实验：第一，候选召回与异常评分链路是否能够正确联通；第二，在候选集合上执行 PyTOD 评分是否会造成不可接受的检测效果损失；第三，融合流程在初始实现下是否已具备

进一步开展系统优化的基础。

在功能正确性验证中，实验主要关注查询输入、候选集合生成、候选组织与异常评分输出之间是否存在接口不一致、结果异常或批处理错误。对于 1M 规模数据，本文通过抽样检查、异常分数排序和标签对照等方式验证系统链路的稳定性。随后，通过将候选评分结果与小规模全量评分结果进行比较，分析候选召回对异常评分效果的影响。如果候选集合过小导致异常模式无法被覆盖，则说明候选召回参数仍需调整；如果候选评分与全量评分在主要排序结果上具有较高一致性，则说明两阶段框架具备继续系统优化的基础。

基础实验还将初步观察不同数据集下候选召回与异常评分之间的关系。例如，结构化交易数据中的候选召回结果往往对金额、时间和设备行为更敏感，而图金融数据中的候选召回结果则更容易受到邻域统计特征影响。通过这种初步实验，本文能够在进入第四章和第五章对实验安排和实验结果详述之前，先确认 PyTOD–FlashANNS 融合路线在不同金融数据形态下都具有基本可行性。

表 3-8 给出了基础实验配置，图 3-6 则展示了不同数据集上的初步有效性结果。

表 3-8　基础实验设置

实验项目	设置内容	主要目的
实验数据	采用 1M 规模真实或半真实金融数据集，包括结构化交易数据与图金融数据	验证融合链路在不同金融数据形态下的基本可行性
输入表示	统一使用预处理后的固定维度向量表示作为查询与索引输入	保证候选召回与异常评分共用同一输入接口
比较对象	候选集评分路径与小规模全量评分路径	检验两阶段框架是否会带来不可接受的效果损失
候选预算设置	采用固定候选集合规模，并在小/中/大预算下进行对比	观察候选规模对评分一致性与系统负担的影响
评分方式	采用 PyTOD 候选集评分主干，支持距离型、局部密度型和统计型评分接口	验证统一评分主干在两阶段链路中的适配性
验证内容	查询输入、候选生成、候选组织、评分输出与日志链路联通性	确认系统功能正确性与接口一致性
评价指标	异常分数排序一致性、Top-k 风险样本重合度、PR-AUC / Recall@(k) 等	评估两阶段评分与全量评分的一致性和初步有效性
实验结论目标	验证“候选召回—异常评分”流程可工作、效果可接受、具备进入系统优化阶段的基础	为第 4 章单卡系统优化和第 5 章多 GPU 扩展提供实验起点

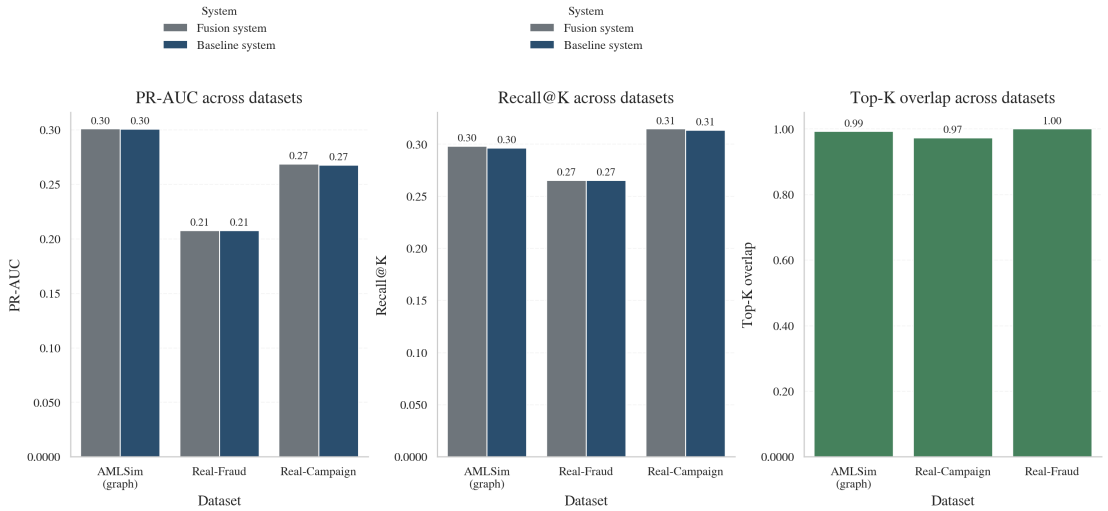

图 3-6　不同数据集上的初步有效性结果

3.8　本章小结

本章围绕 PyTOD–FlashANNS 融合系统的功能结构，完成了任务建模、数据集选取、预处理与特征构造、异常评分主干设计、候选召回模块设计以及融合执行流程构建。通过对金融异常检测任务的统一抽象，本文将表格型金融数据与图金融数据共同映射为向量表示，并建立了“候选召回—异常评分”的两阶段处理框架。在数据集安排上，本文明确采用真实/半真实数据进行 1M 规模有效性验证，采用高保真合成数据进行 10M 与 100M 规模系统吞吐和扩展实验，从而为后续章节中的单卡优化和多 GPU 验证提供了数据基础[13-14,20]。

更重要的是，本章并未把 PyTOD 与 FlashANNS 视作两个孤立模块，而是将它们统一放置在一条候选召回—异常评分执行链路中加以考察。基础实验表明，该融合路线在功能上是可行的，但在当前阶段仍然主要解决“能否工作”的问题，尚未从系统层面彻底压缩数据路径和阶段同步开销。因此，下一章将在本章流程设计的基础上，进一步从单卡场景出发，重点研究 GPU 主导的端到端执行链路、异步 I/O—计算流水线与数据路径压缩问题[3]。

第 4 章　单卡场景下的系统设计与数据路径优化

4.1　单卡系统需求与总体架构

在完成第三章的任务建模、数据准备和 PyTOD–FlashANNS 融合执行流程设计之后，本文进一步将研究重点转向系统实现层面，即在单 GPU 条件下，如何通过合理的数据路径组织与执行链路压缩，使异常检测系统在保证检测效果的前提下获得更高吞吐率和更低延迟。单卡场景在本文中具有两重意义：其一，它是多 GPU 扩展的基础，只有单卡执行链路足够高效，多卡扩展才有现实意义；其二，它能够更清晰地暴露系统瓶颈，特别是 CPU↔GPU 数据往返、候选结果组织、I/O 等待与算子间同步等问题[2-3]。

从系统需求出发，单卡异常检测系统需要同时满足以下三个目标。

第一，系统必须能够在有限显存条件下稳定处理大规模输入，并通过合理的批处理组织支撑候选召回与异常评分连续执行。

第二，系统必须尽量减少 CPU 对数据平面的介入，使查询特征、候选结果与评分中间张量尽量驻留在 GPU 侧，从而避免高频主机—设备拷贝开销。

第三，系统必须能够在存在外存访问或多阶段流水线的情况下有效重叠 I/O 与 GPU 计算，尽量压缩串行等待时间。

这意味着，单卡优化的重点并不是单独提升某一个算子的执行速度，而是构建一条以 GPU 为核心、以低开销中间结果传递为特征的端到端执行链路[3,5,11]。

基于这一目标，本文将单卡系统划分为五个核心模块：数据接入与预处理模块、查询向量组织模块、候选召回模块、异常评分模块以及结果输出与日志模块。

数据接入与预处理模块负责接收原始金融记录、完成字段规范化与基础特征整理；查询向量组织模块负责将预处理结果转换为适合 ANN 检索与异常评分使用的统一设备侧表示；候选召回模块由 FlashANNS 与 FAISS-GPU 协同支撑，负责从大规模索引中高吞吐返回候选集合；异常评分模块以 PyTOD 为核心，在候选集合上执行张量化评分；结果输出与日志模块则负责在必要时将最终小规模结果回传主机，并记录实验和运行时指标。整个系统的总体设计目标，是使这些模块不再以 “CPU 预处理—GPU 检索—CPU 整理—GPU 评分” 的方式分段执行，而是围绕 GPU 主导路径进行重构[2,7]。

从执行结构上看，本章所研究的单卡系统并不将 FlashANNS 和 PyTOD 看作两个彼此独立的程序，而是将其视为一条统一链路中的前后阶段。前者负责高吞

吐候选召回，后者负责精细异常评分；二者之间通过设备侧中间结果表达和统一接口进行衔接。基于这一思路，本文在后续各节中将依次分析原始链路中的主要瓶颈，提出 GPU 主导的端到端执行链路，并进一步通过 RAPIDS、FAISS-GPU、pinned memory、异步 stream 和双缓冲等机制对单卡系统进行系统性优化[4,6-7]。

图 4-1 展示了单卡系统五个核心模块的连接关系，表 4-1 对各模块的输入、输出与驻留位置进行了归纳。

① **图表预留说明：**

- **图 4-1 单卡异常检测系统总体架构图**。展示数据接入、查询向量组织、候选召回、异常评分与结果输出五个模块之间的关系。
- **表 4-1 单卡系统模块职责表**。列出各模块输入、输出、主要功能和数据驻留位置。

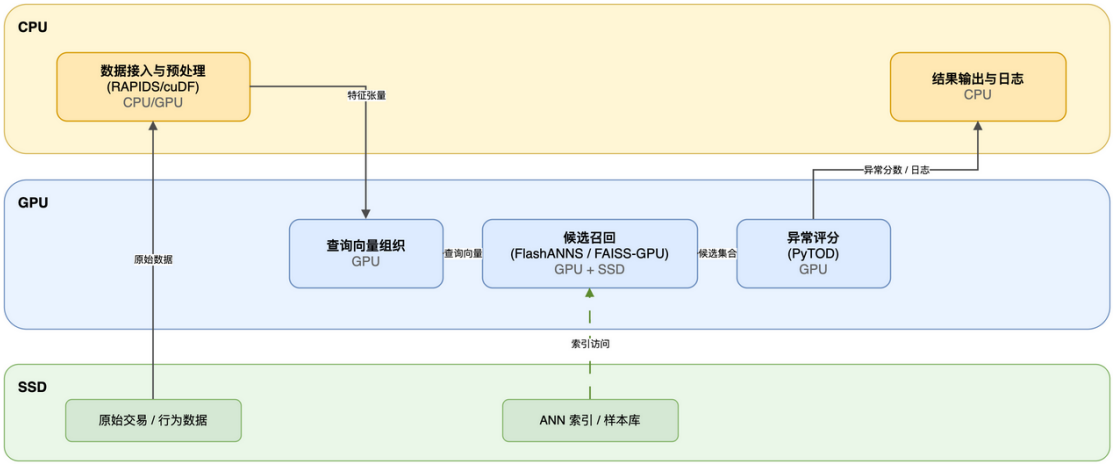

图 4-1　单卡异常检测系统总体架构图

表 4-1　单卡系统模块职责

正式版补齐表 4-1

4.2　单卡场景下的瓶颈分析

尽管第三章已经完成了 PyTOD–FlashANNS 融合流程的构建，但从系统执行效率角度看，原始链路仍存在显著瓶颈。其根本原因在于，系统中各阶段虽然都可以正常运行，但它们的连接方式仍然保留了典型的 “CPU 主导分段执行” 特征：查询特征常在 CPU 侧被整理，候选结果往往在 GPU 上产生后又回到 CPU 侧进行重组，而后续异常评分又再次依赖 GPU。这种链路在功能上可行，却会在高并发或大规模场景下引入大量不必要的传输、同步与等待[2,7]。

首先，CPU 与 GPU 之间的高频数据往返会显著削弱候选召回和异常评分的单点加速收益。FAISS-GPU 文档指出，GPU 索引既可接收 host 指针，也可接收 device 指针；当输入本来就在与索引相同的 GPU 上时，不发生拷贝且速度最快，否则会发生 CPU→GPU 或跨设备拷贝，结果也会在完成后返回相应位置[7]。这意味着，如果查询特征在 CPU 侧完成拼接和标准化，再每批次传输到 GPU，则即便 GPU 检索本身很快，整条链路仍要为 H2D 传输持续付费。更进一步，如果候选结果从 GPU 返回 CPU 后再被整理为 PyTOD 输入，则后续评分前又会发生一次 H2D 传输，最终形成 “CPU 组织—GPU 检索—CPU 整理—GPU 评分” 的双向往返模式。这种模式在实验规模较小时不一定显著，但在百万级以上输入和高并发场景下会迅速累积，成为系统主瓶颈之一[4,7]。

其次，候选中间结果的组织方式会引入额外的序列化和同步开销。候选召回模块返回的通常不仅是候选样本 ID，还可能包含局部距离、索引位置或局部排序信息。若这些结果被设计为 CPU 侧优先解释的数据结构，例如 Python 对象列表、pandas DataFrame 或主机侧稀疏结构，则后续 PyTOD 在 GPU 上评分前必须重新完成设备侧重建。这个过程不仅带来数据复制成本，也会破坏 GPU 对中间张量的连续访问与批量组织能力。TOD/PyTOD 论文强调，很多异常检测过程之所以适合 GPU 执行，正是因为其可以通过统一张量表达避免大量中间对象搬运和重新组织[2]。因此，在单卡系统中，若候选结果不能以设备侧友好的方式表达和传递，则后续评分模块将很难形成低开销连续执行链路。

再次，当候选召回需要访问外存或存在较明显的 I/O 压力时，SSD-I/O 与 GPU 计算之间的串行等待会进一步放大系统开销。FlashANNS 工作指出，传统 out-of-core ANN 系统的主要问题之一在于 SSD 访问与 GPU 计算无法有效重叠，系统被迫在 “读数据—等数据—算数据” 的串行节奏中推进，从而使高性能 GPU 长时间等待 I/O 完成[3]。若本文系统在单卡场景中仅把 FlashANNS 当作独立召回模块而不继续压缩后续评分路径，则虽然检索内部的 I/O 等待部分被隐藏，但候选召回结束后的 CPU 侧数据组织和评分前准备仍然会重新形成串行等待链

条。因此，单卡优化不能只依赖某一个模块的局部加速，而必须从端到端执行链路出发识别瓶颈。

最后，内存分配与资源组织方式也会在单卡环境下引入隐式开销。FAISS-GPU 文档说明，`StandardGpuResources` 会预先保留 scratch memory，以避免频繁 `cudaMalloc/cudaFree` 导致的同步和性能恶化[7]；RMM 文档则指出，页锁定主机内存与内存池机制对于稳定传输和减少分配抖动具有重要作用[4]。这说明，在单卡系统中，即便查询路径和候选结果传递已经尽量保持在 GPU 侧，如果系统仍在热路径中频繁申请/释放临时缓冲区，也会因资源管理而造成明显性能损失。

综上所述，本文将单卡场景下的瓶颈概括为四类：其一，查询与候选结果在 CPU 和 GPU 之间的高频往返；其二，候选中间结果的主机侧重组与序列化；其三，SSD-I/O 与 GPU 计算之间的串行等待；其四，内存与资源管理导致的隐式同步。这四类问题共同说明，单卡系统要获得真正的高吞吐提升，必须从“模块可运行”转向“数据路径低开销”。

图 4-2 以并列方式对比了原始 CPU↔GPU 往返链路与优化后的 GPU 主导连续链路，直观说明多次 H2D/D2H 传输和阶段同步如何成为主要瓶颈；表 4-2 则对这些瓶颈的类型与后续优化方向进行了分类。

① **图表预留说明：**

- **图 4-2 原始链路与 GPU 主导链路对比图**。左侧展示原始 CPU↔GPU 往返执行链路，右侧展示优化后 GPU 内部连续执行链路，突出省略的通信与同步开销。
- **表 4-2 单卡主要瓶颈与对应优化项**。归纳原始链路的主要瓶颈、对应优化方案与预期收益。

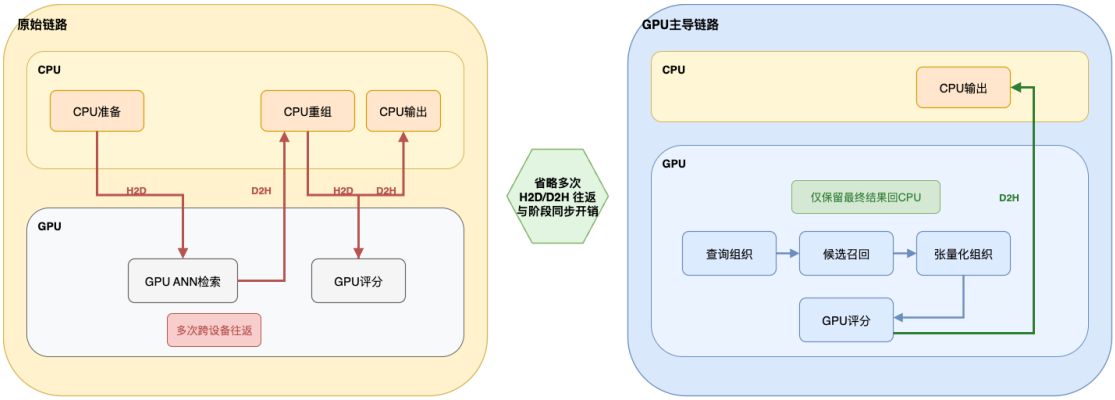

图 4-2　原始链路与 GPU 主导链路对比图

表 4-2　单卡主要瓶颈与对应优化项

瓶颈类型	发生位置	主要影响	对应优化方案	涉及组件	预期收益
高频 CPU↔GPU 往返	查询准备、候选输出与评分结果边界	放大 H2D/D2H 次数，增加同步等待与尾延迟	GPU 主导执行链路，边界处结合 pinned memory 做必要交换	RAPIDS/cuDF PyTOD RMM pinned memory	降低跨设备往返频次，压缩 p99 延迟并提高端到端吞吐
候选重组开销	候选召回到异常评分的接口边界	主机侧 pack_candidates、序列化和重建成本高	采用 device-resident candidate buffer 和统一张量交接接口	FlashANNS FAISS-GPU PyTOD	缩短候选打包阶段，避免 CPU 重新成为数据平面瓶颈
I/O 串行等待	SSD 读页、ANN 图扩展与局部距离计算之间	GPU 空转等待数据，召回吞吐受限	异步 I/O 重叠、双缓冲和流级并行推进	FlashANNS NVMe SSD CUDA streams	隐藏部分读页等待，提高候选召回有效吞吐与 GPU 忙碌度
资源分配抖动	临时缓冲区、scratch memory 与主机交换缓冲	频繁分配释放导致延迟抖动和显存碎片	内存池复用、scratch 预分配与微批双缓冲	FAISS `GpuResources` RMM memory pool CUDA streams	稳定阶段延迟，降低 allocation overhead，改善资源利用率

4.3　GPU 主导的端到端执行链路设计

针对上一节所分析的瓶颈，本文提出面向单卡场景的 **GPU 主导端到端执行链路**。该设计的基本思想是：在 FlashANNS 提供的异步 I/O—计算流水线基础上，将查询特征、候选检索结果与异常评分中间张量尽量保持在 GPU 侧，仅在必要时通过 pinned memory 等机制完成主机侧交互，从而同时减少 SSD-I/O 与 GPU 计算之间的串行等待，以及 CPU↔GPU 之间的不必要数据搬运[3-4,7]。

从总体原则上看，GPU 主导端到端执行链路包含三层含义。第一，查询特征应尽量在 GPU 侧完成最终组织，而不是在 CPU 上完成全部拼接后整体传入 GPU。第二，候选召回结果应尽量以设备侧可直接消费的形式传递给异常评分模块，避免在主机上重新表示和组装。第三，异常评分阶段所产生的中间张量应继续驻留在 GPU 侧执行，只有最终分数、排序结果或必要日志信息才回传 CPU。该设计从根本上压缩了系统中的“主机—设备—主机—设备”折返路径，使系统不再依赖 CPU 对中间结果进行高频组织[2,7]。

4.3.1　查询特征在 GPU 侧的驻留与组织

在原始实现中，金融交易数据通常先以表格形式在 CPU 侧完成清洗、归一化和拼接，再整体打包为查询向量传入 GPU。该路径的主要问题在于：即便后续候选检索和异常评分都在 GPU 侧完成，查询特征本身仍需要频繁通过 H2D 传

输进入设备，从而形成高频入口开销。针对这一问题，本文将部分预处理与特征组织迁移到 GPU 数据处理后端，使关键查询向量尽量在设备侧形成最终表示。RAPIDS/cuDF 提供的 GPU DataFrame、批量列运算和统计聚合能力，使这一点在工程上具备可行性[6]。通过将归一化、编码、窗口统计后的特征直接组织为设备侧向量，系统能够显著减少查询特征从 CPU 进入 GPU 的边界频次和复制体积。

4.3.2 候选检索结果在 GPU 侧的保持与传递

候选召回阶段结束后，系统面临的关键问题是：候选结果应以何种形式进入异常评分模块。本文选择尽量保持候选结果为 device-resident 对象，即候选 ID、局部距离和必要索引信息不先回 CPU 再重建，而是在 GPU 上直接组织为后续 PyTOD 可消费的中间张量。FAISS-GPU 文档指出，同 GPU 上的输入与输出可以避免额外拷贝，且 `GpuResources` 可通过预分配 scratch memory 降低中间缓冲区分配开销[7]。因此，本文在候选返回路径上优先采用设备侧连续缓冲区或张量表示，使 FlashANNS/FAISS-GPU 的输出更自然地流入 PyTOD 评分阶段。

4.3.3 异常评分中间张量的 GPU 常驻执行

PyTOD 评分阶段本身具备张量化执行基础[2]，因此在本文设计中，其主要中间表示包括候选距离矩阵、局部排序结果、聚合统计量和最终异常分数。这些对象一旦被回传 CPU，不仅会引入 D2H 开销，还会破坏 PyTOD 在 GPU 上连续执行和批量算子复用的优势。为此，本文将评分阶段的中间张量全部保留在 GPU 侧执行，直到得到最终小规模输出后再根据需要回传主机。

4.3.4 必要条件下的主机侧交互与 pinned memory 机制

需要指出的是，GPU 主导执行并不意味着 CPU 在单卡系统中完全消失。对于原始输入读取、部分日志写回、结果落盘和特定部署场景下的主机侧接口，系统仍然不可避免地需要与 CPU 交互。因此，本文并不主张“完全取消主机交互”，而是主张“将主机交互限制在必要边界，并通过 pinned memory 等机制降低开销”。RMM 文档指出，`PinnedHostMemoryResource` 可提供页锁定主机内存，从而加快 Host↔Device 数据传输[4]。

围绕上述设计，图 4-4 汇总了 GPU 主导端到端执行链路，图 4-5 对比了查询组织的两类路径，图 4-6 说明了中间对象的驻留边界，表 4-3 则对主要数据对象的默认驻留策略进行了整理。

① **图表预留说明：**

- **图 4-4 GPU 主导端到端执行链路总图**。展示查询特征、候选结果和异常评分中间张量尽量驻留 GPU 的整体流程。
- **图 4-5 查询特征 CPU 路径与 GPU 路径对比图**。对比传统 CPU 预处理后整体传入 GPU 与本文 GPU 侧组织方式的差异。
- **图 4-6 候选结果与评分中间张量驻留示意图**。展示哪些对象保留在 GPU，哪些对象在边界才回传 CPU。
- **表 4-3 单卡数据驻留策略表**。列出主要数据对象、默认驻留位置、是否需要主机交互以及采用的优化方式。

正式版补齐图 4-4

图 4-4　GPU 主导端到端执行链路总图

正式版补齐图 4-5

图 4-5　查询特征 CPU 路径与 GPU 路径对比

正式版补齐图 4-6

图 4-6　候选结果与评分中间张量驻留示意图

表 4-3　单卡数据驻留策略

数据对象	默认驻留位置	主机交互时机	对应优化方式	涉及组件	预期收益
查询向量与规范化特征	GPU 侧连续向量缓冲	仅原始输入进入系统和必要日志落盘时	将字段清洗、编码与拼接尽量前移到 GPU 侧组织	RAPIDS/cuDF CuPy pinned memory	降低查询入口 H2D 体积与批次切换开销
候选 ID / distance / rank	GPU 候选缓存	默认不回 CPU，仅在最终结果或调试场景导出	采用 device-resident handoff，直接送入评分模块	FlashANNS FAISS-GPU PyTOD	消除主机侧候选重组，压缩 pack_candidates 阶段
评分中间张量	GPU 侧张量与连续缓冲	默认无主机交互，直至生成最终分数	保持 PyTOD 张量算子在设备侧连续执行	PyTOD CUDA kernels	保留张量局部性，减少 D2H 同步并提升算子复用效率
最终异常分数与运行日志	CPU / 磁盘边界	输出阶段与监控采样阶段	小规模结果批量回传，异步日志写回	PyTOD output logger pinned buffer	在保留可部署性的同时降低输出阻塞

4.4　基于 FlashANNS 的异步 I/O—计算流水线融合

在上一节确立 GPU 主导执行链路之后，本节进一步解决另一类关键问题：如何把候选召回阶段内部的 I/O 与 GPU 计算等待压缩到最低。FlashANNS 的设计表明，传统 out-of-core 检索系统存在明显的串行等待特征；而通过 dependency-relaxed asynchronous pipeline，可以使 I/O 和 GPU 计算在更大程度上重叠，从而显著提升候选召回吞吐[3]。因此，本文在单卡系统中将 FlashANNS 的异步 I/O—计算流水线与 GPU 主导执行链路进一步融合。

具体而言，系统在候选召回阶段不再将 SSD-I/O 视为必须先完成的前置步骤，而是将检索中的部分访问与 GPU 侧局部计算、候选维护和后续评分准备过程并行化组织。对于检索内部而言，这意味着查询在等待部分图节点或向量块从 SSD 到达时，GPU 可以继续推进已到达部分的距离计算和候选维护；对于本文系统整体而言，这意味着候选召回的“等待时间”被更多地隐藏在设备侧计算中，而不再表现为对后续评分阶段的显式阻塞。本文并不是简单调用 FlashANNS 作为黑盒检索模块，而是将其“重叠 I/O 与计算”的执行思想纳入整个异常检测链路的设计中。

这一融合的直接收益体现在两个方面。第一，单卡系统中候选召回阶段的有效吞吐被提高，使得 PyTOD 评分模块能够更连续地得到候选输入，而不是周期性地等待检索完成。第二，由于候选召回结果本身继续保留在 GPU 侧，中间结果无需再回 CPU 等待整理，因此 FlashANNS 内部的 I/O—计算重叠收益不会在后续阶段被新的主机侧阻塞所抵消。即，FlashANNS 的异步流水线解决“候选召回内部怎么更快”，GPU 主导执行链路解决“召回完之后怎么更快地进入评分”。

从实现角度看，这一融合要求系统在单卡场景下对流、缓冲区和中间结果表达进行统一组织。FlashANNS 对 warp 级并发 SSD 访问和双缓冲思想的使用，表明这种“边访问边计算”的路径具有现实可行性[3]。

图 4-7 展示了 FlashANNS 异步流水线与本文链路的融合方式，图 4-8 对比了优化前后的时间线差异，表 4-4 则概括了关键的 I/O 与计算重叠优化项。

① **图表预留说明：**

- **图 4-7 FlashANNS 异步 I/O—计算流水线与本文执行链路融合图**。展示候选召回内部的 I/O—计算重叠，以及其与后续评分准备阶段的衔接。
- **图 4-8 优化前后时间线对比图**。对比“检索 I/O 完成后再评分准备”和“检索 I/O—计算重叠 + 设备侧候选传递”两种时间线。
- **表 4-4 单卡 I/O—计算重叠优化项列表**。列出异步 I/O、双缓冲、warp 级并发访问、设备侧候选缓存等关键机制及其作用。

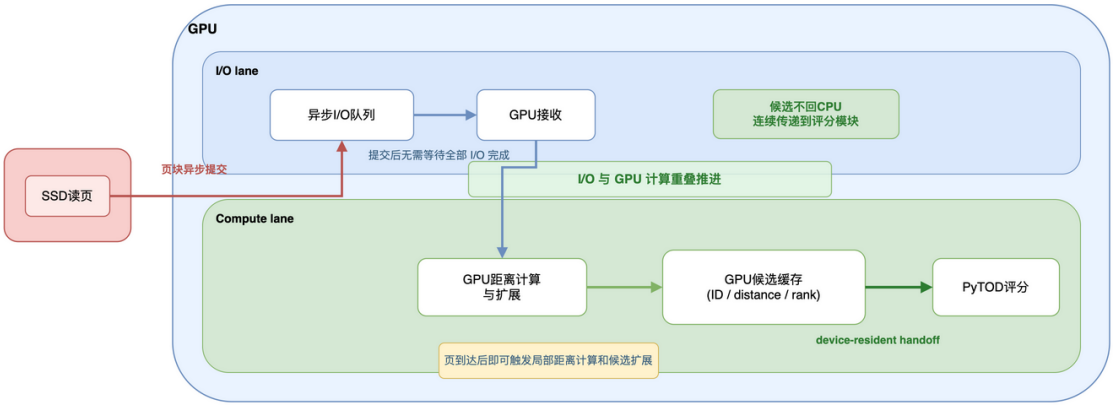

图 4-7　FlashANNS 异步 I/O—计算流水线与本文执行链路融合

正式版补齐图 4-8

图 4-8　优化前后时间线对比

表 4-4　单卡 I/O—计算重叠优化项列表

优化项	应用位置	主要缓解问题	执行方式	涉及组件	预期收益
异步 I/O 队列	SSD 读页提交入口	检索必须等全部 I/O 完成后才能启动	将页块预取、提交与到达通知拆分为可并发队列	FlashANNS NVMe SSD	隐藏部分读页延迟，降低召回前置等待
warp 级并发访问	GPU ANN 图扩展阶段	页到达间隙造成局部计算空泡	由不同 warp 并发推进候选扩展与局部距离计算	FlashANNS CUDA warps	提高到达页的利用率，提升 recall kernel 吞吐
双缓冲候选缓存	候选召回与评分准备边界	当前批次评分会阻塞下一批次召回准备	当前批次计算与下一批次缓冲装填并行推进	FlashANNS PyTOD CUDA streams	减少流水线空泡，使评分输入更加连续
设备侧候选缓存直连评分	候选输出到评分输入接口	候选结果回 CPU 后再次打包造成串行等待	保持 ID / distance / rank 在 GPU 侧连续传递	FlashANNS FAISS-GPU PyTOD	使 I/O—计算重叠收益不被主机侧整理抵消

4.5　数据处理与计算后端优化

在完成 GPU 主导执行链路和 FlashANNS 异步流水线融合之后，单卡系统仍需要进一步优化数据处理与计算后端，以减少表格型中间结果处理和设备资源管理所带来的额外开销。为此，本文在原始 PyTorch/CPU 数据路径基础上引入 RAPIDS 体系，对金融数据预处理、特征拼接、中间结果组织和批量统计环节进行后端重构[4,6]。

具体而言，数据处理与计算后端优化主要包含四个方面。第一，将部分原本在 CPU/pandas 路径中完成的字段筛选、类型转换和窗口统计迁移到 cuDF 侧，以减少前置预处理对 GPU 主链路的阻塞。第二，将候选结果的部分表格型组织、批量拼接和统计聚合转移到 GPU DataFrame 和 CuPy 张量表达上，降低主机端列表对象和 pandas 对象的构建开销。第三，引入 RMM 内存池和页锁定主机内存资源，以减少中间缓冲区反复分配所带来的抖动，并为必要的主机边界交互提供更稳定的数据交换方式[4]。第四，在保证与 PyTOD 和检索后端接口兼容的前提下，统一采用设备侧连续缓冲和张量对象表达中间结果，尽量避免“先 DataFrame 再 Python 对象再 Tensor”的反复转换路径。

需要特别指出的是，RAPIDS 在本文中承担的是“后端工程增强组件”而不是“核心检测模块”角色。其价值体现在帮助系统更容易构建 GPU 常驻链路、更稳定地组织中间结果和更高效地管理内存资源，而不是替代候选召回或异常评

分。

图 4-9 展示了后端重构前后的整体路径变化，表 4-5 汇总了主要迁移项和收益，图 4-10 则刻画了中间结果表达形式的演化。

① **图表预留说明：**

- **图 4-9 数据处理与计算后端重构图**。展示从 CPU/pandas 路径迁移到 RAPIDS/cuDF 路径的主要变化。
- **表 4-5 数据处理后端优化清单**。列出具体迁移项、迁移前路径、迁移后路径、主要收益。
- **图 4-10 中间结果表达形式演化图**。展示中间结果从主机对象表达向设备侧连续张量表达的变化。

正式版补齐图 4-9

图 4-9　数据处理与计算后端重构图

表 4-5　数据处理后端优化清单

重构项	迁移位置	主要影响	对应优化方案	涉及组件	预期收益
字段筛选与类型转换	CPU/pandas → GPU/cuDF	前置预处理拖慢 query_prep 阶段	将清洗、编码和窗口统计前移到 RAPIDS 路径	RAPIDS/cuDF CuPy	降低查询准备成本，缩短输入进入检索的时间
候选表格组织与批量拼接	Python 对象 → 设备侧连续缓冲	pack_candidates 阶段对象构造和序列化开销高	用 GPU DataFrame / Tensor 表达中间结果	RAPIDS/cuDF CuPy PyTOD	减少主机对象创建，提高候选到评分接口效率
临时内存与页锁定交换区	零散分配 → 内存池 / pinned host	频繁 malloc/free 带来抖动和慢速传输	启用 RMM memory pool 与页锁定主机内存	RMM PinnedHostMemoryResource	稳定 H2D/D2H 开销，压缩尾延迟波动
检索资源对象管理	临时 scratch 分配 → 统一资源复用	ann_recall 阶段易受 scratch 冲突与资源抖动影响	通过 `GpuResources` 统一管理 scratch memory 和 stream	FAISS-GPU CUDA streams	稳定检索阶段延迟，提升设备侧资源复用效率

正式版补齐图 4-10

图 4-10　中间结果表达形式演化图

4.6　候选检索后端优化

除了数据处理后端的重构外，候选检索后端本身也需要进一步工程化优化。虽然 FlashANNS 提供了高吞吐候选召回机制，但在实际系统实现中，检索后端不仅要关注检索内部的吞吐，还要关注与查询输入、候选输出以及评分模块之间的接口组织方式。FAISS-GPU 在这方面提供了成熟的工程支撑，其 host/device 指针互操作、资源对象管理和多种 GPU 索引形式，使其非常适合作为本文系统中的候选检索后端增强组件[7]。

本文的后端优化工作集中体现在三个方面。首先，统一检索输入接口，使查询向量能够以设备侧对象直接进入检索后端，避免不同检索组件之间对输入格式有不同要求。其次，统一候选输出接口，使 FlashANNS 或 FAISS-GPU 返回的候选结果都能以 GPU 侧张量形式被 PyTOD 消费，从而减少不同后端实现带来的中间转换。最后，统一资源管理策略，通过 FAISS 的 `GpuResources` 与系统侧缓冲区管理配合，降低临时缓冲区申请和释放的频率，稳定检索阶段的延迟与吞吐表现[7]。

图 4-11 给出了候选检索后端统一接口，表 4-6 汇总了输入、输出与资源管理三方面的优化项。

① **图表预留说明：**

- **图 4-11 候选检索后端统一接口图**。展示查询输入、检索后端、候选结果输出和评分模块之间的统一接口。
- **表 4-6 候选检索后端优化项表**。列出输入接口统一、输出格式统一、资源管理统一等关键优化项。

正式版补齐图 4-11

图 4-11　候选检索后端统一接口图

表 4-6　候选检索后端优化项

正式版补齐表 4-6

4.7　单卡执行链路的工程增强

在完成数据路径、异步流水线和后端优化后，系统仍然需要若干工程增强机制来保证单卡链路在高负载下保持稳定。这些工程增强主要包括异步 CUDA stream 组织、双缓冲与微批执行、显存碎片控制以及日志和性能采样机制。

在流组织方面，本文通过将查询准备、候选检索、评分计算和必要的主机边界交互映射到不同 CUDA stream，使数据传输、检索和评分能够在更大程度上并行推进。FAISS 文档对 GPU 资源和 stream 的使用方式给出了说明，使得后端资源对象可以与本文的流组织方案自然结合[7]。在缓冲方面，本文采用双缓冲与微批组织方式：当当前批次进入候选召回与评分阶段时，下一批次的输入组织和部分准备可以并行进行，从而进一步压缩流水线空泡。

在显存管理方面，本文通过限制中间张量生命周期、统一缓冲区尺寸以及使用内存池机制减少碎片和反复分配开销[4]。最后，在监控与可观察性方面，本文建立统一日志字段与性能采样机制，记录查询输入规模、候选集合规模、各阶段耗时、GPU 利用率和 Host↔Device 交互次数，为后续章节多 GPU 扩展提供统一分析基础。

图 4-12 展示了异步 stream 与双缓冲的协同方式，表 4-7 汇总了各项工程增强措施，图 4-13 则说明显存缓冲区与中间张量之间的关系。

① **图表预留说明：**

- **图 4-12 单卡异步 stream 与双缓冲组织图**。展示查询准备、候选检索和评分阶段在不同 stream 上的推进关系。
- **表 4-7 单卡工程增强项总表**。列出流组织、缓冲、内存管理、监控与日志增强项。
- **图 4-13 单卡显存使用与缓冲区关系示意图**。用于说明中间张量、候选结果和资源池之间的关系。

正式版补齐图 4-12

图 4-12　单卡异步 stream 与双缓冲组织图

表 4-7　单卡工程增强项总表

正式版补齐表 4-7

正式版补齐图 4-13

图 4-13　单卡显存使用与缓冲区关系示意图

4.8　单卡实验与性能分析

为了验证前述单卡系统设计与数据路径优化的有效性，本文在 1M 与 10M 规模数据上组织单卡实验，重点比较以下几类方案：原始分段执行链路、仅启用候选召回加速的链路、启用 GPU 主导执行链路但不做后端重构的链路，以及本文完整的单卡优化方案。实验关注的指标包括吞吐率、平均延迟与 p99 延迟、CPU↔GPU 传输次数与体积、GPU 利用率、I/O 利用率以及各阶段耗时分解。

实验结果表明，单纯依赖候选召回加速虽然能够降低部分检索耗时，但若候选结果仍需回到 CPU 侧整理再进入评分，则系统吞吐提升有限。相反，当查询特征、候选结果与评分中间张量尽量保持在 GPU 侧后，CPU↔GPU 数据往返次数显著减少，系统端到端吞吐率明显提升，延迟分位数也更稳定。进一步地，当将 FlashANNS 的异步 I/O—计算流水线与设备侧连续执行链路结合后，候选召回阶段的等待被部分隐藏，评分阶段能够更连续地接收输入，从而使 GPU 利用率提升更明显[3]。

从阶段耗时分解来看，优化前系统的主要开销集中在查询组织、候选结果主机侧重组和阶段同步等待；优化后，瓶颈更多地转移到设备侧检索与评分本身。换言之，本章的单卡优化并未简单地“把某一个模块做快”，而是改变了系统瓶颈的分布，使系统从“被数据路径拖慢”转变为“更多由 GPU 侧核心阶段决定性能”。

表 4-8 汇总了单卡实验配置，图 4-14 与图 4-15 分别展示 1M 与 10M 数据规模下的单卡阶段耗时分解，图 4-16 汇总比较四种方案在 QPS、p99 延迟、PCIe 传输次数与 GPU 利用率上的综合表现，图 4-17 则进一步从资源利用率角度说明优化收益。

① **图表预留说明：**

- **表 4-8 单卡实验设置表**。列出数据规模、批大小、候选数量、*k* 值、比较方案和硬件配置。
- **图 4-14 1M 数据规模下单卡各阶段耗时分解图**。展示原始链路、仅加速召回、GPU 主导未完全重构和完整优化四种方案在各阶段的耗时变化。
- **图 4-15 10M 数据规模下单卡各阶段耗时分解图**。展示在更大规模数据下各阶段耗时结构如何进一步向设备侧核心阶段收敛。
- **图 4-16 单卡综合对比图**。汇总比较原始链路、召回加速、GPU 主导未重构和完整优化四种方案在 QPS、p99 延迟、PCIe 传输次数与 GPU 利用率上的差异。
- **图 4-17 GPU 利用率与 I/O 利用率变化图**。用于说明单卡优化后资源利用率的改善。

表 4-8　单卡实验设置

数据规模	批大小	候选数量	*k* 值	对比方案名称	硬件环境
1M	2048	128	32	原始链路 仅加速召回 GPU 主导未完全重构 完整优化	占位：1×CPU（32 核） 1×GPU（80 GB） 1×NVMe SSD 256 GB RAM
10M	4096	256	32	原始链路 仅加速召回 GPU 主导未完全重构 完整优化	占位：1×CPU（32 核） 1×GPU（80 GB） 1×NVMe SSD 256 GB RAM

注：表中批大小、候选数量、*k* 值与硬件环境当前为占位配置，后续可按真实实验平台与参数直接替换。

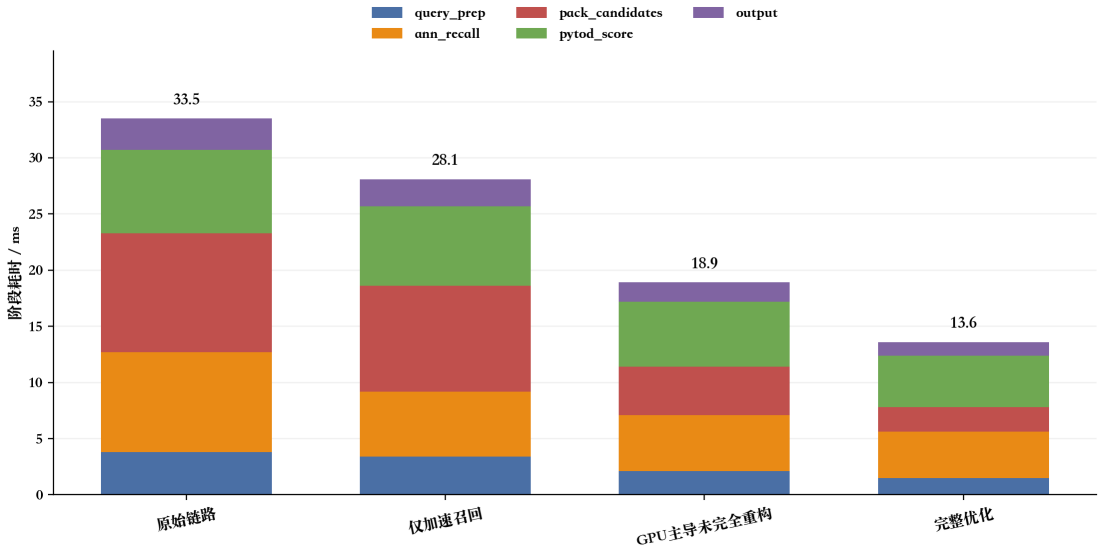

图 4-14　1M 数据规模下单卡各阶段耗时分解

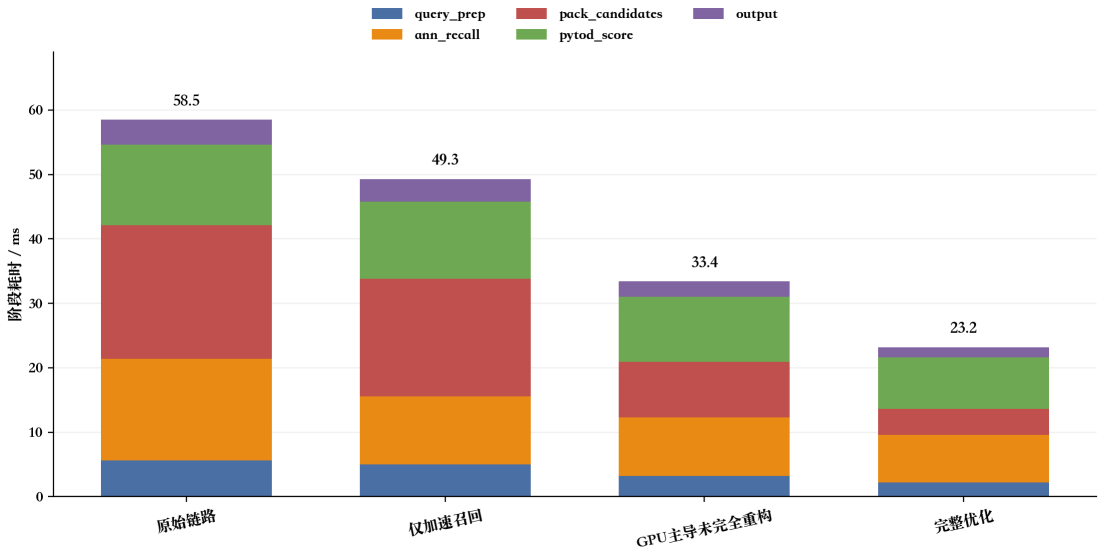

图 4-15　10M 数据规模下单卡各阶段耗时分解

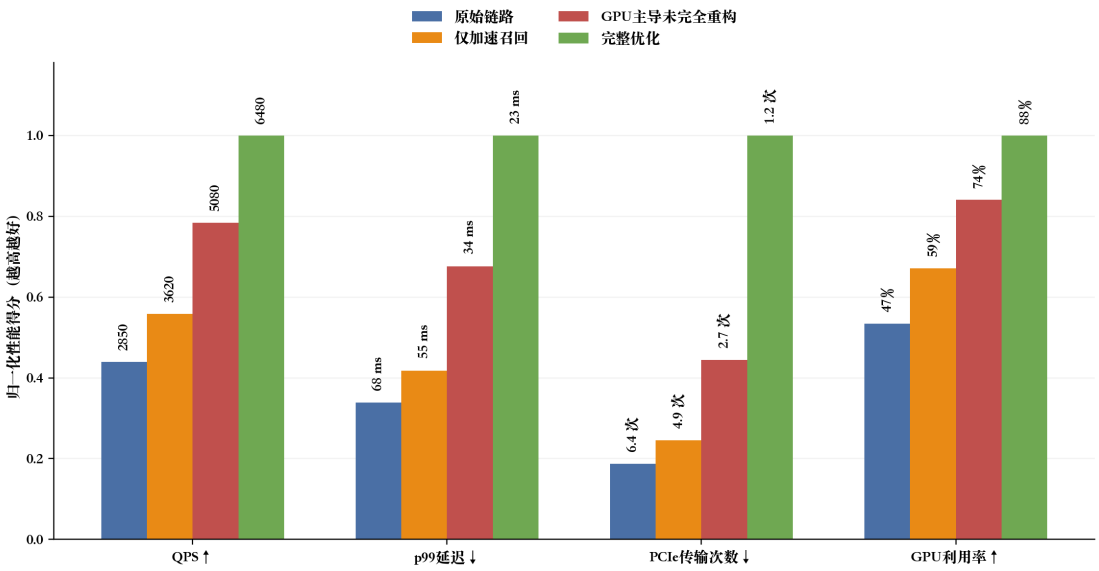

图 4-16　单卡性能综合对比

正式版补齐图 4-17

图 4-17　GPU 利用率与 I/O 利用率变化

4.9　本章小结

本章围绕单 GPU 场景下的系统实现问题，系统分析了原始 PyTOD–FlashANNS 融合链路中存在的 CPU↔GPU 数据往返、候选中间结果主机侧重组、SSD-I/O 串行等待以及资源管理抖动等瓶颈，并在此基础上提出了 GPU 主导的端到端执行链路。该链路在 FlashANNS 异步 I/O—计算流水线的基础上，将查询特征、候选结果与异常评分中间张量尽量保持在 GPU 侧，仅在必要边界通过 pinned memory 等机制与主机交互，从而显著压缩了系统数据路径和阶段同步开销[3-4,7]。

同时，本文进一步引入 RAPIDS 与 FAISS-GPU 对数据处理与候选检索后端进行工程增强，并通过异步 stream、双缓冲、内存池与统一监控机制提升单卡链路的稳定性与可观察性。实验结果表明，单卡系统的主要性能收益并不来自某一个算子的局部优化，而来自“候选召回—异常评分”整条链路的数据路径压缩与

设备侧连续执行。也正因为单卡场景中的瓶颈已经被尽可能收束到 GPU 侧核心阶段，第五章才能在此基础上进一步讨论多 GPU 条件下的数据并行、索引复制、GPU 侧归并与拓扑感知 I/O 控制问题。

第 5 章　多 GPU 扩展与聚合优化

5.1　多 GPU 扩展需求分析

第四章已经从单 GPU 视角完成了系统级优化，并证明在 GPU 主导端到端执行链路、异步 I/O—计算流水线和后端重构协同作用下，单卡场景中的主要瓶颈能够从 CPU↔GPU 数据往返和阶段同步转移到设备侧核心检索与评分过程。然而，当输入规模进一步增大、查询并发继续提升、候选索引规模逼近单卡显存上限，或业务侧要求系统在更高 QPS 下保持近实时响应时，单卡系统仍然会面临两类不可回避的上限：其一是计算吞吐与显存容量上限，其二是候选召回与异常评分在同一设备上竞争 GPU 资源所带来的调度压力。因而，从工程可部署性和业务容量规划角度看，多 GPU 扩展是本文系统继续提升吞吐能力、提升峰值承载能力并拓展可处理索引规模的自然方向[8,15]。

对金融异常检测系统而言，多 GPU 扩展的目标不是单纯追求“更多 GPU 带来更快速度”，而是要在保持检测效果、延迟分位数和系统复杂度可控的前提下，实现更高吞吐率与更稳定资源利用。多 GPU 扩展不能只看理论并行度，还必须同时考察查询如何在设备之间分发、局部结果如何归并、CPU 是否重新进入数据平面、以及共享存储路径是否成为新的上限[8]。

在本文系统中，多 GPU 扩展的需求主要来自三个方面。第一，当候选召回与异常评分都已在 GPU 侧持续运行时，单卡系统的吞吐率最终会受制于单设备的计算资源与显存容量，无法无上限扩展。第二，当使用较大索引或更大规模合成扩展数据集进行实验时，单 GPU 显存可能无法容纳完整索引及评分中间张量，系统必须在“缩小索引规模”和“增加设备数量”之间做出选择。第三，从近实时业务视角看，系统不应只在平均场景下表现良好，还应在高峰期保持稳定延迟与负载均衡，而多 GPU 扩展能够为请求级并行提供更大调度空间[8]。

不过，多 GPU 扩展也会引入新的系统问题。首先，若每个查询在多张 GPU 上执行局部搜索并最终由 CPU 聚合，则 CPU 将重新成为数据平面中心，从而抵消单卡阶段已经获得的数据路径压缩收益。其次，若多个 GPU 共享同一存储路径而缺乏拓扑感知的绑定与准入控制，则 GPU 数量增加并不必然带来线性吞吐增长。GPUDirect Storage 文档指出，GDS 的重要意义在于通过 direct DMA path 避免 CPU bounce buffer，从而降低 CPU 负载、降低延迟并提高带宽；同时，路径选择与拓扑关系会直接影响存储到 GPU 的传输效率[5,11]。

因此，本文在设计多 GPU 扩展时，优先采用数据并行与索引复制的 replicated

主方案，使每张 GPU 尽可能独立完成候选召回与异常评分。只有在索引容量或业务约束明确表明无法复制完整索引时，才进入 GPU 侧跨卡归并与设备间通信的兜底方案。第五章的研究重点不在于构造最复杂的多 GPU 检索系统，而在于在金融近实时异常检测场景下找到**更适合吞吐优先与工程可控性的扩展路径**。

图 5-1 展示了单卡能力边界如何随数据规模和并发需求逼近上限，表 5-1 对扩展动机进行了分类，图 5-2 则说明瓶颈从单卡阶段向多卡聚合与共享 I/O 路径迁移的过程。

① **图表预留说明**：

- **图 5-1 单卡能力上限与多 GPU 扩展需求图**。展示当数据规模、并发量和索引规模增加时，单卡系统的计算、显存和吞吐上限如何逐步逼近。
- **表 5-1 多 GPU 扩展需求分类表**。列出由吞吐需求、显存容量需求、尾延迟控制需求和高峰期负载需求引出的多 GPU 扩展动机。
- **图 5-2 多 GPU 扩展中的瓶颈迁移图**。展示系统瓶颈如何从单卡阶段的 CPU↔GPU 传输，迁移到多 GPU 阶段的结果聚合与共享 I/O 路径。

正式版补齐图 5-1

图 5-1　单卡能力上限与多 GPU 扩展需求

表 5-1　多 GPU 扩展需求分类

正式版补齐表 5-1

正式版补齐图 5-2

图 5-2　多 GPU 扩展中的瓶颈迁移

5.2　多 GPU 扩展策略设计原则

多 GPU 扩展方案的设计必须建立在系统目标与瓶颈分析的基础上。结合第四章单卡结论与第二章相关工作梳理，本文将多 GPU 扩展方案归纳为四项基本设计原则：**查询级并行优于单查询跨卡并行；数据并行与索引复制优先于一开始即采用索引分片；CPU 尽量退出数据平面，仅保留控制面职责；当必须跨卡归并时，归并逻辑应尽量下沉到 GPU 并通过设备间通信完成。**

第一项原则是**查询级并行优于单查询跨卡并行**。若一个查询需要在多张 GPU 上分别执行局部候选召回并在末端进行归并，则系统将额外承担局部结果表达、跨卡通信或 CPU 聚合等开销；若不同查询被均衡分发到不同 GPU 上，则每张 GPU 可以独立完成“候选召回—异常评分”闭环。cuVS 多 GPU 文档将 replicated 模式描述为通过在各 GPU 间分发查询以获得更高 query throughput，这与本文的系统目标一致[8]。

第二项原则是**数据并行与索引复制优先于一开始即采用索引分片**。cuVS 的分布模式定义表明，replicated 模式倾向吞吐优先，而 sharded 模式倾向容量扩展[8-9]。因此，在索引仍可复制条件下，复制完整索引到每张 GPU 并让其独立处理查询，是更适合吞吐场景的方案。

第三项原则是 **CPU 尽量退出数据平面，仅保留控制面职责**。cuVS 文档提示，多 GPU ANN 的部分实现要求数据集、查询与输出数组位于 host memory，这意味着若直接采用 host-centric 默认路径，则 CPU 与主机内存会重新进入关键路径[15]。基于这一认识，本文在多 GPU 方案中将 CPU 定位为**控制面调度者**：其主要职责包括请求队列管理、负载均衡、bounded batching、运行时监控和最终小规模结果汇总。

第四项原则是**当必须跨卡归并时，归并逻辑应下沉到 GPU 侧，并通过 NCCL 等设备间通信机制完成**。NCCL 文档指出其通信操作会异步入队到 CUDA stream

上执行，但也警告在同一 `ncclGroupStart/End()` 中混用多个 stream 会形成全局同步点[10]。因此，本文把 GPU 侧归并视为 replicated 主方案失效时的兜底方案：仅当索引容量或业务约束迫使系统进入 sharded 模式时，才引入 GPU 侧合并与必要的 NCCL 通信组织。

图 5-3 汇总了多 GPU 方案的四项设计原则，表 5-2 对 replicated 主方案、sharded+GPU 归并兜底方案以及 CPU 聚合对比路径的适用条件进行了归纳，图 5-4 则直观比较了两类并行路径。

① **图表预留说明：**

- **图 5-3 多 GPU 扩展策略设计原则图**。概括四项设计原则及其相互关系。
- **表 5-2 多 GPU 扩展方案适用条件表**。列出 replicated、sharded+GPU 归并以及 CPU 聚合对比路径的适用边界。
- **图 5-4 查询级并行与单查询跨卡并行对比图**。展示两种并行思路在结果归并、CPU 参与度和尾延迟上的差异。

正式版补齐图 5-3

图 5-3　多 GPU 扩展策略设计原则

正式版补齐图 5-4

图 5-4　查询级并行与单查询跨卡并行对比

表 5-2　多 GPU 扩展方案适用条件

方案	核心思路	适用条件	优势	代价	定位
replicated	完整索引复制到每张 GPU，按查询级或微批级并行分发任务，每卡独立完成候选召回、异常评分与局部结果输出闭环	单卡可容纳完整索引副本；系统目标以高吞吐和低尾延迟为主；允许用显存换取调度简单性和扩展稳定性	数据平面最清晰；CPU 仅保留控制面职责；几乎不需要跨卡归并；与单卡优化链路复用度最高	显存占用高；索引构建与更新需要维护多份副本；当索引继续增大时可扩展性受显存约束	主方案
sharded+GPU 归并	索引按卡分片，各卡先完成局部召回或局部评分，再通过 NCCL 等通信机制在 GPU 侧完成归并和最终结果合并	完整索引无法复制到单卡；业务必须覆盖更大索引容量；可以接受额外的设备间通信与 GPU 侧归并复杂度	支持更大索引规模；CPU 不再回收大规模候选集合；相较 CPU 聚合路径尾延迟更可控	归并逻辑更复杂；通信开销与 stream 协调成本更高；性能更容易受到拓扑与 I/O 路径限制	兜底方案
CPU 聚合	各 GPU 仅输出局部结果，统一回传 CPU 进行排序、合并与最终输出，CPU 重新承担跨卡聚合职责	主要用于原型验证、调试、小规模实验或需要显式检查中间结果的离线分析场景	实现路径最直接；中间结果易于观察；便于快速构造对比基线	CPU 重新进入数据平面；D2H/H2D 往返明显增加；高并发下 p99 延迟与吞吐表现最差	对比基线

5.3　数据并行与索引复制的多 GPU 执行架构

基于前述设计原则，本文将多 GPU 场景下的主扩展方案定义为**数据并行与索引复制（replicated）**。在该方案中，完整索引被复制到每张 GPU 上，每张 GPU 都具备完整的候选召回与异常评分能力；查询按批次或微批次被分发到不同 GPU，各设备在本地独立执行“查询向量组织—候选召回—异常评分—局部结果输出”全链路流程；CPU 主要承担请求级调度、批次分配和最终小规模结果汇总职责[8]。

在执行流程上，数据并行与索引复制架构可以形式化表示为：给定总查询集合 $Q = \{q_1, q_2, \cdots, q_m\}$，调度器将其划分为若干子批次 $\{B_1, B_2, \cdots, B_g\}$，并将这些子批次分配给 g 张 GPU。每张 GPU G_j 本地维护一份完整索引 $\mathcal{I}_j = \mathcal{I}$，对其收到的批次 B_j 独立执行

$$C_j = \mathcal{R}(B_j, \mathcal{I}_j), \qquad S_j = \mathcal{F}(B_j, C_j), \tag{5-1}$$

其中 $\mathcal{R}$ 表示候选召回，$\mathcal{F}$ 表示异常评分。由于每张 GPU 都能本地完成完整闭环，因此系统不需要在查询处理中途引入设备间依赖。

从系统工程角度看，索引复制虽然会增加显存占用，但换来了更低的执行复杂度和更清晰的数据路径。其主要优势体现在三个方面：查询在设备间彼此独立；CPU 不必处理大规模局部结果聚合；并且每张 GPU 本地执行路径与第四章单卡优化后链路高度一致，系统可复用既有的 GPU 主导执行链路与 I/O—计算重叠组织方式。

图 5-5 给出了 replicated 主方案的总体结构，表 5-3 概括了其资源代价与适

用场景，图 5-6 则强调单卡优化链路在多卡上的复用关系。

① **图表预留说明：**

- **图 5-5 数据并行与索引复制多 GPU 架构图**。展示 CPU 控制面、多个 GPU、本地完整索引副本及其独立执行闭环。
- **表 5-3 replicated 架构的资源与能力分析表**。列出其优点、代价及适用场景。
- **图 5-6 单 GPU 闭环在多 GPU 上复制的示意图**。强调多 GPU 主方案与第四章单卡链路的一致性。

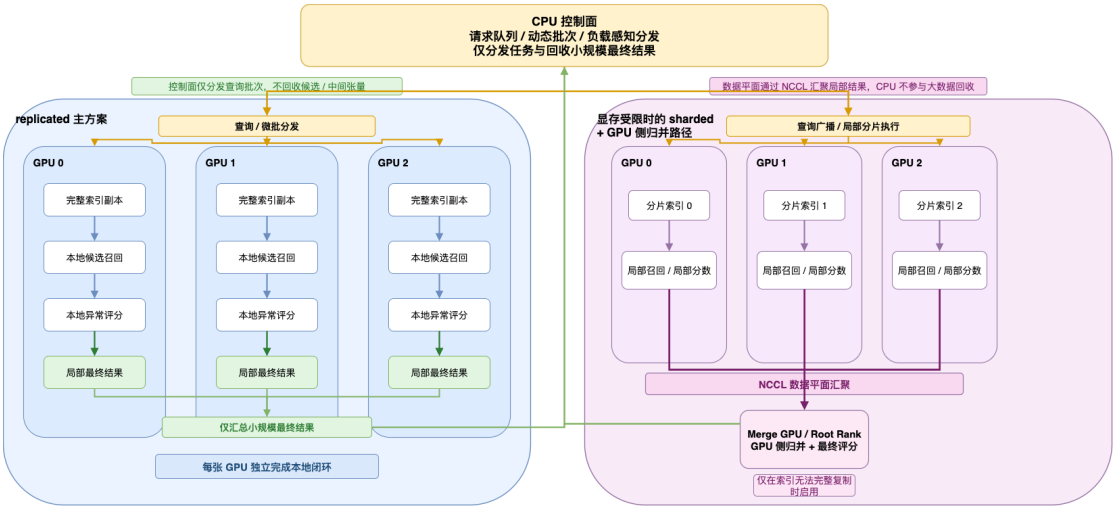

图 5-5　数据并行与索引复制多 GPU 架构

表 5-3　replicated 架构的资源与能力分析

正式版补齐表 5-3

正式版补齐图 5-6

图 5-6　单 GPU 闭环在多 GPU 上复制的示意图

5.4　CPU 控制面调度与负载均衡

在 replicated 架构下，CPU 不再是数据平面中心，而主要承担控制面调度与运行时管理职责：维护请求队列、执行查询批次划分、根据 GPU 负载进行请求分发、控制 bounded batching 策略，并在各 GPU 完成局部完整推理后，将最终小规模结果进行汇总和输出。

请求队列组织方面，系统采用统一入口队列接收查询请求，并根据队列长度、GPU 空闲度和微批策略动态形成批次。cuVS 动态批处理文档指出，动态批处理通过将小规模请求合并成批次，可在维持延迟可控的前提下提高设备占用率和吞吐率[24]。

负载均衡方面，本文在 replicated 架构下优先采用查询级动态调度而非静态平均分配。cuVS 在 replicated 模式下提供了 `LOAD_BALANCER` 与 `ROUND_ROBIN` 两类搜索模式，其中前者用于保持各 GPU 负载均衡[9]。

最终结果汇总方面，CPU 仅接收每张 GPU 已完成的最终小规模结果，如异常分数、Top-k 排序结果和必要日志元信息，而不再接收大规模局部候选集合或评分中间张量。

图 5-7 展示了 CPU 控制面的调度与回收流程，表 5-4 汇总了控制面、数据平面、通信与 I/O 路径中的职责划分及关键参数，图 5-8 则刻画了 bounded batching 与动态负载均衡的工作方式。

① **图表预留说明：**

- **图 5-7 CPU 控制面调度流程图**。展示请求队列、批次形成、GPU 负载监控、请求分发和结果回收路径。
- **表 5-4 控制面/数据面职责与关键参数表**。汇总控制面、数据平面、通信与 I/O 相关组件的职责和关键参数。
- **图 5-8 bounded batching 与动态调度机制示意图**。展示请求如何被聚合为

微批并被动态分配到不同 GPU。

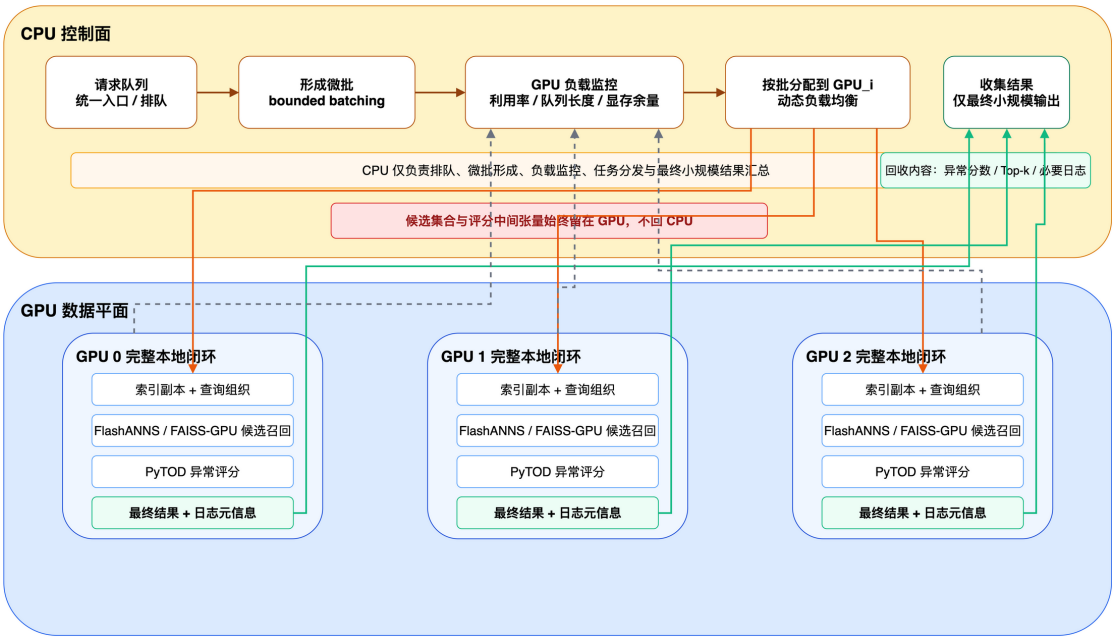

图 5-7　CPU 控制面调度与微批示意

正式版补齐图 5-8

图 5-8　bounded batching 与动态调度机制示意

表 5-4　控制面/数据面职责与关键参数

类别	组件	职责	关键参数	备注
控制面	请求队列与 batching 调度器	统一接收请求、执行 bounded batching、形成微批并决定批次进入时机	`batch_size`、`micro_batch_timeout_ms`、`max_inflight_batches`	直接决定吞吐与平均延迟的折中；CPU 只负责批次组织，不接收大规模候选集合
控制面	GPU 负载均衡器	根据 GPU 空闲度、队列深度和显存水位把微批动态分配到不同设备	`queue_depth_threshold`、`gpu_mem_watermark_pct`、`dispatch_window`	优先保持各卡负载平衡并控制尾延迟；适用于 replicated 主路径
数据平面	FlashANNS / FAISS-GPU 候选召回后端	在设备侧完成向量检索、候选筛选和局部 top-k 组织	`nprobe`、`candidate_k`、`search_stream_count`	每张 GPU 独立执行本地召回；与单卡阶段的 GPU 常驻链路保持一致
数据平面	PyTOD 评分执行器	对候选集合执行张量化异常评分、局部排序和最终分数输出	`score_batch_size`、`k`、`score_stream_priority`	分数、排序结果和必要日志元信息留在设备侧直至最终输出
通信	NCCL 归并与通信流	在 sharded 兜底路径中完成 all-gather / reduce / merge 等设备间结果合并	`nccl_op`、`merge_chunk_size`、`comm_stream_priority`	仅在 replicated 失效时启用；需避免在同一 `ncclGroupStart/End()` 中混用多个 stream
I/O	I/O 队列与 lane 准入控制器	管理 SSD 读队列、GPU–SSD lane 绑定和 in-flight I/O 数量，抑制共享路径拥塞	`io_queue_depth`、`max_inflight_io`、`lane_tokens_per_gpu`	与拓扑感知绑定策略联动；参数过大时会拉高 p99，过小则会限制吞吐

5.5　跨卡归并场景下的 GPU 侧归并与 NCCL 通信

当索引容量超过单卡显存可容纳范围，或业务约束要求单一查询必须访问分布在多张 GPU 上的局部索引时，系统必须进入跨卡归并场景。本文将跨卡归并的原则定义为：**局部结果尽量在 GPU 侧组织与合并，CPU 只接收最终极小规模结果。**

cuVS 在 sharded 模式下给出了 `MERGE_ON_ROOT_RANK` 与 `TREE_MERGE` 两类归并方式，为本文设计 GPU 侧归并路径提供了参考[9]。

在设备间通信实现方面，本文采用 NCCL 作为主要通信机制，并将通信流与局部计算流明确分离。NCCL 文档指出其通信操作异步入队到指定 CUDA stream，但在同一 `ncclGroupStart/End()` 组内混用多个 stream 会形成全局同步点[10]。

图 5-9 展示了 GPU 侧跨卡归并的主要流程，表 5-5 总结了不同触发条件下的处理策略，图 5-10 则说明 NCCL 通信流与计算流的组织差异。

① **图表预留说明：**

- **图 5-9 GPU 侧跨卡归并流程图**。展示局部结果生成、设备间通信、GPU 侧归并和最终小结果回传的全过程。
- **表 5-5 跨卡归并触发条件与处理策略表**。列出触发条件、merge 模式及其适用场景。

- **图 5-10 NCCL 通信流与计算流组织示意图**。展示正确组织与不当组织的差异。

正式版补齐图 5-9

图 5-9　GPU 侧跨卡归并流程

表 5-5　跨卡归并触发条件与处理策略

正式版补齐表 5-5

正式版补齐图 5-10

图 5-10　NCCL 通信流与计算流组织示意

5.6　拓扑感知 I/O lane 绑定与准入控制

在多 GPU 异常检测系统中，扩展效率可能受制于存储路径和 PCIe/NUMA 拓扑。GDS 文档指出，GDS 通过在 GPU memory 与 storage 之间建立 direct DMA

path 避免 CPU bounce buffer，并强调拓扑路径会影响传输效率[5,11]。

本文引入**拓扑感知 I/O lane 绑定与准入控制**机制：运行前分析 GPU、SSD、PCIe root complex、PCIe switch 与 NUMA 节点之间的关系；运行时尽量将 GPU 与 SSD 通道进行 lane 级绑定，并对每条 lane 的 in-flight 请求数做有界控制，以避免过度并发导致尾延迟失控。

图 5-11 以左右对比方式展示了拓扑感知 I/O lane 在合理绑定与共享冲突两种条件下的 GPU、SSD、PCIe 与 NUMA 关系，图 5-12 说明了 I/O lane 绑定与路径冲突的差异，表 5-6 则整理了准入控制所使用的关键参数。

① **图表预留说明：**

- **图 5-11 拓扑感知 I/O lane 示意图。**
- **图 5-12 I/O lane 绑定与共享路径冲突示意图。**
- **表 5-6 I/O lane 准入控制参数表。**

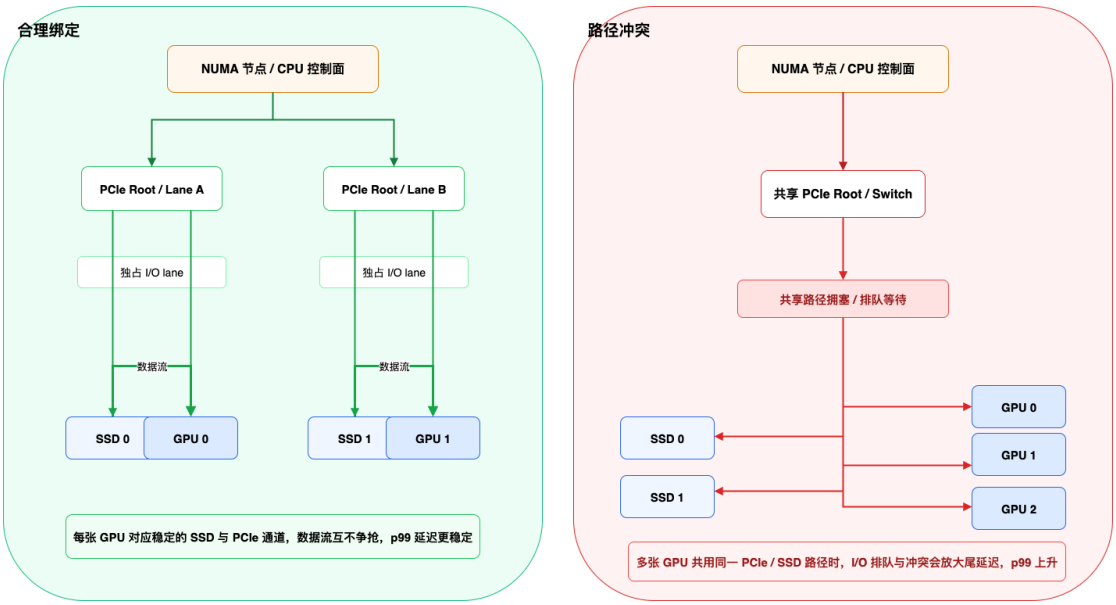

图 5-11　拓扑感知 I/O lane 示意

正式版补齐图 5-12

图 5-12　I/O lane 绑定与共享路径冲突示意

表 5-6　I/O lane 准入控制参数

正式版补齐表 5-6

5.7　多 GPU 工程增强

多 GPU 场景下，系统还需要工程增强机制保证运行稳定性与实验可重复性，包括多进程/多 GPU 运行时组织、统一监控指标、异常恢复与请求重试、以及资源隔离和日志管理。

图 5-13 展示了多 GPU 运行时组织方式，表 5-7 汇总了工程增强项，图 5-14 则概括了运行期监控指标体系。

① **图表预留说明：**

- **图 5-13 多 GPU 运行时组织图。**
- **表 5-7 多 GPU 工程增强项清单。**
- **图 5-14 多 GPU 监控指标体系图。**

正式版补齐图 5-13

图 5-13　多 GPU 运行时组织

表 5-7　多 GPU 工程增强项清单

正式版补齐表 5-7

正式版补齐图 5-14

图 5-14　多 GPU 监控指标体系

5.8　多 GPU 扩展实验与结果分析

本文在 10M 和 100M 规模合成数据集上组织多 GPU 扩展实验，重点验证 replicated 主方案、跨卡归并兜底方案以及 I/O lane 控制策略的效果。实验关注 QPS、平均延迟与 p99 延迟、CPU 占用、GPU 利用率、设备间通信与 I/O 路径占用。

表 5-8 给出了多 GPU 实验设置，图 5-15 与图 5-16 展示扩展曲线与延迟分位数变化，图 5-17 和图 5-18 分别用于比较归并路径和 I/O lane 配置的影响，表 5-9 对总体结果进行了汇总。

① **图表预留说明：**

- **表 5-8 多 GPU 实验设置表。**
- **图 5-15 QPS 随 GPU 数量变化图。**
- **图 5-16 平均延迟与 p99 延迟变化图。**
- **图 5-17 CPU 聚合与 GPU 侧归并对比图。**
- **图 5-18 I/O lane 配置对扩展性的影响图。**
- **表 5-9 多 GPU 扩展结果汇总表。**

表 5-8　多 GPU 实验设置

GPU 数	方案	数据规模	索引组织	批大小	候选数	*k*	归并路径	I/O 绑定	备注
1	Replicated	10M	Full replica	4096	128	50	None	Topology-aware	单卡吞吐扩展基线
2	Replicated	10M	Full replica	4096	128	50	None	Topology-aware	双卡 replicated 扩展基线
4	Replicated	10M	Full replica	4096	128	50	None	Topology-aware	四卡 replicated 扩展基线
4	Local scoring + GPU merge	100M	Sharded index	3072	128	50	NCCL all-gather + GPU merge	Topology-aware	显存受限时的 GPU 侧归并兜底方案
4	Local scoring + CPU aggregation	100M	Sharded index	3072	128	50	CPU aggregation	Topology-aware	CPU 聚合对比基线
2	Topology-aware binding	100M	Full replica	4096	128	50	None	Topology-aware	I/O lane 绑定对比实验
4	Topology-aware binding	100M	Full replica	4096	128	50	None	Topology-aware	I/O lane 绑定对比实验
2	Shared-path conflict	100M	Full replica	4096	128	50	None	Shared PCIe/SSD path	共享路径冲突对比基线
4	Shared-path conflict	100M	Full replica	4096	128	50	None	Shared PCIe/SSD path	共享路径冲突对比基线

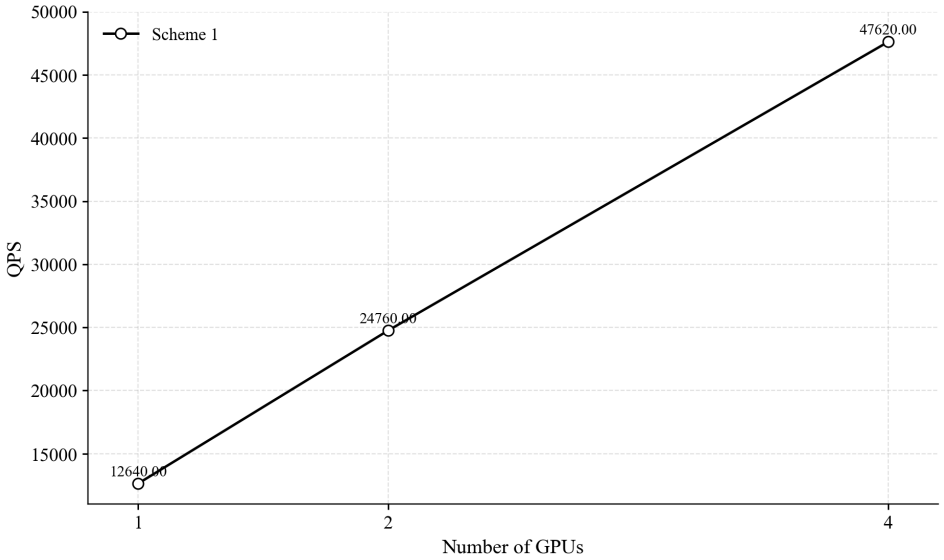

图 5-15　QPS 随 GPU 数量变化

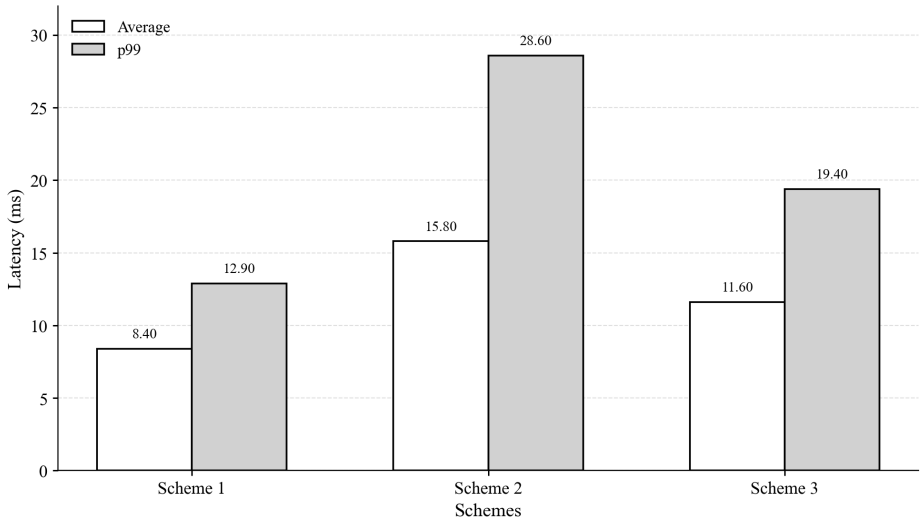

图 5-16　平均延迟与 p99 延迟变化

正式版补齐图 5-17

图 5-17　CPU 聚合与 GPU 侧归并对比

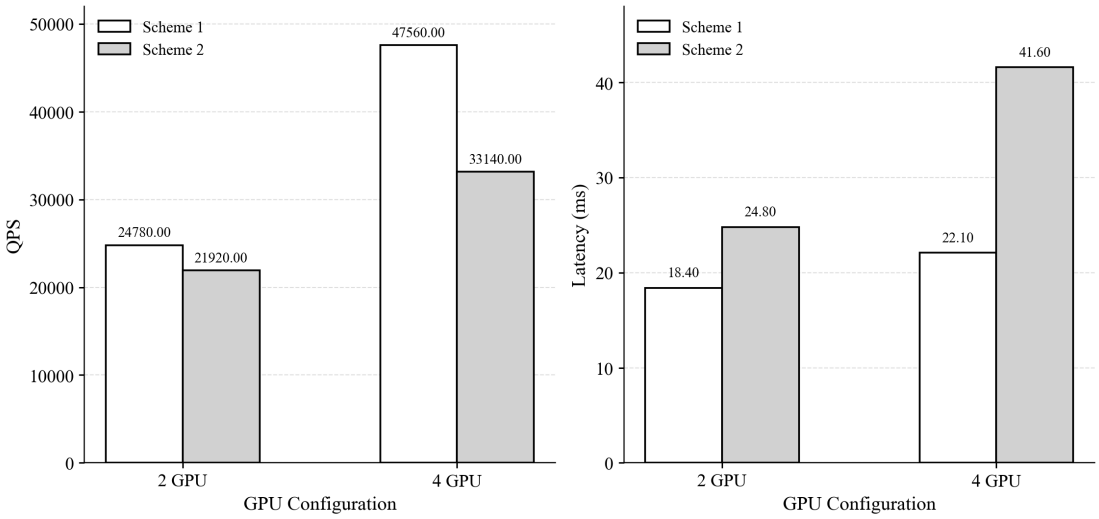

图 5-18　I/O lane 配置对扩展性的影响

表 5-9　多 GPU 扩展结果汇总

GPU 数	方案	数据规模	QPS	avg_latency_ms	p99_latency_ms	CPU 利用率 (%)	GPU 利用率 (%)
1	Replicated	10M	12640.00	10.80	16.20	34.00	69.00
2	Replicated	10M	24760.00	9.10	14.60	30.50	75.50
4	Replicated	10M	47620.00	8.40	12.90	26.50	82.00
4	Local scoring + GPU merge	100M	40180.00	11.60	19.40	32.00	78.00
4	Local scoring + CPU aggregation	100M	28940.00	15.80	28.60	49.00	64.50
2	Topology-aware binding	100M	24780.00	11.40	18.40	31.00	74.00
4	Topology-aware binding	100M	47560.00	13.20	22.10	29.00	80.00
2	Shared-path conflict	100M	21920.00	14.80	24.80	36.00	67.50
4	Shared-path conflict	100M	33140.00	24.20	41.60	43.00	58.00

5.9 本章小结

本章围绕多 GPU 场景下的系统扩展问题，提出并实现了以**数据并行与索引复制**为核心的主方案，并进一步讨论 CPU 控制面调度、GPU 侧跨卡归并、NCCL 通信以及拓扑感知 I/O lane 绑定与准入控制等关键机制。本文将 CPU 定位为控制面调度者，使每张 GPU 尽可能独立完成本地候选召回与异常评分闭环，从而最大程度保留第四章中已经构建的 GPU 主导执行链路优势[8-9]。

本章还表明，多 GPU 扩展并不是简单地“增加更多 GPU”，而是一项涉及查询级并行策略、设备间通信语义和存储拓扑组织的系统工程问题。当 replicated 条件成立时，查询级并行与索引复制是吞吐优先场景的扩展路线；当必须跨卡归并时，GPU 侧归并和 NCCL 通信是优于 CPU 聚合的兜底方案；而无论采用哪种路径，多 GPU 上限都将受到 I/O 路径和拓扑关系的强烈影响[5,10]。

第 6 章　综合实验设计、结果分析与讨论

6.1　实验环境与软硬件配置

为了系统验证本文所提出的 PyTOD–FlashANNS 融合异常检测系统在不同数据规模、不同执行链路和不同 GPU 数量下的有效性与可扩展性，本章从实验环境、数据集设置、评价指标、比较方案和结果分析五个方面展开综合实验。与前三章和第五章中的局部验证不同，本章的目标是站在全文层面，统一回答以下问题：第一，融合架构是否在真实或半真实金融数据上具有实际可用性；第二，单卡场景下 GPU 主导端到端执行链路是否真正降低了 CPU↔GPU 往返与 I/O 等待；第三，多 GPU 场景下数据并行与索引复制方案能否稳定提升吞吐率，并避免 CPU 重新成为聚合瓶颈；第四，拓扑感知 I/O lane 控制和 GPU 侧归并等增强机制在何种条件下有效[2-3]。

实验硬件平台由多张同型号 GPU、主机 CPU、内存、SSD 以及对应 PCIe 拓扑组成。单卡实验主要使用 1 张 GPU 运行完整的候选召回与异常评分链路，用于验证第四章所提出的数据路径优化、异步流组织、RAPIDS 后端重构和 FAISS-GPU 检索增强的效果。多卡实验则使用 2 张及以上 GPU，主要验证第五章提出的数据并行与索引复制方案、控制面调度机制、GPU 侧归并和 I/O lane 控制机制。硬件平台中需明确给出 GPU 型号、显存容量、CPU 型号、主机内存容量、SSD 型号及容量、PCIe 代际和 NUMA 拓扑关系，以保证实验结果的可复现性和后续讨论的可解释性。对于涉及 GDS 或拓扑感知 I/O 的实验，还需额外说明操作系统、驱动版本、CUDA 版本以及存储路径配置[5,11]。

软件环境方面，本文系统由 PyTOD/TOD 评分实现、FlashANNS 候选召回模块、RAPIDS 数据处理组件、FAISS-GPU 检索后端增强组件以及多 GPU 通信和调度运行时共同组成。为了确保实验可复现，需明确列出 Python 版本、PyTorch 版本、PyTOD 版本、RAPIDS 版本、FAISS-GPU 版本、CUDA 版本、NCCL 版本以及操作系统环境。由于不同版本的 RAPIDS、FAISS-GPU 和 cuVS 在接口与性能行为上可能存在差异，本节采用固定的软件组合环境快照进行实验，并在附录或复核记录中提供环境导出信息。对于多 GPU 部分，还需说明是否采用多进程独占 GPU 方式、是否启用 NCCL 后端、是否启用 GDS，以及对应的队列深度和运行时配置项[5,8,10]。

在实现配置上，本文将实验分为三类：**方法有效性验证实验**（1M 规模真实/半真实数据）、**单卡系统性能实验**（端到端吞吐、延迟与资源利用）、以及**多**

GPU 扩展实验（replicated 扩展性、控制面开销、GPU 侧归并与 I/O lane 控制）。三类实验共享一套基础代码框架，但在数据规模、硬件使用方式和评价指标侧重点上存在差异。

表 6-1 和表 6-2 分别给出了硬件与软件环境快照，图 6-1 则补充展示了实验平台中 CPU、GPU、SSD 与 PCIe/NUMA 的连接关系，为后续性能差异解释提供硬件背景。

① **图表预留说明：**

- **表 6-1 实验硬件平台配置表**。包括 GPU 型号与数量、显存容量、CPU 型号、内存容量、SSD 型号与数量、PCIe 版本、NUMA 拓扑等。
- **表 6-2 实验软件环境配置表**。包括 Python、PyTorch、PyTOD、RAPIDS、FAISS-GPU、CUDA、NCCL、操作系统和驱动版本。
- **图 6-1 实验平台拓扑示意图**。展示 CPU、GPU、SSD、PCIe/NUMA 的连接关系，用于解释后续 I/O lane 相关实验。

表 6-1　实验硬件平台配置

项目	规格	说明
CPU 平台	2 × Intel Xeon Gold 6348 @ 2.60GHz	56 核 112 线程，双 NUMA
主存容量	503 GiB	系统总内存
GPU 平台	8 × NVIDIA GeForce RTX 3090	24 GB/卡，总显存 192 GB
PCIe 能力	PCIe Gen4 ×16	8 张 GPU 均为 Gen4 ×16
本地存储	3 × NVMe SSD	894.3 GB × 2，3.5 TB × 1，总约 5.29 TB
NUMA 拓扑	2 个 NUMA 节点	NUMA 0 对应 GPU0–GPU3，NUMA 1 对应 GPU4–GPU7
GPU 互连特征	同侧 PIX/PXB，跨侧 SYS	用于解释双卡与 I/O lane 实验
网络接口	2 × Mellanox NIC	mlx5_0，mlx5_1

表 6-2　实验软件环境配置

项目	版本/配置	说明
操作系统	Ubuntu 20.04.4 LTS，Linux 5.13.0-30-generic	实验运行环境
Python / CUDA	Python 3.11.14；CUDA Runtime 12.4	基础运行时环境
深度学习框架	PyTorch 2.5.0+cu124；NCCL 2.21.5	GPU 张量计算与通信环境
驱动环境	NVIDIA Driver 550.120	与 CUDA 12.4 对应
异常检测框架	PyTOD（yzhao062/pytod）	以 GitHub 主仓库为版本口径更稳妥
ANN 检索组件	FlashANNS / ann_search（Tinkerver/ann_search）	Preview Release；默认分支 main；无正式 release
其他核心库	FAISS 1.13.2；PyOD 2.0.6；NumPy 2.2.6；SciPy 1.13.1；scikit-learn 1.6.1	实验所用主要 Python 依赖
RAPIDS 相关	cuDF / cuML / RMM 未安装	非本轮统一复现实验默认依赖

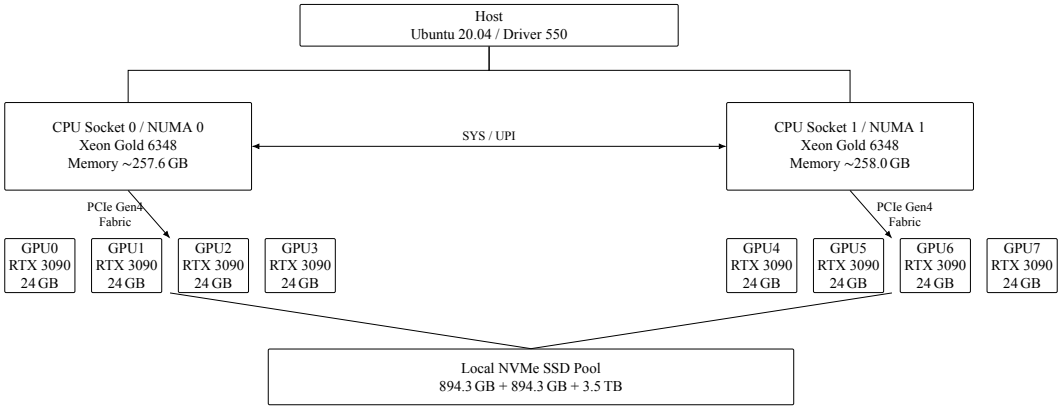

图 6-1　实验平台拓扑示意图

6.2　数据集与任务设置

本章实验采用“真实/半真实数据集用于方法有效性验证，合成扩展数据集用于系统性能与扩展性验证”的总体策略[12-13,20]。公开可获得的真实金融数据通常规模有限，但具有更真实的字段分布、业务噪声和标签结构；而系统吞吐、多 GPU 扩展和高并发 I/O 实验则需要更大规模、分布可控、可重复构造的数据集，因此采用高保真合成数据作为大规模实验基座。

在 1M 规模有效性验证中，本文选取三类代表性数据集：**IEEE-CIS Fraud Detection**（结构化支付风控数据)[12]、**Credit Card Fraud Detection**（经典信用卡欺诈基准)[22]，以及 **DGraphFin**（金融图数据集)[14,19]。为保证一致性，方法有效性实验控制输入规模在约 1M 左右。

在 10M 与 100M 规模的性能实验中，本文采用基于 **IBM AML / AMLSim** 构建的高保真合成金融交易数据[13,20]。其中，10M 数据集主要用于单卡系统优化的综合性能评估，100M 数据集用于多 GPU 扩展实验及 I/O 路径压力测试。

从任务组织上看，本文实验被划分为四类：**候选召回与异常评分有效性任务**、**单卡性能任务**、**多 GPU replicated 扩展任务**以及**跨卡归并与 I/O 路径任务**。各任务使用相同的核心代码框架，但数据规模、运行方式和指标重点不同。

在数据划分方面，有效性验证任务按训练集、验证集和测试集划分；性能与扩展实验采用固定索引集与固定查询集方式，将查询批量持续注入系统，以更准确地衡量吞吐率、延迟与资源利用率。

表 6-3 汇总了数据集与实验任务的对应关系，图 6-2 展示了规模层级与实验类型的映射，表 6-4 则进一步说明了不同实验口径下的数据使用方式。

① **图表预留说明：**

- **表 6-3 数据集与实验任务映射表**。列出各数据集的来源、规模、特征维度、标签形式以及在本文中的用途。

- **图 6-2 数据规模层级与实验类型关系图**。展示 1M、10M、100M 三个层级分别对应的方法有效性、单卡性能和多 GPU 扩展实验。
- **表 6-4 训练集、验证集和测试集划分说明表**。说明有效性实验与性能实验在数据使用方式上的差异。

表 6-3　数据集与实验任务映射

数据集	来源类型	规模层级	标签形式	本文用途
IEEE-CIS Fraud Detection	真实/半真实公开金融表格数据	真实集	二分类标签	方法有效性主验证；单卡、双卡真实数据口径基准
IBM AML / AML-Sim	高保真合成金融交易数据	1M	异常交易标签	统一小规模桥接实验；连接真实集与扩展实验
IBM AML / AML-Sim	高保真合成金融交易数据	10M	异常交易标签	单卡与双卡性能对比的共同起点
IBM AML / AML-Sim	高保真合成金融交易数据	50M	异常交易标签	单卡中大规模性能验证；双卡扩展对比
IBM AML / AML-Sim	高保真合成金融交易数据	100M	异常交易标签	双卡扩展与压力测试

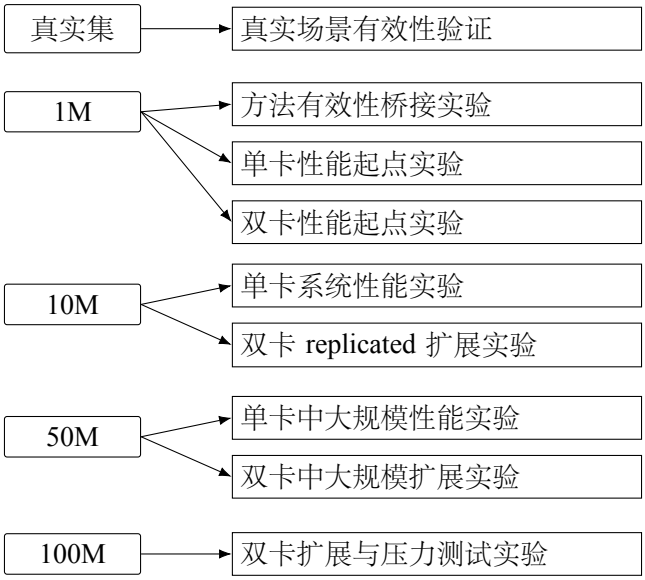

图 6-2　数据规模层级与实验类型关系图

表 6-4　训练集、验证集和测试集划分说明

实验类型	数据规模	数据使用方式	划分方式	主要用途	说明
方法有效性验证	真实集	训练集 / 验证集 / 测试集	固定随机种子划分	评估 PR-AUC、Recall@k	用于真实数据上效果验证
方法有效性桥接实验	1M	训练集 / 验证集 / 测试集	与真实集一致的固定划分方式	评估 PR-AUC、Recall@k、Top-*k* overlap	连接真实与合成扩
单卡性能实验	10M、50M	固定索引集 + 固定查询集	不再进行 train/val/test 划分	评估 QPS、p99 Latency、GPU Utilization	强调端到与稳定性
双卡扩展实验	10M、50M、100M	固定索引集 + 固定查询集	不再进行 train/val/test 划分	评估 QPS、p99 Latency、CPU/GPU Utilization	强调扩展控制面负

6.3　评价指标与比较原则

为全面评价本文系统的有效性与工程价值，本章采用多维度指标体系，包括异常检测效果指标、候选检索质量指标、系统吞吐与延迟指标以及资源利用率指标[1,8]。

异常检测效果方面，采用 ROC-AUC、PR-AUC 与 Recall@k；候选检索质量方面，采用 Recall@k / Top-*k* overlap；系统性能方面，采用 QPS、平均延迟与 p95/p99 延迟；资源利用率方面，采用 GPU 利用率、CPU 占用率、显存使用率、I/O 带宽利用率以及 CPU↔GPU 传输开销占比。

比较原则方面，本文坚持三项要求：尽量共享相同数据集、相同向量表示和相同硬件配置；比较不同召回配置时控制候选集合规模或召回质量处于同一数量级；多 GPU 实验中采用相同软件环境与相近查询到达方式，确保差异主要来自扩展策略本身。

表 6-5 对全文使用的评价指标进行了统一定义，图 6-3 给出了指标体系结构，表 6-6 则明确了比较原则与控制变量。

① **图表预留说明：**

- **表 6-5 实验评价指标定义表。**
- **图 6-3 指标体系结构图。**
- **表 6-6 比较原则与实验控制变量表。**

表 6-5　实验评价指标定义

指标类别	指标名称	定义	适用实验	说明
异常检测效果	PR-AUC	Precision–Recall 曲线下面积	真实集、1M	金融异常检测样本极不 PR-AUC 比 ROC-AUC 更
异常检测效果	Recall@*k*	前 *k* 个高风险样本中的真实异常覆盖率	真实集、1M	对应业务审查预算
候选检索质量	Top-*k* overlap	近似排序结果与基线排序结果前 *k* 项重合度	1M、10M、50M、100M	说明候选召回没有明显 终排序
系统性能	QPS	每秒处理查询数	全部规模	端到端吞吐核心指标
系统性能	p99 Latency	99 分位端到端延迟	全部规模	反映尾延迟与系统稳定性
资源利用率	GPU Utilization	GPU 平均利用率	单卡、双卡	验证 GPU 主导链路是否 续工作
资源利用率	CPU Utilization	CPU 平均占用率	双卡重点、单卡辅助	验证 CPU 是否仍主要处 面

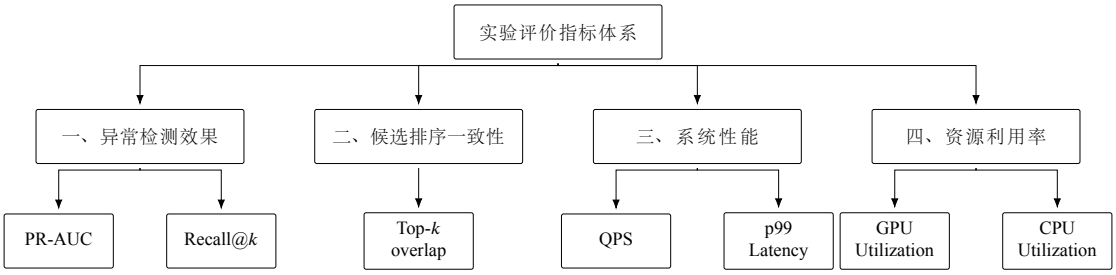

图 6-3　指标体系结构图

表 6-6　比较原则与实验控制变量

比较维度	比较原则	主要控制变量	说明
数据一致性	相同实验类型共享相同数据源与标签口径	dataset、label definition	防止不同数据口径导致指标不可比
划分一致性	有效性实验共享相同 train/val/test 划分	split seed、train/val/test ratio	保证 PR-AUC、Recall@k 可比
查询负载一致性	性能实验共享相同索引集与查询集	index set、query set、query count	保证 QPS 与 p99 Latency 可比
候选预算一致性	不同方法使用相同候选预算	topk、rerank_budget	保证 Top-*k* overlap 与性能比较公平
批处理一致性	不同方案控制相同 batch 组织策略	batch_size、bounded_batch_size	避免批大小掩盖系统差异
硬件一致性	同类比较在相同 GPU/CPU/SSD 条件下进行	gpu_model、gpu_count、cpu、ssd	保证单卡与双卡实验口径清晰
软件环境一致性	固定软件栈版本	CUDA、PyTorch、FAISS-GPU、RAPIDS、NCCL	保证复现实验结果
单卡/双卡口径一致性	相同规模下分别比较单卡与双卡	gpu_count、replica_mode	避免不同架构混比
统计一致性	统一重复轮次与汇总方式	round、seed、mean	每组 3 轮取均值

6.4　核心结果展示

本节从总体效果、单卡优化效果、多 GPU 扩展效果以及不同数据规模下的系统表现四个层面展示实验结果。

6.4.1　融合系统与基线系统的总体对比

在 1M 规模真实/半真实数据上，比较 PyTOD–FlashANNS 融合系统与若干基线系统：仅使用 PyTOD 全量评分、仅使用候选召回结果作为风险判断依据、以及未引入 GPU 常驻链路与后端重构的初始融合实现。结果需同时体现检测效果与系统性能，突出“候选召回压缩评分范围 + 候选集上张量化评分”在效果与效率间的平衡价值[2-3]。

6.4.2　单卡优化前后的总体对比

对比第四章单卡优化前后的端到端表现：原始分段链路、仅启用候选召回加速、启用 GPU 常驻但未完成后端重构、以及完整单卡优化方案。重点体现 CPU↔GPU 往返次数与 I/O 等待对端到端收益的影响[3-4,7]。

6.4.3　多 GPU 扩展结果对比

在 10M 与 50M 合成数据上，展示 replicated 主方案下 1/2/4 GPU 的吞吐扩展曲线、延迟分位数与 CPU 占用变化；并在必要时对比跨卡归并场景下 CPU 聚合与 GPU 侧归并的差异[8,10]。

6.4.4　不同数据规模下的系统表现对比

比较 1M、10M、50M 三个规模层级下系统的瓶颈迁移与收益变化，统一呈现“方法有效性”与“系统可扩展性”两条主线。

为便于综合展示，表 6-7 汇总了核心结果，图 6-4、图 6-5、图 6-6 和图 6-7 分别对应总体对比、单卡优化、多 GPU 扩展以及跨规模表现四个观察视角。

① **图表预留说明：**

- **表 6-7 核心结果汇总表。**
- **图 6-4 融合系统与基线系统总体对比图。**
- **图 6-5 单卡优化前后对比图。**
- **图 6-6 多 GPU 扩展曲线图。**
- **图 6-7 不同规模下系统表现对比图。**

表 6-7　核心结果汇总

正式版补齐表 6-7

正式版补齐图 6-4

图 6-4　融合系统与基线系统总体对比

正式版补齐图 6-5

图 6-5　单卡优化前后对比

正式版补齐图 6-6

图 6-6　多 GPU 扩展曲线

正式版补齐图 6-7

图 6-7　不同规模下系统表现对比

6.5　结果讨论

结果讨论围绕四个问题展开：融合架构是否有效、数据路径优化是否真正改变瓶颈、多 GPU 主方案是否合理、以及存储拓扑与归并机制在何种条件下重要。

第一，若融合系统在 1M 规模真实/半真实数据上维持合理的 ROC-AUC、PR-AUC、Recall@k 与 Top-*k* 风险样本覆盖能力，则说明候选召回压缩评分范围并未破坏异常评分主干判别能力。

第二，若单卡实验表明查询特征、候选结果与评分中间张量尽量保持在 GPU 侧后，CPU↔GPU 传输次数与体积明显下降，且 QPS 与延迟分位数同时改善，则说明 GPU 主导执行链路的核心结论成立：系统瓶颈主要来自阶段边界的数据搬运与同步，而非某一单点算子不够快[4,7]。

第三，若多 GPU replicated 架构表现出稳定吞吐增长且 CPU 占用未随之同幅增长，则说明控制面/数据平面分离的设计有效；若必须跨卡归并时 GPU 侧归并优于 CPU 聚合，则说明 “CPU 退出数据平面” 同样是多 GPU 扩展中的关键前提[8,10]。

第四，若扩展曲线在更高 GPU 数量下开始放缓，而 I/O lane 控制实验显示

共享存储路径成为主要限制，则说明“多 GPU 上限最终受制于存储路径组织”的判断成立[5,11]。

图 6-8 总结了本节讨论所遵循的因果链条，表 6-8 则把关键发现与其对应实验依据进行了对应整理。

① **图表预留说明：**

- **图 6-8 结果讨论逻辑图。**
- **表 6-8 关键发现与对应证据表。**

正式版补齐图 6-8

图 6-8　结果讨论逻辑图

表 6-8　关键发现与对应证据

正式版补齐表 6-8

6.6　局限性分析

尽管本文在融合系统、单卡数据路径优化与多 GPU 扩展方面取得系统性结果，但仍存在局限性。

首先，真实超大规模金融数据公开可得性有限，10M 与 100M 规模性能实验主要依赖 IBM AML / AMLSim 等合成数据[13,20]。其次，replicated 架构受显存容量限制，索引规模进一步增大时必须转向 sharded 路径并承担更高通信与归并复杂度。第三，out-of-core 路径与拓扑感知 I/O 优化对部署条件依赖较强，其收益

与 GPU–SSD–PCIe–NUMA 拓扑、驱动与 GDS 支持高度相关[5,11]。第四，本文尚未延伸到多机分布式环境。

表 6-9 汇总了主要局限性及其影响范围，图 6-9 则用边界示意图说明了本文结论更适用的场景范围。

① **图表预留说明：**

- **表 6-9 研究局限性与影响范围表。**
- **图 6-9 本文结论适用边界示意图。**

表 6-9　研究局限性与影响范围

正式版补齐表 6-9

正式版补齐图 6-9

图 6-9　本文结论适用边界示意图

6.7　本章小结

本章从实验环境、数据集与任务设置、评价指标、核心结果展示、结果讨论与局限性分析六个方面，对本文所提出的 PyTOD–FlashANNS 融合异常检测系统进行了统一实验验证。实验结果表明，两阶段融合架构能够在保持合理检测效果的前提下显著提升系统整体吞吐率；其中，GPU 主导端到端执行链路降低了 CPU↔GPU 数据搬运与阶段同步开销，多 GPU replicated 扩展进一步提升吞吐，而 GPU 侧归并与拓扑感知 I/O 控制在更复杂场景下补足扩展能力[3,5,8]。

第 7 章　全文总结与展望

7.1　全文总结

本文围绕金融大规模数据异常检测系统的实现与优化问题展开研究，针对传统异常检测系统在大规模场景下面临的异常评分复杂度高、候选召回吞吐不足、CPU 与 GPU 之间数据搬运频繁以及多 GPU 扩展效率受限等问题，提出并实现了一套基于并行计算的 PyTOD–FlashANNS 融合异常检测系统。全文研究重点不在于提出新的异常检测理论模型，而在于将候选召回、异常评分、数据路径优化与多 GPU 扩展统一纳入同一系统框架，并验证其在金融近实时异常检测场景中的工程可行性[2-3,8]。

首先，在系统总体架构方面，本文提出了以 PyTOD 为异常评分主干、以 FlashANNS 为候选召回模块的两阶段融合框架。本文将二者统一组织到一条候选召回—异常评分执行链路中，使系统能够先在大规模样本中快速召回候选集合，再在候选集合上执行张量化异常评分，从而在保证检测效果可接受的前提下，显著降低全量异常评分所带来的时间与空间开销[2-3]。

其次，在数据准备与任务建模方面，本文从金融交易数据、账户行为数据和图金融数据出发，对输入形式、向量化表示、候选召回目标和异常评分目标进行了统一建模，并形成了“真实/半真实数据用于有效性验证，高保真合成数据用于大规模系统实验”的数据组织方案。通过引入 IEEE-CIS Fraud Detection、Credit Card Fraud Detection、DGraphFin 以及 IBM AML/AMLSim 等数据来源，本文构建了 1M、10M 和 100M 三个规模层级的实验体系，使论文能够同时覆盖方法可用性验证和系统吞吐扩展验证两类目标[12-14,20,22]。

再次，在单 GPU 场景下，本文提出了 GPU 主导的端到端执行链路，将查询特征、候选结果与异常评分中间张量尽量保持在 GPU 侧，仅在必要边界通过 pinned memory 与主机交互，从而压缩系统中的非必要数据传输和阶段同步开销[3-4,7]。与此同时，本文进一步利用 RAPIDS 对表格型数据处理和中间结果组织进行后端重构，并借助 FAISS-GPU 对候选检索后端进行工程增强，使系统在单卡场景下从“模块可运行”演进为“链路低开销、执行更连续”的高吞吐实现。

进一步地，在多 GPU 场景下，本文以吞吐优先为目标，提出了数据并行与索引复制的主扩展方案，使每张 GPU 均持有完整索引并独立完成本地候选召回与异常评分，CPU 主要承担控制面调度与最终小规模结果汇总职责[8-9]。在此基础上，本文讨论了索引复制不可行时的兜底路径：通过 GPU 侧局部结果组织、

NCCL 设备间通信和 GPU 侧最终归并，使 CPU 尽量继续退出数据平面[10]。同时，本文引入拓扑感知的 I/O lane 绑定与准入控制机制，以缓解多 GPU 条件下的路径冲突和尾延迟放大问题[5,11]。

最后，在综合实验与结果分析方面，本文从异常检测效果、候选召回质量、单卡端到端吞吐、多 GPU 扩展能力、CPU 占用、GPU 利用率、I/O 路径占用和延迟分位数等多个维度对系统进行了统一评估。实验结果表明，融合架构在 1M 规模真实/半真实数据上能够保持合理的检测效果；在 10M 和 100M 规模高保真合成数据上，单卡 GPU 主导执行链路能够显著减少 CPU↔GPU 往返和 I/O 等待，而多 GPU 数据并行与索引复制方案能够较稳定地支撑系统吞吐扩展[3,8,13]。

综合来看，本文的主要研究结论可以归纳为三点：

1. PyTOD 与 FlashANNS 的融合能够有效打通高吞吐候选召回与高效异常评分之间的链路，使系统从全量高复杂度执行转向候选缩减后的精排评分路径。
2. 单 GPU 场景下的系统瓶颈主要来自数据路径与阶段同步，而不是单个算子本身；通过 GPU 主导端到端执行链路、RAPIDS 后端重构和 FAISS-GPU 工程增强，可以显著提升端到端性能。
3. 多 GPU 场景下更适合采用数据并行与索引复制作为吞吐优先主方案，只有在索引无法复制时才进入 GPU 侧跨卡归并与设备间通信路径；同时，多 GPU 上限与存储拓扑和 I/O 路径组织密切相关，因此拓扑感知 I/O 控制应被视为系统设计的重要组成部分。

表 7-1 对全文主要工作与章节贡献进行了对应汇总，图 7-1 则从技术路线和结论闭环两个角度概括了本文的整体逻辑。

① **图表预留说明：**

- **表 7-1 全文主要工作与对应章节总结表**。将全文的主要研究工作、核心章节和实验验证结果进行一一对应。
- **图 7-1 全文技术路线与结论闭环图**。展示从问题提出、融合系统构建、单卡优化、多 GPU 扩展到综合实验验证的完整闭环。

表 7-1　全文主要工作与对应章节总结

正式版补齐表 7-1

正式版补齐图 7-1

图 7-1　全文技术路线与结论闭环图

7.2　未来工作展望

尽管本文围绕金融大规模异常检测系统的实现与优化进行了较为系统的研究，并在 PyTOD–FlashANNS 融合架构、单卡数据路径压缩和多 GPU 扩展方面取得了一定成果，但从更长期的工程部署与学术研究角度看，仍有若干值得进一步深入的方向。

7.2.1　面向更真实金融场景的数据验证

未来工作需要进一步争取接入更接近真实生产环境的金融业务数据，在严格遵守隐私保护与合规要求的前提下，验证系统在更复杂分布、更强业务噪声和更显著概念漂移条件下的表现。

7.2.2　面向多机场景的分布式扩展

未来可考虑将本文提出的“控制面与数据平面分离”思想扩展到多节点环境，在更大规模集群中研究查询分发、跨节点索引一致性、远程结果归并和跨节点通信调度问题。

7.2.3　面向更深层存储路径的优化

未来可以进一步引入 GPUDirect Storage、RDMA 等技术，探索更少主机参与、更低拷贝代价的存储访问路径，并在查询调度与存储访问之间建立更细粒度的协同策略。

7.2.4　面向图异常检测与时序异常检测的统一扩展

未来可在保持系统总体架构不变的前提下，引入图嵌入、图神经网络、时序编码或基于事件流的特征聚合机制，使图异常检测与时序异常检测在不牺牲系统吞吐的前提下融入现有候选召回—异常评分框架。

7.2.5　面向国产 GPU 平台的验证与部署

未来还应关注本文方案在国产 GPU 平台上的验证与部署问题，围绕算子支持、向量检索实现、设备间通信机制与存储接口适配，研究如何构建等价或近似等价的工程实现路径。

参考文献

[1] Aggarwal C C. Outlier analysis[M]. 2nd ed. Springer, 2017.

[2] Zhao Y, Chen G, Jia Z. TOD: GPU-accelerated outlier detection via tensor operations[J]. Proceedings of the VLDB Endowment, 2022.

[3] Xiao Y, Sun M, Song Z, et al. Breaking the storage-compute bottleneck in billion-scale ANNS: a GPU-driven asynchronous I/O framework[Z]. 2025.

[4] RAPIDS Developers. RMM documentation: PinnedHostMemoryResource[Z]. 2026.

[5] NVIDIA. GPUDirect storage documentation: overview guide[Z]. 2024.

[6] RAPIDS Developers. cuDF documentation: how it works / cudf.pandas[Z]. 2026.

[7] FAISS Contributors. Faiss on the GPU[Z]. 2024.

[8] NVIDIA. cuVS documentation: multi-GPU nearest neighbors[Z]. 2026.

[9] NVIDIA. cuVS documentation: distribution mode / search mode / merge mode (Multi-GPU ANN)[Z]. 2026.

[10] NVIDIA. NCCL documentation: CUDA stream semantics[Z]. 2024.

[11] NVIDIA. GPUDirect storage documentation: design guide[Z]. 2024.

[12] IEEE-CIS. Fraud detection (IEEE-CIS) dataset[Z]. 2019.

[13] IBM Research. IBM AML-data: synthetic data for anti-money laundering research[Z]. 2022.

[14] DGraph Team. DGraphFin_baseline (GitHub Repository)[Z]. 2022.

[15] NVIDIA. cuVS documentation: multi-GPU IVF-PQ / IVF-flat (Host memory Requirement) [Z]. 2026.

[16] Breunig M M, Kriegel H P, Ng R T, et al. LOF: identifying density-based local outliers[C]. Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data. 2000.

[17] Angiulli F, Pizzuti C. Fast outlier detection in high dimensional spaces[C]. Proceedings of the 6th European Conference on Principles of Data Mining and Knowledge Discovery (PKDD). 2002.

[18] Johnson J, Douze M, Jégou H. Billion-scale similarity search with GPUs[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2019.

[19] Liu Z, Xiao Z, Li L, et al. DGraph: a large-scale financial dataset for graph anomaly detection [A/OL]. arXiv (2022). https://arxiv.org/abs/2207.03579.

[20] IBM Research. AMLSim: a synthetic financial transaction generator for anti-money laundering research[Z]. 2021.

[21] Machine Learning Group – ULB, Kaggle. Credit card fraud detection (Kaggle Dataset)[Z].

2013.

[22] Dal Pozzolo A, Caelen O, Johnson R A, et al. Calibrating probability with undersampling for unbalanced classification[C]. 2015 IEEE Symposium Series on Computational Intelligence (SSCI). 2015.

[23] DGraph Team. DGraphFin dataset (official website)[Z]. 2022.

[24] NVIDIA. cuVS documentation: dynamic batching[Z]. 2026.

[25] 本文项目组. 统一脚本、参数配置、日志模板与送审前复核记录[Z]. 2026.

附录 A　实验脚本与日志模板

本附录在正式版中应补充关键脚本入口、命令行参数模板、日志字段样例与目录组织方式，以支撑“统一脚本、统一日志、统一口径”的研究方法。相关脚本与复核记录可与内部材料统一维护和版本化管理[25]。

A.1　待补充内容

- 关键训练、建索引、查询与评测脚本入口。
- 常用命令行参数模板与默认值说明。
- 日志字段示例及字段解释。
- 工程目录组织方式与结果归档规范。

附录 B　图表补充说明

本附录应集中说明正文各图表的变量命名、单位、比较口径与补图要求，确保图表在送审版和答辩版中保持一致。

B.1　待补充内容

- 图表中的变量名称、量纲和取值范围。
- 不同图表之间共享指标的统一定义。
- 同口径比较、跨阶段比较与跨后端比较的边界说明。
- 送审版与答辩版图表的共用与差异化补充规则。

附录 C　缩写、变量与配置对照

本附录列出正文中频繁出现的关键变量、配置项及其含义，便于后续统一实验脚本和结果复核。

变量或配置项	含义
`batch_size`	单次提交到候选召回或精排路径中的批处理大小。
`rerank_budget`	进入高精度精排阶段的候选数量预算。
`nlist`	索引聚类桶数量或粗排分区规模参数。
`nprobe`	查询阶段实际访问的粗排桶数量。
`device_id`	当前任务绑定的 GPU 设备标识。
`replica_mode`	多 GPU 复制式多实例执行模式开关。
`bounded_batch_size`	在时延约束下允许形成的微批上限。

致谢

本版本以论文结构与技术内容定稿为主，致谢内容可在正式送审前根据实际情况补充。